RUHR-UNIVERSITÄT BOCHUM

RUB

PHD THESIS

Optimizing Industrial Processes through Reinforcement Learning

submitted by

Abhijeet Pendyala

For the Degree of **Doktor-Ingenieur**
from the Faculty of Computer Science
at Ruhr Universität Bochum

September 5, 2025.

1st Supervisor: Prof. Dr.Tobias Glasmachers
2nd Supervisor: Prof. Dr.Laurenz Wiskott

Abstract

Optimizing complex industrial processes, particularly those involving resource allocation under uncertainty and strict constraints, presents significant challenges for traditional control methods. This thesis addresses the optimization of container management in a real-world, high-throughput plastic sorting facility, a critical stage impacting sustainability. The core task involves scheduling the emptying of multiple containers, which accumulate different materials at stochastic rates, into a limited number of shared Processing Units (PUs). This process is complicated by factors inherent to industrial settings: stochastic material inflow, noisy sensor readings, significant delays between actions and rewards, safety constraints preventing container overflow, and the need to balance competing objectives such as maximizing throughput, minimizing energy consumption, and ensuring operational safety.

Initial investigations revealed that standard Reinforcement Learning (RL) algorithms (e.g., PPO, TRPO, DQN), while promising, struggle to learn effective policies directly due to these complexities. The challenges of sparse rewards, the need for long-term planning under uncertainty, and the difficulty of managing shared resource contention often lead to sub-optimal or unsafe behaviors. To establish a clear basis for research, the problem was first formalized and released as 'ContainerGym', an open-source RL benchmark environment derived directly from the industrial use case, capturing its inherent difficulties.

To overcome the limitations of baseline RL methods, this research proposes and evaluates a progressively sophisticated set of techniques centered around Proximal Policy Optimization (PPO). First, a multi-stage Curriculum Learning (CL) strategy combined with meticulous reward engineering was developed. This approach systematically guides the agent's learning process, starting with simplified environmental dynamics and gradually introducing complexity, stochasticity, resource constraints, and refined reward signals. This CL methodology (PPO-CL) proved effective in enabling the agent to handle delayed rewards, learn basic safety protocols, manage energy considerations, and achieve high performance, significantly outperforming naive PPO training.

However, even with CL, managing "collision" situations—where multiple containers require the scarce PU simultaneously, leading to potential overflows—remained a significant challenge. Purely model-free RL approaches struggle to anticipate and proactively mitigate these multi-container interactions effectively. To address this, a novel hybrid approach was developed, augmenting the PPO-CL agent with an inference-time planning mechanism. An offline-trained Collision Model (CM),

built using Monte Carlo simulations and an XGBoost classifier, predicts the likelihood of imminent collisions. At inference time, if the PPO-CL agent proposes a "do-nothing" action and a high collision risk is detected, the system overrides the agent's inaction and forces a preemptive empty, creating the PPO-CL-CM agent.

Comprehensive experimental evaluations across various container-to-PU ratios (from 7:1 to 12:1) demonstrate the efficacy of this hybrid strategy. The PPO-CL-CM agent significantly reduces the frequency of collision events and safety-limit violations compared to the PPO-CL agent, while maintaining or improving overall throughput. The analysis provides quantitative insights into the scalability of the system, offering practical guidelines for facility design regarding the optimal number of containers that can be managed by a single PU.

This thesis contributes a validated RL benchmark for industrial resource allocation, a robust curriculum learning framework for tackling complex real-world RL problems with delayed rewards and safety constraints, and a novel hybrid RL-planning approach for effective collision avoidance in resource-constrained systems. The findings demonstrate the potential of combining structured learning techniques with domain-specific predictive models to develop safe, efficient, and scalable control solutions for challenging industrial control problems.

Zusammenfassung

Die Optimierung komplexer industrieller Prozesse, insbesondere solcher, die eine Ressourcenallokation unter Unsicherheit und strengen Randbedingungen erfordern, stellt traditionelle Steuerungs- und Regelungsmethoden vor erhebliche Herausforderungen. Diese Dissertation befasst sich mit der Optimierung des Containermanagements in einer realen Hochdurchsatz-Kunststoffsortieranlage, einer kritischen Prozessstufe mit Auswirkungen auf die Nachhaltigkeit. Die Kernaufgabe besteht in der Planung der Entleerung mehrerer Container, in denen sich unterschiedliche Materialien mit stochastischen Raten ansammeln, in eine begrenzte Anzahl gemeinsam genutzter Verarbeitungseinheiten Processing Units (PUs). Dieser Prozess wird durch inhärente Faktoren industrieller Umgebungen erschwert: stochastischer Materialzufluss, verrauschte Sensordaten, erhebliche Verzögerungen zwischen Aktionen und Belohnungen, Sicherheitsbeschränkungen zur Verhinderung von Containerüberläufen und die Notwendigkeit, konkurrierende Ziele wie Durchsatzmaximierung, Energieverbrauchsminimierung und Gewährleistung der Betriebssicherheit auszubalancieren.

Erste Untersuchungen zeigten, dass Standardalgorithmen des Verstärkenden Lernens (Reinforcement Learning, RL) (z.B. PPO, TRPO, DQN) aufgrund dieser Komplexität Schwierigkeiten haben, direkt effektive Policies zu erlernen. Die Herausforderungen durch seltene (sparse) Belohnungen, die Notwendigkeit langfristiger Planung unter Unsicherheit und die Schwierigkeit der Bewältigung von Konflikten um gemeinsam genutzte Ressourcen führen oft zu suboptimalem oder unsicherem Verhalten. Um eine klare Forschungsgrundlage zu schaffen, wurde das Problem zunächst formalisiert und als ContainerGym veröffentlicht – eine Open-Source-RL-Benchmark-Umgebung, die direkt aus dem industriellen Anwendungsfall abgeleitet wurde und dessen inhärente Schwierigkeiten abbildet.

Um die Einschränkungen von Basis-RL-Methoden zu überwinden, schlägt diese Arbeit eine Reihe progressiv anspruchsvollerer Techniken vor, die auf der Proximal Policy Optimization (PPO) basieren, und bewertet diese. Zunächst wurde eine mehrstufige Curriculum-Learning-Strategie (CL) in Kombination mit sorgfältigem Reward Engineering (Entwurf von Belohnungsfunktionen) entwickelt. Dieser Ansatz leitet den Lernprozess des Agenten systematisch an, beginnend mit vereinfachten Umgebungsdynamiken und der schrittweisen Einführung von Komplexität, Stochastizität, Ressourcenbeschränkungen und verfeinerten Belohnungssignalen. Diese CL-Methodik (PPO-CL) erwies sich als effektiv, um den Agenten in die Lage zu versetzen, mit verzögerten Belohnungen umzugehen, grundlegende Sicherheitsprotokolle

zu erlernen, Energieaspekte zu berücksichtigen und eine hohe Leistung zu erzielen, die das naive PPO-Training deutlich übertrifft.

Jedoch blieb auch mit CL die Handhabung von "Kollisionssituationen" – in denen mehrere Container gleichzeitig die knappe PU benötigen, was zu potenziellen Überläufen führt – eine wesentliche Herausforderung. Rein modellfreie RL-Ansätze haben Schwierigkeiten, diese Multi-Container-Interaktionen effektiv zu antizipieren und proaktiv zu entschärfen. Um dies zu adressieren, wurde ein neuartiger hybrider Ansatz entwickelt, der den PPO-CL-Agenten um einen Inferenzzeit-Planungsmechanismus erweitert. Ein offline trainiertes Kollisionsmodell (Collision Model, CM), erstellt mittels Monte-Carlo-Simulationen und einem XGBoost-Klassifikator, sagt die Wahrscheinlichkeit bevorstehender Kollisionen voraus. Wenn der PPO-CL-Agent zur Inferenzzeit eine „Nichtstun“-Aktion vorschlägt und ein hohes Kollisionsrisiko erkannt wird, setzt das System die Untätigkeit des Agenten außer Kraft und erzwingt eine präventive Entleerung. Diese Architektur wird als PPO-CL-CM-Agent bezeichnet.

Umfassende experimentelle Evaluationen über verschiedene Container-zu-PU-Verhältnisse (von 7:1 bis 12:1) demonstrieren die Wirksamkeit dieser hybriden Strategie. Der PPO-CL-CM-Agent reduziert die Häufigkeit von Kollisionsereignissen und Sicherheitsgrenzverletzungen im Vergleich zum PPO-CL-Agenten signifikant, während der Gesamtdurchsatz beibehalten oder verbessert wird. Die Analyse liefert quantitative Einblicke in die Skalierbarkeit des Systems und bietet praktische Richtlinien für das Anlagendesign hinsichtlich der optimalen Anzahl von Containern, die von einer einzelnen PU verwaltet werden können.

Diese Dissertation trägt einen validierten RL-Benchmark für industrielle Ressourcenallokation bei, ein robustes Curriculum-Learning-Framework zur Bewältigung komplexer realweltlicher RL-Probleme mit verzögerten Belohnungen und Sicherheitsbeschränkungen, sowie einen neuartigen hybriden RL-Planungsansatz zur effektiven Kollisionsvermeidung in ressourcenbeschränkten Systemen. Die Ergebnisse demonstrieren das Potenzial der Kombination strukturierter Lerntechniken mit domänenspezifischen Vorhersagemodellen zur Entwicklung sicherer, effizienter und skalierbarer Steuerungslösungen für anspruchsvolle industrielle Regelungsprobleme.

Official Declaration

I hereby declare that I have not submitted this thesis in this or similar form for any other examination at the Ruhr-Universität Bochum or any other institution of higher education.

I officially affirm that this thesis has been written solely by myself. I affirm that I have not used any other sources but those stated by me. Any and all parts of the text which constitute quotes in original wording or in their essence have been explicitly referred by me by using official marking and proper citation. This is also valid for used drafts, pictures, and similar formats.

Declaration on the Use of Artificial Intelligence Tools

I acknowledge the use of Artificial Intelligence (AI) based tools, including Large Language Models (LLMs), during the preparation of this thesis. These tools were utilized for the following purposes: assisting with LaTeX formatting and implementation, improving grammar and linguistic expression, and supporting deep research into related literature. I affirm that the intellectual content, the core research contributions, the interpretation of results, and the conclusions drawn are entirely my own. I have critically reviewed and verified the output generated by these tools and assume full responsibility for the accuracy and integrity of this thesis.

Acknowledgements

First and foremost, I wish to express my deepest gratitude to my supervisor, Prof. Dr. Tobias Glasmachers. Your mentorship has been invaluable, extending back to my Master's thesis. Thank you for believing in my potential and giving me the crucial opportunity to transition from Computational Science into the fascinating field of Machine Learning. Your insightful feedback, unwavering support, and intellectual rigor have helped me grow as a person and a researcher. I feel truly fortunate to have spent the early stages of my career working with the best person and boss I have ever known.

I am also sincerely grateful to Prof. Dr. Laurenz Wiskott for agreeing to serve as my second supervisor. Thank you for your time, valuable perspectives, and constructive comments during this process.

I would like to extend my thanks to Dr. Asma Atamna. Your guidance and collaboration during the initial phases of my PhD were instrumental in defining the direction of this research and navigating the early challenges.

A PhD requires a supportive intellectual community, and I was lucky to find one within the Theory of Machine Learning group. I want to thank my friends and intellectual sparring partners—Nils Müller, Simon Hakenes, Tom Maus, Stephan Frank, Dr. Nico Zengeler, Alexander Jungeilges, and Justin Dettmer—for the stimulating discussions and the camaraderie that made the long hours enjoyable.

I also wish to thank my friend and BITS college junior, Harisyam Manda, who first introduced me to the concepts of Machine Learning and sparked the curiosity that ultimately led me down this path.

My heartfelt thanks go to my family, whose love and support have been the foundation of my achievements. To my father, a lifelong teacher, thank you for instilling in me the value of education and perseverance. To my mother, thank you for your unwavering confidence in me, which always encouraged me to aim higher. And to my younger brother, thank you for your constant support and encouragement. I am deeply proud and humbled to be the first doctorate in our extended family.

Finally, and most importantly, I want to thank my wife, Sai. Your immense support, patience, understanding, and love have been my anchor throughout this challenging journey. This achievement is as much yours as it is mine.

Contents

List of Figures

List of Tables

1 Motivation and Background

1.1 Intelligent Industrial Optimization

Modern industries operate within an increasingly complex landscape, driven by demands for higher efficiency, stricter environmental regulations, cost reduction, and enhanced sustainability. Optimizing industrial processes is no longer merely a desirable goal but a critical necessity for competitiveness and compliance. However, many real-world industrial systems—from manufacturing lines and energy grids to logistics networks—are characterized by intricate dynamics, inherent uncertainties, and competing operational objectives, making optimization a formidable challenge.

Traditional control methods, often relying on predefined rules, heuristics, or classical optimization techniques based on simplified mathematical models, frequently fall short in such complex environments. They struggle to adapt dynamically to inherent stochasticity (e.g., sensor noise), handle high-dimensional state spaces, manage resource contention effectively, or optimize strategies when feedback (rewards or penalties) is significantly delayed.

1.1.1 The Role of Reinforcement Learning

Reinforcement Learning (RL) has emerged as a powerful paradigm for addressing sequential decision-making problems in these complex and uncertain environments [32]. Unlike traditional methods, RL agents learn optimal strategies, or policies, through direct interaction with their environment. This trial-and-error learning process allows RL agents to discover complex control policies without explicit programming of the optimal behavior, making them well-suited for tasks where system dynamics are noisy, partially unknown, or too complex to model accurately. RL's ability to learn adaptive strategies based on experience holds significant promise for optimizing dynamic industrial processes.

1.2 The Industrial Context: High-Throughput Plastic Sorting

A critical application area for advanced optimization lies within the waste management sector. This research is motivated by the urgent global need to transition towards a circular economy, driven by environmental concerns and stringent regulatory mandates (such as the German Packaging Act (VerpackG) and broader EU directives on packaging waste [1]). Realizing these goals requires the development of highly efficient, high-throughput waste sorting facilities.

1.2.1 Sutco RecyclingTechnik Facilities

This thesis addresses a real-world optimization challenge derived from the operations of Sutco RecyclingTechnik GmbH, a leading global manufacturer of sorting and treatment systems. Sutco designs and constructs large-scale facilities essential for processing complex plastic streams, such as Lightweight Packaging (LVP).[1]

The scale of these operations is substantial. For example, the reference plant in Gernsheim, Germany, is designed to process 100,000 tons of LVP waste annually. The primary objective is to transform this high volume of heterogeneous input into high-purity, marketable fractions (e.g., PET, HDPE, aluminum, film) with purity levels often exceeding 95%.

1.2.2 The Anatomy of a Modern Sorting Facility

To achieve this throughput and purity, the facilities employ a complex, multi-stage sequence of technologies:

1. **Pre-treatment and Mechanical Separation:** Initial processing involves bag opening, followed by separation based on size and shape using technologies like Trommel Screens (screening drums) and Ballistic Separators.
2. **Sensor-Based Sorting:** The core of the process relies on advanced sensor technology. Near-Infrared (NIR) detectors identify different material types on high-speed conveyors, while magnets and eddy current separators extract metals.
3. **Storage and Baling:** The purified fractions are transported to storage containers before final processing.

[1] Throughout this thesis, the terms "container" and "bunker" are used interchangeably to refer to the storage units within the sorting facility.

Figure 1.1: Overview of a typical high-throughput plastic sorting facility. The process involves multiple stages of separation and sorting, culminating in the container management system. (Source: [30])

The complexity is evident in the sheer volume of equipment required; the Gernsheim plant, for instance, utilizes 4 screening drums, 3 ballistic separators, and 18 NIR units [31].

1.2.3 The Critical Bottleneck: Bunker Management

At the final stage, the sorted material fractions accumulate in dedicated storage bunkers (containers). This stage, known as **Bunker Management**, represents a critical bottleneck.

The bunkers serve as a buffer before the materials are compressed into bales by a Processing Unit (PU), commonly a baling press. Crucially, the architecture is characterized by a significant resource constraint: there are substantially more bunkers than PUs. A typical configuration might involve 10 to 20 bunkers feeding into only one or two shared PUs via a conveyor system. An illustrative example of such a configuration is shown in Figure 1.2.

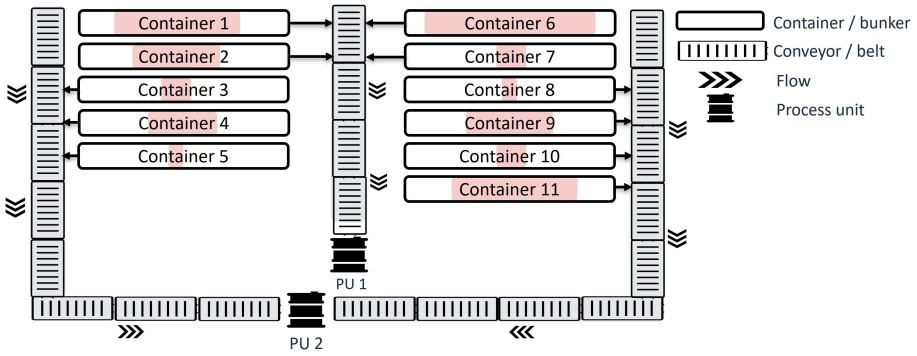


Figure 1.2: Illustrative layout sketch of a representative bunker management system architecture (Example: 11 containers and 2 Processing Units). Containers are filled from above (stochastic inflow) and emptied onto a shared conveyor belt system leading to the PUs. The current fill states are indicated by the shaded areas. This visualization highlights the core resource constraint addressed in this thesis.

1.2.4 The Control Challenge

The core operational task is to decide precisely when to empty each bunker (as depicted in Figure 1.2). This decision is highly complex due to the inherent dynamics and interactions within the system:

- **Stochastic and Heterogeneous Inflow:** The rate at which materials arrive is highly stochastic, depending on the variable composition of the input waste. Furthermore, different materials have vastly different densities (e.g., low-density film fills a container rapidly, while aluminum fills slowly), requiring distinct management strategies.
- **Resource Contention and Scheduling:** The PU is a scarce, shared resource. If multiple containers require emptying simultaneously, a sophisticated scheduling decision must be made. The time taken to process a container temporarily blocks the PU from servicing others.
- **Optimal Bale Quality:** To maximize efficiency and market value, the bales produced must meet specific quality standards regarding weight and density. This implies an optimal volume at which a container should be emptied. Emptying too early results in underweight bales and energy inefficiency; emptying too late risks overflows.
- **Hard Safety Constraints:** Allowing a container to overflow is unacceptable, as it requires manual intervention and causes significant facility downtime.

While automated bunker management systems exist, they typically rely on simplified prioritization rules and predefined thresholds based on current fill levels. These heuristic methods are often tuned for a specific facility layout (e.g., the Gernsheim plant) and struggle to adapt to dynamic fluctuations in inflow. Crucially, they fail to generalize effectively across different plant configurations (e.g., varying ratios of containers to PUs) and lack the foresight to anticipate future contention for the PU in a globally optimal manner.

The research in this thesis aims to surpass the capabilities of these existing systems by developing a **generalizable methodology** based on RL. The goal is to create an approach capable of handling the stochasticity and complex interactions robustly and adaptively, applicable to a wide range of facility configurations within this domain.

1.3 Problem Statement

This thesis addresses the optimization of the container (bunker) management process described above. The central challenge is not merely to optimize a single facility configuration, but rather to develop a **generalizable methodology** capable of producing intelligent control policies for arbitrary configurations of containers (N) and processing units (M). This methodology must dynamically manage the emptying schedule, navigating the trade-offs between throughput, efficiency, and safety under conditions of stochasticity, severe resource constraints, and complex interaction effects.

The specific factors contributing to the complexity of this control problem, which are formalized and addressed progressively throughout this research, include:

1. **Stochasticity and Uncertainty:** Material inflow rates are stochastic and influenced by upstream processes. Furthermore, sensor readings for container volumes are subject to noise. Control policies must be robust to this uncertainty.
2. **Severe Resource Constraints (The Bottleneck):** The scarcity of PUs (e.g., 1 or 2 PUs for 7-12 containers) creates a critical bottleneck. The PUs are shared resources, necessitating effective scheduling and coordination to manage contention.
3. **Optimal Emptying Volumes and Precision Control:** Each container has one or more ideal volumes at which emptying yields the best results (e.g., optimal bale quality or throughput). Emptying significantly below or above these volumes is sub-optimal. Later stages of the research explicitly address a **dual-peak scenario**, where a higher volume offers better throughput but carries more risk, while a lower volume acts as a safer fallback. Learning to target these volumes precisely is a key objective.

4. **Hard Safety Constraints:** The requirement to prevent container overflows imposes a strict safety constraint that must be respected to avoid costly downtime.
5. **Competing Objectives:** The system must simultaneously optimize for multiple, often conflicting, goals: maximizing material throughput, minimizing energy usage (potentially favoring fewer, larger empties), ensuring PU availability, and strictly adhering to safety constraints.
6. **Scalability and Generalizability:** As the number of containers increases, the state space grows. The competition for the shared PU means the optimal action for one container is highly dependent on the state of all others. This interdependence makes coordinated decision-making difficult.
7. **Delayed Rewards and Long Horizons:** The consequences of an action (or inaction) may not be realized until much later. For example, delaying the emptying of one container might seem benign locally but could lead to an unavoidable overflow of another container hundreds of timesteps later due to subsequent PU unavailability. This requires long-horizon planning capabilities.

Ultimately, an effective policy must learn to anticipate future states and manage PU contention to prevent overflows. It must do so while navigating the complex trade-offs between competing objectives—such as throughput, efficiency, and safety—and adapting its strategy as the research focus shifts from baseline performance to multi-criteria optimization and finally to robust collision avoidance.

1.4 Research Questions and Objectives

Based on the challenges outlined above, this thesis investigates the application and advancement of reinforcement learning techniques for optimizing the container management process. The research is structured progressively to address the following key questions, corresponding to the three core contributions:

Phase 1: Formalization and Benchmarking (Chapter 7)

- **RQ1 (Modeling):** How can the complex, stochastic, and resource-constrained container management problem be effectively modeled as a reinforcement learning environment suitable for rigorous investigation and benchmarking?
- **RQ2 (Baseline Performance):** How do standard, state-of-the-art RL algorithms perform on this real-world task, and what are their primary limitations when faced with its inherent challenges (e.g., delayed rewards, safety constraints, resource scarcity)?

Phase 2: Advanced RL for Multi-Criteria Control (Chapter 8)

- **RQ3 (Curriculum Learning and Reward Engineering):** Can a structured training methodology, combining Curriculum Learning and tailored Reward Engineering, effectively overcome the limitations of standard RL and enable an agent (specifically PPO) to learn a robust policy that balances the competing objectives of volume optimization, energy efficiency, and safety in a complex, realistic configuration?

Phase 3: Hybrid Architectures and Scalability (Chapter 9)

- **RQ4 (Collision Avoidance and Foresight):** How can the critical issue of Processing Unit (PU) collisions be effectively addressed, particularly in high-contention scenarios (e.g., single PU) requiring long-term foresight?
- **RQ5 (Hybrid RL and Planning):** Can integrating the RL policy with planning techniques, specifically using an inference-time collision prediction model, enhance safety and collision avoidance compared to a purely reactive RL approach?
- **RQ6 (Scalability and Design Insights):** What practical insights regarding facility design and operational limits (e.g., the viable ratio of containers to PUs) can be derived from evaluating these advanced control strategies across varying system configurations?

1.5 Key Contributions

This thesis makes several key contributions to the field of reinforcement learning and its application to industrial process optimization, developed progressively through the research stages:

1. **A Real-World RL Benchmark Environment (ContainerGym) [Chapter 7]:** We introduce and validate ContainerGym, a novel, open-source RL benchmark environment derived directly from the industrial resource allocation task. It accurately captures key challenges prevalent in industrial applications, including stochastic dynamics, resource scarcity, delayed rewards, and the need for safe operation, providing a valuable tool for the research community [20].
2. **Curriculum Learning Framework for Complex Industrial Tasks [Chapter 8]:** We develop and demonstrate the effectiveness of a structured, multi-stage curriculum learning (CL) strategy tailored for the container management problem. Combined with carefully engineered reward functions, this approach (PPO-CL) enables a PPO agent to successfully learn a complex, multi-criteria

control policy despite sparse rewards and long time horizons, significantly outperforming naive RL training [18].

3. **Hybrid RL and Planning Architecture for Collision Avoidance [Chapter 9]:** We propose and validate a novel hybrid method (PPO-CL-CM) that integrates the curriculum-trained RL agent with an efficient, inference-time planning component (a learned Collision Model). This architecture proactively identifies and overrides potentially risky actions, leading to significantly improved collision avoidance and safety in high-contention scenarios [19].
4. **Empirical Insights for Facility Design and Scalability [Chapter 9]:** Through systematic evaluation of the proposed control strategies across varying container-to-PU ratios, this research provides empirical evidence on system scalability. The results offer actionable insights and quantitative data that can inform practical decisions regarding facility design, specifically addressing how many containers can be effectively managed by a single processing unit [19].

1.6 Thesis Outline

The remainder of this thesis is structured into two main parts:

Part I: Foundations and Methodologies This part establishes the theoretical background and the advanced RL techniques utilized in this research.

- **Chapter 2 (Reinforcement Learning and Planning):** Provides foundational knowledge on RL, including MDPs, the Bellman equations, Dynamic Programming, and model-free methods. It also introduces the taxonomy unifying RL and Planning.
- **Chapter 3 (Policy Gradient Methods and PPO):** Details the theory behind policy gradient methods, the Actor-Critic architecture, and provides an in-depth look at Proximal Policy Optimization (PPO), the core algorithm used in this thesis.
- **Chapter 4 (Reward Engineering):** Discusses the principles and challenges of designing effective reward functions, including PBRS, heuristic shaping, and the mitigation of reward hacking.
- **Chapter 5 (Curriculum Learning):** Explores methodologies for structuring the RL training process, focusing on task sequencing and stable transfer protocols to solve complex tasks.
- **Chapter 6 (Hybrid Control Architectures):** Investigates the integration of RL with planning techniques, focusing on the use of learned predictive models and inference-time intervention for safety and foresight.

Part II: Core Contributions and Publications This part presents the core research contributions through three peer-reviewed publications.

- **Chapter 7 (Benchmark Environment):** Introduces ContainerGym and presents the benchmarking of baseline RL algorithms (Addresses RQ1, RQ2; Contribution 1).
- **Chapter 8 (Optimizing Container Management with CL):** Details the development and evaluation of the PPO-CL methodology for multi-criteria control (Addresses RQ3; Contribution 2).
- **Chapter 9 (Collision Avoidance using Hybrid RL and Planning):** Introduces the PPO-CL-CM hybrid architecture and analyzes system scalability and collision avoidance (Addresses RQ4, RQ5, RQ6; Contributions 3, 4).

Conclusion

- **Chapter 10 (Conclusion and Future Work):** Summarizes the main findings and contributions of the thesis, discusses limitations, and outlines directions for future research.

Theoretical Foundations

2 Reinforcement Learning and Planning

This chapter establishes the theoretical foundations for the core problem of this thesis: optimal sequential decision-making under uncertainty. Historically, this problem has been approached from two principal paradigms: **Planning**, originating from classical artificial intelligence and control theory, and **Reinforcement Learning** (RL), a subfield of machine learning. While these fields have distinct origins and methodologies, they are fundamentally concerned with optimizing behavior in the same formal structure: the Markov Decision Process (MDP) [32, 14]. This chapter moves beyond a simple summary to deconstruct these fields into a unified set of principles, thereby building the theoretical framework for the methods developed in this thesis.

We begin by formalizing the problem domain using the MDP framework. Next, we introduce a critical distinction based on an algorithm's *access to the environment's dynamics* and the *scope of its computed solution*, providing a clear lens to differentiate planning, model-free RL, and model-based RL [14]. From there, we delve into the core mechanics of the field—the Bellman equations and the framework of Generalized Policy Iteration (GPI)—which organize most solution methods. The chapter culminates in a re-contextualized taxonomy of algorithms, establishing the theoretical basis for the advanced ***hybrid*** methods developed later in this work.

2.1 The Optimal Control Problem

The fundamental challenge addressed by both reinforcement learning and planning is that of optimal control: determining a sequence of actions for a decision-making agent interacting with a dynamic environment to optimize a cumulative objective. At each discrete timestep t, the agent exists in a state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$. The environment responds by transitioning to a new state s_{t+1} and providing a scalar reward signal, $r_{t+1} \in \mathbb{R}$. This interaction generates a trajectory, or history, $H_t = (s_0, a_0, r_1, s_1, a_1, \ldots, a_{t-1}, r_t, s_t)$. The agent's goal is to construct a strategy, or policy, that maximizes a long-term function of the sequence of rewards.

A foundational assumption that renders this problem tractable is that the state signal s_t is a sufficient statistic of the history, a condition known as the Markov property.

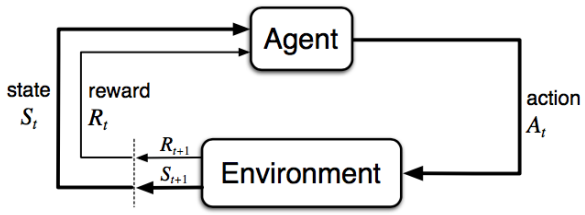


Figure 2.1: The agent-environment interaction loop in Reinforcement Learning. The agent selects an action A_t based on the state S_t. The environment responds with a reward R_{t+1} and a new state S_{t+1}.

This allows the problem to be modeled as a Markov Decision Process, which provides the formal language for the remainder of this work.

2.2 The Markov Decision Process

2.2.1 Markov Property

A state representation s_t is said to possess the **Markov Property** if the future is conditionally independent of the past, given the present state and action.

Definition 2.2.1 (The Markov Property)**.** *A state s_t is defined as Markov if and only if:*

$$\Pr(S_{t+1} = s', R_{t+1} = r | S_t, A_t) = \Pr(S_{t+1} = s', R_{t+1} = r | H_t, A_t) \tag{2.1}$$

In essence, this equation states that the probability of the next state and reward depends only on the current state and action, not on the entire history of preceding states and actions.

When this property holds, the history can be discarded without loss of information relevant to future decision-making. A system satisfying this property can be modeled as a Markov Decision Process (MDP), formally a 5-tuple $\langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$:

- $\mathcal{S}$ is the set of states. For formal simplicity, we consider finite sets, though these concepts extend to continuous state spaces where function approximation is required.
- $\mathcal{A}$ is the set of actions. $\mathcal{A}(s)$ denotes the set of actions available in state $s \in \mathcal{S}$.
- $p : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability function, where $p(s'|s, a) \doteq \Pr(S_{t+1} = s' | S_t = s, A_t = a)$.

- $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, specifying the expected immediate reward: $r(s, a) \doteq \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$.
- $\gamma \in [0, 1]$ is the discount factor.

2.2.2 Planning vs. RL

A critical distinction between algorithms that solve MDPs lies in the nature of their access to the environment's dynamics, $p(s'|s, a)$. Following the framework proposed by Moerland et al. [14], we can classify this access along two primary axes, which allows for a precise differentiation between Planning and RL.

Access to the Environment Model. This dimension concerns the ability to query the dynamics.

- **Reversible Access:** The algorithm possesses a **model** of the environment, which it can query for any state-action pair at will. This is the domain of classical **Planning**.
- **Irreversible Access:** The algorithm has no prior model and must learn from direct interaction by sampling transitions. This is the domain of **Model-Free RL**.

Scope of the Computed Solution. This dimension describes the nature of the computed solution.

- **Local Solution:** The algorithm computes a temporary solution valid for a small subset of the state space, such as a search tree constructed to find the best action from the current state. This is characteristic of **Planning**.
- **Global Solution:** The algorithm computes and permanently stores a solution—such as a value function or policy—that is valid for the entire state space. This is the hallmark of **Reinforcement Learning**.

This leads to a more rigorous taxonomy, which is essential for situating the hybrid methods developed in this thesis.

Definition 2.2.2 (Algorithmic Taxonomy based on Model and Solution Scope)**.**

- ***Planning*** *refers to a class of algorithms that require a model of the MDP (reversible access) and compute a temporary, local solution.*
- ***Reinforcement Learning*** *refers to a class of algorithms that compute a permanent, global solution. This family is further divided into:*
  - ***Model-Free RL****, which operates without a model (irreversible access).*
  - ***Model-Based RL****, which utilizes a model (reversible access) to help compute its global solution.*

2.2.3 Objective Function and the Role of Discounting

The agent's behavior is specified by a **policy**, π, which represents its strategy. A policy maps states to a probability distribution over actions, $\pi(a|s) = \Pr(A_t = a|S_t = s)$, for a *stochastic* policy, or provides a direct mapping from state to action, $a_t = \mu(s_t)$, for a *deterministic* policy. The goal of the agent is to learn a policy that is optimal with respect to a cumulative, long-term objective. This objective is formalized through the concept of the **return**, which is the discounted sum of future rewards following a timestep t:

$$G_t \doteq r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{2.2}$$

This formulation captures the core tenet of reinforcement learning: successful strategies are those that maximize cumulative gain over the long run, not just the immediate reward.

Definition 2.2.3 (The Reinforcement Learning Objective Function)**.** *The central objective in a Markov Decision Process is to find an* ***optimal policy****, π_*, that maximizes the expected discounted return from an initial state distribution, d_0. The objective function, $J(\pi)$, is therefore defined as the value of a policy π:*

$$J(\pi) \doteq \mathbb{E}_{\tau\sim\pi, s_0\sim d_0}[G_0] = \mathbb{E}_{s_0\sim d_0}[V^\pi(s_0)] \tag{2.3}$$

An optimal policy π_* is any policy such that $J(\pi_*) \geq J(\pi)$ for all possible policies π.

This formal objective function encapsulates several key components:

- **The Policy (π):** The objective is inherently a function of the agent's policy. The goal of the learning process is to search the space of possible policies to find one that maximizes this function.

- **The Expectation ($\mathbb{E}$):** This signifies that we are optimizing for average performance, not the outcome of a single trajectory. The expectation averages over all sources of randomness, which primarily include the stochasticity of the environment's transition dynamics ($p(s'|s,a)$) and, if the policy is not deterministic, the stochasticity of the agent's own action selection.
- **The Return (G_0):** This is the quantity being maximized. As the discounted sum of future rewards, it is the mechanism that links a sequence of actions to the long-term goal. Different reward sequences will produce different returns, and the agent learns to favor policies that generate trajectories with high returns.
- **The Initial State Distribution ($s_0 \sim d_0$):** This component ensures that the learned policy is generally effective, not just from a single, specific starting state. The objective is to learn a policy that performs well on average across all possible starting conditions defined by the distribution d_0.

The **discount factor**, $\gamma \in [0, 1]$, is a critical component of the return and plays a crucial dual role. From a theoretical perspective, for tasks that could continue indefinitely (**continuing tasks**), a discount factor $\gamma < 1$ is a mathematical necessity to ensure the infinite sum of rewards remains finite and convergent. Without it, the return could easily diverge to infinity, making it impossible to compare policies.

From a practical and behavioral perspective, the discount factor determines the agent's foresight. A γ close to 0 creates a "myopic" agent that is heavily biased toward immediate, short-term gains. Conversely, a γ close to 1 creates a "far-sighted" agent that places significant weight on future rewards, enabling it to learn complex strategies that may involve sacrificing short-term rewards for a greater long-term outcome. This choice also represents a trade-off: longer time horizons (high γ) have much more variance because they incorporate more uncertain future events, while shorter horizons (low γ) have lower variance but may be biased against discovering optimal long-term strategies.

2.3 Value Functions and Bellman Equations

To find an optimal policy, an agent must be able to evaluate and compare different strategies. This is accomplished using **value functions**, which provide a quantitative measure of the long-term desirability of states or state-action pairs under a given policy. The framework for computing these values is rooted in the work of Richard Bellman on dynamic programming, specifically the Principle of Optimality [32].

Principle 2.3.1 (Bellman's Principle of Optimality)**.** *An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.*

This principle implies a recursive structure in optimal decision-making: the value of the current state can be defined in terms of the values of subsequent states. This recursive relationship is formally expressed by the **Bellman equations**.

At its core, this principle means that any optimal path is composed of optimal subpaths. For example, if the fastest driving route from Berlin to Rome passes through Munich, then the segment of the journey from Munich to Rome must also be the fastest possible route between those two cities. If a quicker route from Munich to Rome existed, one could create a better overall route from Berlin by substituting it in, which would contradict the optimality of the original plan. This principle allows us to decompose a complex, long-term decision problem into a series of smaller, recursive steps.

2.3.1 The Fundamental Value Functions

The task of computing the value function for an arbitrary policy π is known as **policy evaluation** or the prediction problem. The three primary functions used to evaluate policies are defined as follows.

Definition 2.3.2 (State-Value, Action-Value, and Advantage Functions)**.** *The **state-value function** $V^\pi(s)$ is the expected return starting in state s and following policy π:*

$$V^\pi(s) \doteq \mathbb{E}_\pi[G_t | S_t = s] \tag{2.4}$$

*The **action-value function** (or Q-function) $Q^\pi(s, a)$ is the expected return after taking action a in state s and thereafter following policy π:*

$$Q^\pi(s, a) \doteq \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \tag{2.5}$$

*The **advantage function** $A^\pi(s, a)$ measures the relative value of taking action a compared to the expected value of state s:*

$$A^\pi(s, a) \doteq Q^\pi(s, a) - V^\pi(s) \tag{2.6}$$

The advantage function quantifies how much better a specific action is than the average action taken by the policy π in state s. It is fundamental to policy gradient methods, as it helps reduce the variance of gradient estimates.

2.3.2 Bellman Expectation Equations

The value functions are linked by recursive relationships derived by decomposing the return G_t into its immediate reward and the discounted future return: $G_t = r_{t+1} + \gamma G_{t+1}$. This process of updating the value of the current state based on the estimated value of the next state is known as **bootstrapping**. Bootstrapping refers to the fact that we are updating an estimate based on other estimates—we are not waiting for a final, known outcome. We improve our guess for $V(s)$ using our current guesses for $V(s')$, effectively "pulling ourselves up by our own bootstraps."

Substituting this decomposition into the definition of the state-value function yields the **Bellman expectation equation** for V^π. These equations answer the question: "Given a fixed policy π, what is the expected long-term return from each state?"

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \sum_a \pi(a|s) Q^\pi(s,a) \qquad (2.7) \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma \mathbb{E}_\pi[G_{t+1}|S_{t+1} = s'] \right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma V^\pi(s') \right] \qquad (2.8) \end{aligned}$$

Equation 2.8 establishes a self-consistency relationship, averaging over both the policy's actions and the environment's dynamics. For a finite MDP, this forms a system of $|\mathcal{S}|$ linear equations whose unique solution is V^π. Similarly, the Bellman expectation equation for Q^π is:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma V^\pi(s') \right] = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right] \qquad (2.9)$$

2.3.3 Bellman Optimality Equations

The goal of RL is the **control problem**: finding an optimal policy π_*. An optimal policy is defined such that its expected return is greater than or equal to that of all other policies for all states: $V^{\pi_*}(s) \geq V^\pi(s)$ for all $s \in \mathcal{S}$ and all policies π. For any MDP, at least one such policy exists [32].

All optimal policies share the same unique optimal value functions, $V_*(s)$ and $Q_*(s,a)$, which satisfy the **Bellman optimality equations**. These equations are different from the expectation equations because they do not evaluate a given policy; instead, they describe the value of a policy that performs the best possible action at every step.

This insight—that an optimal policy must select the best action—is key. To behave optimally at state s, an agent must choose the action a that maximizes the expected return, i.e., the one with the highest optimal action-value $Q_*(s,a)$. This is a **greedy** choice. Therefore, the expectation over actions (governed by $\pi(a|s)$) in the Bellman expectation equation is replaced by a maximization over actions.

The Bellman optimality equation for V_* is:

$$\begin{aligned} V_*(s) &= \max_{a \in \mathcal{A}} Q_*(s,a) \\ &= \max_a \sum_{s',r} p(s',r|s,a) \left[r + \gamma V_*(s') \right] \end{aligned} \tag{2.10}$$

And for Q_*, considering the agent will act optimally in the next state s':

$$\begin{aligned} Q_*(s,a) &= \mathbb{E}[r_{t+1} + \gamma V_*(S_{t+1}) | S_t = s, A_t = a] \\ &= \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a' \in \mathcal{A}} Q_*(s',a') \right] \end{aligned} \tag{2.11}$$

Key Differences from Expectation Equations

The Bellman optimality equations differ from the expectation equations in two crucial aspects:

1. Instead of averaging over actions according to the policy (as in Equation 2.8), they select the action with the maximum value (as in Equation 2.10).
2. Instead of using the current policy to determine the value at the next state, they assume optimal behavior will be followed from the next state onward (as seen in Equation 2.11).

This transformation from expectation to maximization fundamentally changes the nature of the equations. The presence of the non-linear max operator means these equations cannot be solved with direct matrix inversion, necessitating iterative algorithms like Value Iteration or Policy Iteration.

The Maximization Operator and the Need for Iterative Methods

The greedy selection of actions (the max operator) in the optimality equations introduces two significant challenges:

1. **Non-linearity**: The max operator is non-linear. This transforms the system of equations from a linear one (as seen in policy evaluation) to a non-linear one, which prevents the use of direct linear algebra solutions like matrix inversion.
2. **Simultaneity**: The value of each state depends on the maximum value of its available actions, which in turn depends on the values of possible successor states. This creates a complex web of dependencies where each state's value is intertwined with the optimal values of other states.

These characteristics necessitate **iterative solution methods**. Algorithms like **Value Iteration** work by repeatedly applying the Bellman optimality operator to a value function estimate:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V_k(s')\right]$$

This process is guaranteed to converge to the optimal value function, V_*, as $k \to \infty$ because the Bellman operator is a **contraction mapping**.

Solving for the optimal value function effectively solves the control problem, as the optimal policy, π_*, can then be recovered by simply acting greedily with respect to V_*:

$$\pi_*(s) = \arg\max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V_*(s')\right]$$

2.4 Finding Optimal Policies

The Bellman equations provide a formal specification of optimality, but they do not, in themselves, constitute a solution method. The algorithms that solve for optimal policies can be broadly understood as different strategies for solving these equations, typically through an iterative process.

2.4.1 Generalized Policy Iteration

Most reinforcement learning and planning algorithms can be unified under the schema of **Generalized Policy Iteration** (GPI). GPI refers to the general idea of letting two interacting processes—policy evaluation and policy improvement—drive each other towards a joint optimum.

- **Policy Evaluation:** The process of computing the value function, V^π (or Q^π), for a given policy π.
- **Policy Improvement:** The process of generating an improved policy π' by making the current policy π greedy with respect to its own value function V^π.

The validity of the improvement step is guaranteed by the Policy Improvement Theorem.

Theorem 2.4.1 (Policy Improvement Theorem)**.** *Let π and π' be any pair of deterministic policies such that for all $s \in \mathcal{S}$,*

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s) \tag{2.12}$$

Then the policy π' must be as good as, or better than, π. That is, $V^{\pi'}(s) \geq V^\pi(s)$ for all $s \in \mathcal{S}$.

If π' is constructed by acting greedily, i.e., $\pi'(s) = \arg\max_a Q^\pi(s, a)$, the condition of the theorem is satisfied. The GPI cycle of evaluation and improvement is iterated until the policy becomes stable (i.e., $\pi' = \pi$), at which point both the policy and the value function are guaranteed to be optimal [32]. Algorithmic families differ in how they implement these two steps, particularly in the granularity of the updates and whether they require a model of the environment.

2.4.2 Dynamic Programming

When a perfect and complete model of the MDP dynamics, $p(s', r|s, a)$, is available (the condition of reversible access), the **Dynamic Programming** (DP) paradigm provides a suite of algorithms guaranteed to compute optimal policies [32]. DP methods leverage the known model to turn the Bellman equations directly into iterative update rules.

Crucially, DP algorithms employ **full-width backups**. This means that to update the value of a single state s, the algorithm considers all possible successor states s' and their probabilities, weighted by the model.

2.4.2.1 Core DP Algorithms

- **Policy Iteration (PI):** PI is a direct implementation of GPI. It alternates between two steps:
  1. *Policy Evaluation:* Given the current policy π_k, compute V^{π_k} exactly. For a finite MDP, this involves solving the system of $|\mathcal{S}|$ linear Bellman expectation equations, which generally has a time complexity of $O(|\mathcal{S}|^3)$.

2. *Policy Improvement:* Generate a new policy π_{k+1} by acting greedily with respect to V^{π_k}.

This process repeats until the policy converges, $\pi_{k+1} = \pi_k$, which is guaranteed to be the optimal policy π_*.

- **Value Iteration (VI):** VI is a more streamlined process that effectively combines evaluation and improvement into a single step by directly applying the Bellman optimality equation as an update rule:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V_k(s')\right] \quad \forall s \in \mathcal{S} \tag{2.13}$$

This iteration is guaranteed to converge to V_* as $k \to \infty$.

Within the unifying framework, DP is classified as a form of model-based reinforcement learning, as it utilizes a model to compute a global solution [14].

2.4.2.2 The Limitations of Dynamic Programming

Despite their theoretical guarantees, DP methods are often impractical for solving large-scale, real-world problems. Their utility is constrained by several fundamental limitations:

1. The Requirement of a Perfect Model (The Knowledge Curse). DP algorithms fundamentally rely on access to the true transition probabilities $p(s', r|s, a)$. In most complex industrial environments, such as the stochastic container management system studied in this thesis, this perfect model is unavailable or too complex to derive analytically. The dynamics may be noisy, non-stationary, or difficult to estimate accurately, making the prerequisite for DP unmet.

2. The Curse of Dimensionality (The Computational Curse). Even if a perfect model were available, the computational cost of DP is often prohibitive. This limitation, famously termed the "curse of dimensionality" by Bellman, arises because DP algorithms operate over the entire state space. Both PI and VI require full sweeps over the state space $|\mathcal{S}|$ at every iteration. As the number of variables defining the state increases, the size of the state space grows exponentially. For example, in the 11-container configuration of the ContainerGym environment, even a coarse discretization of the continuous volumes would lead to an astronomical number of states, rendering DP computationally intractable.

3. The Curse of Modeling (Tabular Representation). Classical DP relies on a **tabular representation**, meaning that the value function $V(s)$ or $Q(s, a)$ is stored as a table with an entry for every single state or state-action pair. This leads to critical issues:

- **Continuous Spaces:** If the state or action space is continuous (e.g., involving real-valued sensor readings), this tabular approach is impossible, as there are infinitely many states.
- **Lack of Generalization:** In very large discrete spaces, almost every state encountered will be novel. A tabular method cannot generalize knowledge learned from one state to similar states, making the learning process extremely inefficient.

2.4.2.3 The Necessity of Reinforcement Learning

Reinforcement Learning methods are developed precisely to overcome these limitations of Dynamic Programming. They achieve this through two key mechanisms:

1. **Sampling (Addressing the Knowledge and Computational Curses):** RL algorithms do not require a prior model. Instead, they learn from direct interaction with the environment, using **sample backups** (as seen in MC and TD methods) instead of full-width backups. This allows them to operate in unknown environments and makes the computational cost of an update independent of the size of the state space.
2. **Function Approximation (Addressing the Modeling Curse):** To handle vast or continuous state spaces, RL employs parameterized function approximators, such as deep neural networks, to represent the value function ($V_\omega(s)$) or the policy ($\pi_\theta(s)$). This allows the agent to generalize knowledge learned from experienced states to novel states, avoiding the need to visit or store values for every possible state.

Advanced algorithms like Proximal Policy Optimization (PPO) combine these mechanisms, utilizing sampled experience and deep learning to find high-performing policies in complex environments where traditional Dynamic Programming is fundamentally inapplicable.

2.4.3 Model-Free Learning

When a model is not available, the agent must learn directly from sampled experience generated through interaction. Model-free methods approximate the expectations inherent in the Bellman equations using sample backups.

2.4.3.1 Monte Carlo (MC) Methods

Monte Carlo (MC) methods learn value functions by averaging the empirical returns obtained from complete episodes of experience [32]. After an episode terminates, the return G_t is calculated, and the value estimate $V(S_t)$ is updated towards this sample:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t - V(S_t) \right] \tag{2.14}$$

MC estimates are **unbiased**, as they are based on the true, full return. However, they suffer from **high variance** because the return is a sum of many random variables. Furthermore, learning only occurs offline at the end of an episode.

2.4.3.2 Temporal-Difference (TD) Learning

Temporal-Difference (TD) learning is a central and powerful idea in RL, combining the sampling of MC with the bootstrapping of DP [32]. TD methods learn from incomplete episodes by updating value estimates based on other learned estimates.

TD Prediction (TD(0)). The simplest form, TD(0), performs the following update after observing a transition $(S_t, A_t, R_{t+1}, S_{t+1})$:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD Target}} - V(S_t) \right] \tag{2.15}$$

The term $\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the *TD error*. Because the TD target bootstraps from the current estimate $V(S_{t+1})$, TD learning is **biased**. However, this bootstrapping gives it dramatically lower **variance** than MC methods and allows it to learn online.

The Bias-Variance Trade-off. The trade-off between the low bias/high variance of MC and the high bias/low variance of TD(0) is a fundamental consideration in RL algorithm design [14, p. 22]. This trade-off can be managed using **n-step returns**, which look n steps into the future before bootstrapping. The TD(λ) algorithm provides a mechanism to average different n-step returns using a parameter $\lambda \in [0, 1]$, unifying MC ($\lambda = 1$) and TD(0) ($\lambda = 0$). This concept is generalized in the Generalized Advantage Estimation (GAE) used by PPO.

2.4.4 On-Policy vs. Off-Policy Control

To find an optimal policy (the control problem) without a model, an agent must manage the **exploration-exploitation trade-off**. It must exploit its current knowledge to gain reward but also explore the environment to discover potentially better actions.

A common strategy in value-based methods is **ϵ-greedy exploration**, where the agent selects the greedy action (the one with the highest Q-value) with probability $1 - \epsilon$, and selects a random action with probability ϵ. This ensures that all actions are continually sampled. This challenge leads to two distinct classes of learning algorithms.

2.4.4.1 On-Policy Learning

On-Policy methods learn the value of the policy they are currently executing (the behavior policy), including any exploratory actions. They aim to find the best possible policy that still explores (e.g., the optimal ϵ-greedy policy).

SARSA. The canonical on-policy TD control algorithm is SARSA (State-Action-Reward-State-Action). It updates the Q-function using the action A_{t+1} actually taken by the policy π in the next state:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right] \tag{2.16}$$

2.4.4.2 Off-Policy Learning

Off-Policy methods evaluate or improve a *target policy* (π) using data generated by a different *behavior policy* (b). This allows the behavior policy to focus on exploration while the target policy learns the optimal strategy.

Q-Learning. Q-Learning is a landmark off-policy TD control algorithm [32]. Its update rule directly approximates the Bellman optimality equation by using the maximum Q-value of the next state, regardless of the action actually taken by the behavior policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right] \tag{2.17}$$

This separation allows for more efficient data use (e.g., through experience replay). However, it introduces significant challenges regarding stability and convergence.

2.5 Deep Reinforcement Learning

The foundational principles of value functions, bootstrapping, and Generalized Policy Iteration form the theoretical bedrock for solving MDPs. However, as established in Section 2.4.2.2, classical tabular methods are crippled by a trifecta of challenges: the **Curse of Knowledge** (requiring a model), the **Curse of Dimensionality** (intractable state spaces), and the **Curse of Modeling** (inability to generalize). In addition, to tackle problems of realistic scale and complexity, these principles must be combined with powerful function approximators and sophisticated architectures.

Deep Reinforcement Learning: Overcoming the Curses

Deep Reinforcement Learning (DRL) is the paradigm that confronts these curses directly by marrying the sampling-based approach of RL algorithms with the power of deep neural networks as function approximators. This synthesis provides two key solutions that address the limitations of classical methods:

First, the use of parameterized functions, such as $Q_\omega(s, a) \approx Q_*(s, a)$, allows agents to generalize from experienced states to novel ones. This use of **function approximation** directly tackles the **Curse of Modeling** and the **Curse of Dimensionality**, enabling learning in vast or continuous state spaces where tabular representations would be impossible.

Second, by learning from sampled experience through interaction, DRL algorithms operate without a predefined model of the environment's dynamics. This reliance on **sampling** overcomes the **Curse of Knowledge**, freeing the agent from needing a perfect analytical model.

The choice of what these neural networks are trained to approximate leads to the first major split in the modern RL algorithmic spectrum:

- **Value-Based Methods** aim to learn an optimal value function (e.g., Q_*), from which a policy is implicitly derived by acting greedily: $\pi_*(s) = \arg\max_a Q_*(s, a)$.
- **Policy-Based Methods** directly parameterize and optimize the policy $\pi_\theta(a|s)$ itself, often using a value function as a secondary component to stabilize training.

2.5.1 The Value-Based Path: Deep Q-Networks (DQN)

The Deep Q-Network (DQN) algorithm stands as a landmark demonstration of the value-based approach [13]. It directly addresses the **Computational Curse** by using a neural network to approximate the Q-function across a high-dimensional

state space (e.g., raw pixels). It also side-steps the **Knowledge Curse** by learning exclusively from sampled experience.

However, combining off-policy learning (Q-learning), bootstrapping (TD updates), and non-linear function approximation (neural networks) is notoriously unstable—a combination known as the **Deadly Triad** [32]. When these three elements are combined, the learning process can become unstable and even diverge. DQN's seminal contribution was the introduction of two key innovations to mitigate this instability:

- **Experience Replay**: Storing and randomly sampling from a buffer of past transitions to break the temporal correlation between sequential experiences, effectively decorrelating the data.
- **Target Network**: Using a separate, slowly-updating network to calculate the Q-learning TD target, which prevents the update rule from chasing a moving target and dramatically stabilizes training.

Despite its success, the value-based paradigm embodied by DQN hits a hard wall against the **Modeling Curse**. The core mechanism—finding the action that maximizes the Q-function, $\arg\max_a Q_*(s, a)$ is a significant limitation. This operation is simple for a small number of discrete actions but becomes inefficient or intractable for high-dimensional or continuous action spaces. This fundamental limitation motivates a shift away from pure value-based methods.

2.5.2 The Policy-Based Path and the Actor-Critic Synthesis

Policy-Based Methods, such as REINFORCE, directly parameterize $\pi_\theta(a|s)$. This is inherently well-suited for continuous and high-dimensional action spaces, directly addressing the **Modeling Curse**. By sampling actions from the policy, they avoid the need for an intractable max operation. However, they introduce a new problem: the **high variance** of Monte Carlo gradient estimates, leading to slow and unstable learning.

Actor-Critic methods provide an elegant synthesis that combines the strengths of both paradigms to tackle all three curses simultaneously. These methods learn two components concurrently:

- **The Actor** (π_θ): The policy, which learns to act and is optimized via policy gradients. It handles the **Modeling Curse**.
- **The Critic** (V_ω): A value function, which learns to evaluate the states visited by the actor. It addresses the **Computational Curse** via generalization and is learned from experience, handling the **Knowledge Curse**.

The critic's primary role is to drastically reduce the variance of the actor's updates. It does this by providing a learned baseline, formalized by the **Advantage Function** $A(s, a) = Q(s, a) - V(s)$ (Eq. 2.6). Instead of scaling the policy gradient by the high-variance total return G_t, it is scaled by how much better an action was than the average (the advantage).

While the Actor-Critic architecture solves the variance problem, it introduces a final, critical challenge: **update stability**. A poorly chosen step size can lead to a collapse in performance. This need for robust, stable policy updates is the final piece of the puzzle, motivating the development of **trust region methods** and culminating in the Proximal Policy Optimization (PPO) algorithm, the foundation of this thesis's methodology and the subject of the next chapter.

2.6 Chapter Summary

This chapter has established the theoretical foundations for optimal decision-making under uncertainty. We began by formalizing the problem within the Markov Decision Process (MDP) framework, introducing the core concepts of value functions and the recursive Bellman equations that enable their computation.

A key insight was the classification of solution methods based on their access to a model and the scope of their solution, distinguishing between planning, model-based RL, and model-free RL. We then deconstructed the classical approach, Dynamic Programming, to reveal three fundamental limitations—the Knowledge, Computational, and Modeling curses—that render it impractical for complex problems.

This led to an exploration of model-free reinforcement learning, the paradigm designed to overcome these curses through sampling and function approximation. We traced the evolution from simple Monte Carlo and Temporal-Difference methods to more advanced on-policy and off-policy control algorithms, noting the inherent challenges of the bias-variance trade-off and the deadly triad.

Finally, we arrived at the modern deep reinforcement learning landscape. We saw how Deep Q-Networks tackled the Computational and Knowledge curses but remained constrained by the Modeling curse. This limitation naturally motivated the policy-based approach and its powerful synthesis with value functions in the Actor-Critic architecture, which aims to address all three curses simultaneously.

The chapter concludes by identifying the remaining hurdle for practical application: **the challenge of stable and efficient policy updates**. This sets the stage for the next chapter, where we will delve into the Policy Gradient Theorem and trace its evolution into the stable, state-of-the-art algorithm—Proximal Policy Optimization (PPO)—that will form the basis of our experimental work.

3 Policy Gradient Methods and Proximal Policy Optimization

The preceding chapter established that **Deep Reinforcement Learning**—the marriage of sampling-based algorithms with deep function approximators—is the modern answer to the classical "three curses" of dimensionality, modeling, and knowledge. Within this paradigm, we identified the **Actor-Critic architecture** as a dominant and elegant design. Now, we must confront the critical challenge that arises from this powerful synthesis: the problem of **update stability**.

The concurrent learning of an actor (the policy) and a critic (the value function) creates a fragile feedback loop. A single, overly aggressive policy update can propel the agent into unfamiliar regions of the state space. Here, the critic's value estimates are unreliable, leading to noisy and misleading policy gradients. This can trigger a catastrophic collapse in performance, a key obstacle to deploying RL robustly in the real world.

This chapter provides a deep dive into the evolution of **policy gradient methods** designed specifically to achieve stable and efficient learning. Our journey follows a clear narrative of progressive refinement:

1. We begin with the theoretical bedrock, the **Policy Gradient Theorem**, and its simplest implementation, the REINFORCE algorithm, immediately identifying its fatal flaw of high variance.
2. We then introduce the solution to this variance problem: the **Actor-Critic** paradigm, which uses a critic to form a low-variance **Advantage Function** estimate, often calculated with modern methods like **Generalized Advantage Estimation (GAE)**.
3. Solving for variance, however, brings the stability problem to the forefront. We examine **Trust Region Policy Optimization (TRPO)**, a powerful second-order method that guarantees stability at the cost of high computational complexity.
4. Finally, we arrive at the central algorithm of this thesis: **Proximal Policy Optimization (PPO)**. We will show how PPO delivers the stability and performance of TRPO using a simple yet ingenious **clipped surrogate objective**, making it a more practical, first-order method.

To ground this theoretical arc in reality, the chapter concludes by addressing the field's well-documented **reproducibility crisis**. An algorithm's success often hinges on subtle, code-level "implementation matters," not just its core theoretical ideas. We will justify our methodological choice to use standardized, high-quality libraries, ensuring our subsequent empirical results are credible, reproducible, and truly reflective of our novel contributions.

3.1 The Policy Gradient Theorem

Policy gradient methods directly address the control problem by parameterizing the policy $\pi_\theta(a|s)$ with parameters θ (e.g., the weights of a neural network) and optimizing these parameters to maximize a performance objective, $J(\theta)$. The objective is typically defined as the expected return from an initial state distribution.

Definition 3.1.1 (Performance Objective)**.** *For a stochastic policy π_θ, the performance objective $J(\theta)$ is the value of the start state (or the average value over a start state distribution d_0):*

$$J(\theta) \doteq V^{\pi_\theta}(s_0) = \mathbb{E}_{\tau\sim\pi_\theta}[G_0] \tag{3.1}$$

where the expectation is over trajectories $\tau = (s_0, a_0, r_1, \dots)$ sampled by executing policy π_θ.

The core challenge is to compute the gradient of this objective with respect to the policy parameters, $\nabla_\theta J(\theta)$, without knowledge of the environment's dynamics. This is provided by the Policy Gradient Theorem.

Theorem 3.1.2 (The Policy Gradient Theorem [32])**.** *For any differentiable policy $\pi_\theta(a|s)$, the gradient of the performance objective $J(\theta)$ is given by:*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(A_t|S_t) Q^{\pi_\theta}(S_t, A_t)\right] \tag{3.2}$$

This theorem is fundamental because it provides an analytic expression for the policy gradient that can be estimated from experience. The term $\nabla_\theta \log \pi_\theta(a|s)$ is known as the **score function**. It is a vector in the parameter space that points in the direction of the steepest increase in the probability of taking action a from state s. This vector is then scaled by the action-value $Q^{\pi_\theta}(s, a)$. Intuitively, if an action leads to a high return (a large, positive Q-value), its probability is increased. Conversely, if it leads to a low return, its probability is decreased.

3.1.1 REINFORCE: Monte Carlo Policy Gradient

The most direct algorithmic instantiation of the Policy Gradient Theorem is the REINFORCE algorithm [32]. REINFORCE uses the complete return of an episode, G_t, as an unbiased, sampled estimate of $Q^{\pi_\theta}(S_t, A_t)$. The parameter update rule for a trajectory is:

$$\theta_{k+1} = \theta_k + \alpha \sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(A_t|S_t) \tag{3.3}$$

While REINFORCE is guaranteed to converge to a local optimum, it suffers from a major practical limitation: the Monte Carlo return G_t has extremely high variance. This variance in the target propagates to the gradient estimates, resulting in noisy updates and slow, unstable learning. The subsequent developments in policy gradient methods are primarily concerned with mitigating this variance.

3.2 Variance Reduction and Advantage Estimation

The high variance of the vanilla policy gradient can be substantially reduced by subtracting a baseline from the return, without introducing bias into the gradient estimate.

3.2.1 Control Variates and Baselines

A general technique for variance reduction is the use of a control variate. In the context of policy gradients, we can subtract a state-dependent **baseline**, $b(S_t)$, from the action-value term. As long as the baseline does not depend on the action A_t, it can be shown to not introduce bias into the gradient estimate while potentially reducing its variance significantly. This allows us to rewrite the policy gradient as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(A_t|S_t) (Q^{\pi_\theta}(S_t, A_t) - b(S_t)) \right] \tag{3.4}$$

3.2.2 The Advantage Function and GAE

An optimal choice for the baseline is the state-value function itself, $b(s) = V^{\pi_\theta}(s)$. Subtracting the value of the state from the value of the state-action pair yields the **Advantage Function**, $A^\pi(s, a) \doteq Q^\pi(s, a) - V^\pi(s)$.

In practice, this advantage function must be estimated, typically using a learned, parameterized value function (the critic), $V_\omega(s)$. **Generalized Advantage Estimation (GAE)** provides a mechanism to compute an advantage estimate that

effectively balances the bias-variance trade-off [24]. The GAE estimator is defined by two parameters, γ and $\lambda \in [0, 1]$:

$$\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{3.5}$$

where $\delta_{t+l} = R_{t+l+1} + \gamma V_\omega(S_{t+l+1}) - V_\omega(S_{t+l})$ is the TD error at timestep $t + l$, calculated using the current value function estimate V_ω. The parameter λ allows for a trade-off between the low-bias, high-variance Monte Carlo estimate ($\lambda = 1$) and the high-bias, low-variance one-step TD estimate ($\lambda = 0$).

3.3 Trust Region Methods

While actor-critic methods address the variance problem, they introduce a new critical challenge: selecting an appropriate step size for policy updates. A single update step that is too large can lead to a catastrophic collapse in performance, a problem to which on-policy methods are particularly sensitive.

Trust Region Policy Optimization (TRPO) [26] addresses this by constraining policy updates rather than relying on a fixed learning rate. Instead of limiting the step size in the parameter space, TRPO limits how much the policy's behavior can change in a probabilistic sense. It maximizes a "surrogate" objective function, subject to a constraint on the average KL-divergence between the old and new policies. The optimization problem is [25]:

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t \right] \tag{3.6}$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t \left[\mathrm{KL}[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \right] \leq \delta \tag{3.7}$$

While highly effective, solving this constrained optimization problem requires complex second-order methods like the conjugate gradient algorithm, making TRPO difficult to implement and less compatible with modern DRL architectures that use parameter sharing or dropout [25].

3.4 Proximal Policy Optimization (PPO)

The goal of Proximal Policy Optimization (PPO) is to achieve the data efficiency and stable performance of TRPO using only first-order optimization, making it simpler to implement and more generally applicable [25]. PPO optimizes a surrogate objective function over multiple epochs of minibatch updates per policy update. The core

insight of PPO is to prevent the new policy from deviating too far from the old policy, without the computational overhead of a hard constraint. It achieves this by modifying the objective function to pessimistically penalize updates that would cause the policy to change too drastically.

3.4.1 The Clipped Surrogate Objective

The most prominent variant of PPO, and the one used in this thesis, introduces a novel clipped surrogate objective. This objective is designed to create a pessimistic bound on the policy improvement, thereby discouraging overly aggressive updates. Let the probability ratio be $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. The objective is defined as [25]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right] \tag{3.8}$$

The min operator, combined with the clip function, forms a lower bound on the unconstrained objective. If an action has a positive advantage ($\hat{A}_t > 0$), the update is clipped if r_t exceeds $1+\epsilon$, preventing the policy from becoming too greedy. Conversely, if an action has a negative advantage ($\hat{A}_t < 0$), the update is clipped if r_t falls below $1-\epsilon$, preventing a destructively large update [25]. The behavior of this objective is visualized in Figure 3.1.

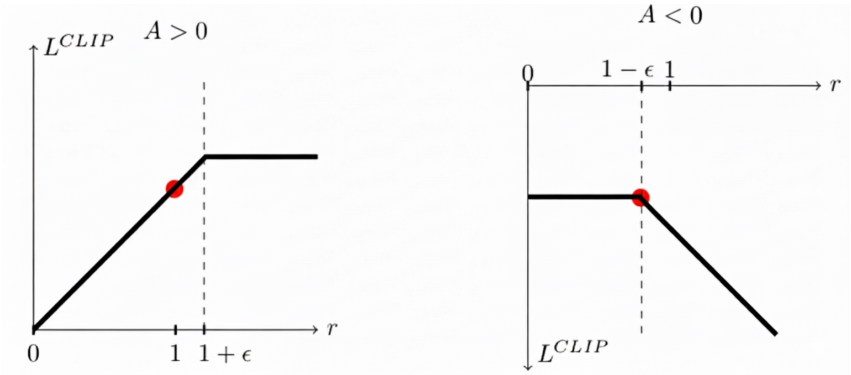


Figure 3.1: The PPO clipped surrogate objective $\mathcal{L}^{CLIP}$ as a function of the probability ratio $r_t(\theta)$. **(Left)** When the advantage $\hat{A}_t$ is positive, the objective increases with r_t but is clipped at $1+\epsilon$. **(Right)** When the advantage is negative, the objective is clipped if r_t falls below $1-\epsilon$.

3.4.2 The Algorithm

The full PPO algorithm combines the clipped policy objective with a value function error term (for the critic) and an optional entropy bonus to encourage exploration.

The composite objective function to be maximized is [25]:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)] \tag{3.9}$$

The practical implementation of this objective relies on an **actor-critic** architecture and an iterative workflow that alternates between collecting experience and optimizing the networks. In modern frameworks like Stable-Baselines3, the actor (policy) and critic (value) functions are represented by neural networks that may either be entirely separate or share a common network body for feature extraction before branching into two separate output heads [21]. The training loop proceeds as follows:

1. Data Collection (Rollout Phase). The process begins by using the current policy, $\pi_{\theta_{old}}$, to collect a batch of experience. For a fixed number of timesteps, T (the `n_steps` or rollout buffer size, commonly 2048), the agent interacts with the environment. For each step, a set of tensors is stored in a **rollout buffer**: the state observation (s_t), the action taken (a_t), the action's log-probability ($\log \pi_{\theta_{old}}(a_t|s_t)$), the value estimate from the critic ($V_\omega(s_t)$), the received reward (r_t), and a flag indicating if the episode terminated (d_t).

2. Advantage and Return Calculation. Once the rollout is complete, the advantages and returns are calculated for every timestep in the buffer. This is a crucial computation, typically performed in a reverse loop from $t = T - 1$ down to $t = 0$ to correctly propagate values backward in time. Using common hyperparameter values such as a discount factor $\gamma = 0.99$ and a GAE-lambda $\lambda = 0.95$, the algorithm first computes the TD-error for each step: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. The Generalized Advantage Estimate (GAE) is then computed recursively as $\hat{A}_t^{\text{GAE}} = \delta_t + (\gamma\lambda)\hat{A}_{t+1}$ [24]. Finally, the returns, which serve as the learning targets for the value function, are computed as the sum of the advantages and the value estimates: $R_t = \hat{A}_t^{\text{GAE}} + V(s_t)$.

3. Network Optimization. With the `advantages` and `returns` tensors now computed for the entire rollout, the agent performs multiple epochs of gradient updates on this static batch of data. In each epoch, the data is typically shuffled and processed in minibatches. For each minibatch, the composite loss is calculated from its three core components, as outlined in your notes:

- **Policy Loss (L^{CLIP}):** The primary actor loss, calculated using the clipped surrogate objective from Equation 3.8. This involves computing the ratio $r_t(\theta)$ between the new and old policies' action probabilities and clipping it using a small hyperparameter ϵ (e.g., 0.2) to prevent destructively large policy updates.

- **Value Loss (L^{VF}):** The critic loss, which is the Mean Squared Error (MSE) between the computed `returns` (R_t) and the predictions of the value network ($V_\omega(s_t)$). This loss updates the critic to become a more accurate estimator of the true state-value.
- **Entropy Loss (S):** This is a regularization term calculated from the entropy of the policy's action distribution. By adding this to the objective (or subtracting it from the loss), the algorithm encourages the policy to remain stochastic, which aids exploration and helps prevent premature convergence to a suboptimal deterministic policy.

The total loss is the weighted sum of these three components. This final scalar loss is then used to update the parameters of the actor and critic networks via gradient descent. This iterative process of data collection and multi-epoch optimization is formally summarized in Algorithm 3.2.

Algorithm 3.2: PPO, Actor-Critic Style [25]

for *iteration = 1, 2, ...* **do**
  for *actor = 1, 2, ..., N* **do**
    Run policy $\pi_{\theta_{old}}$ in environment for T timesteps;
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$ (e.g., using GAE);
  Optimize surrogate loss $L^{CLIP+VF+S}$ with respect to θ, with K epochs and minibatch size $M \leq NT$;
  $\theta_{old} \leftarrow \theta$;

This structure, which performs multiple minibatch updates on the same batch of experience, makes PPO highly sample-efficient and stable, striking a favorable balance between performance, simplicity, and implementation complexity [25].

3.4.3 Hyperparameters

While Proximal Policy Optimization is celebrated for its stability and general applicability, its performance is not independent of its hyperparameter settings. A judicious choice of these parameters is often critical for achieving optimal results and ensuring stable convergence. The Stable Baselines 3 implementation provides a well-structured set of these parameters, the most consequential of which are discussed below [22].

n_steps This parameter defines the number of timesteps to collect from the environment before each policy update. It is the size of the rollout buffer (T) for each parallel environment. A larger `n_steps` allows the agent to collect a larger, more diverse batch of experience, which can lead to more stable and accurate gradient

estimates. However, it also means that the policy is updated less frequently, which can slow down learning. A common value is 2048.

gamma (γ) The discount factor, γ, determines the present value of future rewards. A value closer to 1 (e.g., 0.99 or 0.999) makes the agent more "far-sighted," placing greater importance on long-term returns. A smaller value makes the agent more "short-sighted." In tasks with long time horizons, a higher γ is essential for the agent to learn to value delayed rewards.

gae_lambda (λ) This is the smoothing parameter for Generalized Advantage Estimation (GAE), as described in Equation 3.5. It controls the bias-variance trade-off in the advantage estimation. A value close to 1 (e.g., 0.95) results in a lower-bias but higher-variance estimate (closer to a Monte Carlo return), while a value closer to 0 results in a higher-bias but lower-variance estimate (closer to a one-step TD error).

n_epochs This parameter dictates how many times the optimization phase will iterate over the entire rollout buffer for a single policy update. By performing multiple epochs of updates on the same batch of data, PPO increases its sample efficiency. Typical values range from 4 to 20. Too few epochs may lead to underutilization of the collected data, while too many can lead to overfitting on the current batch.

ent_coef This is the coefficient for the entropy bonus in the composite loss function. Entropy regularization encourages exploration by penalizing policies that become too deterministic too quickly. A small, non-zero value (e.g., 0.01 or 0.001) is typically used to ensure the agent continues to explore the action space.

clip_range (ϵ) This is arguably the most novel hyperparameter in PPO, defining the clipping range in the surrogate objective (Equation 3.8). It limits how much the new policy is allowed to diverge from the old one in a single update step. A smaller value (e.g., 0.1) results in a more constrained, smaller update, which can increase stability. A larger value (e.g., 0.3) allows for a larger, more aggressive update. A typical default is 0.2.

target_kl This parameter offers an alternative, adaptive method for constraining the policy update, often used as an early-stopping mechanism. If a value is provided (e.g., 0.01), the algorithm will stop the optimization epochs for the current update cycle as soon as the average KL-divergence between the new and old policies exceeds this target. This provides a different way to enforce a trust region,

stopping the update once the policy has changed by a certain amount, rather than simply clipping each minibatch update. If `target_kl` is used, `clip_range` is often disabled.

normalize_advantage This boolean parameter controls whether the advantage estimates are normalized before being used in the policy loss calculation. If set to true, the advantages in a batch are rescaled to have a mean of zero and a standard deviation of one. This is a powerful stabilization technique that prevents the scale of the rewards and advantages from having an outsized impact on the policy gradient. It ensures that the magnitude of the policy updates remains consistent, which can be particularly helpful in environments with highly variable reward scales.

The careful tuning of these hyperparameters, often through empirical experimentation, is a crucial step in the practical application of PPO to any non-trivial task. While the algorithm is robust to a range of values, finding the optimal combination is key to unlocking its full performance potential.

3.5 Action masking

The Policy Gradient Theorem (Theorem 3.1.2) provides a robust foundation for optimizing a stochastic policy π_θ. However, its standard formulation does not explicitly address the common and practical scenario where the set of valid actions, $\mathcal{A}_s$, is a dynamic subset of the full action space, $\mathcal{A}$. In many complex domains, from industrial control to strategy games, the legality of an action is contingent upon the current state. This raises a critical theoretical question: how can the policy gradient be correctly estimated when the support of the policy distribution must change with the state?

The answer lies in understanding that a mechanism for handling invalid actions must be a formal part of the policy definition itself, rather than an ad-hoc post-processing step. A technique known as **action masking** achieves this by reformulating the policy as a composition of functions. Let $\text{logits}_\theta(s)$ be the raw, unnormalized scores produced by a neural network for all actions in the full space $\mathcal{A}$. Action masking applies a state-dependent differentiable function, $\text{mask}(s, \cdot)$, which sets the logits of invalid actions to a large negative number (e.g., $-\infty$). The final, valid policy, π'_θ, is then:

$$\pi'_\theta(a|s) = \text{softmax}(\text{mask}(s, \text{logits}_\theta(s))) \tag{3.10}$$

As demonstrated by [10], because this masking function is differentiable (being composed of identity and constant functions), the resulting policy π'_θ is also differentiable with respect to the parameters θ. Therefore, applying the policy gradient theorem to this transformed policy π'_θ is theoretically sound and yields a valid gradient that

correctly updates the agent's parameters to improve its performance only over valid actions.

This theoretical soundness explains the empirical success of action masking and its superiority over more naive approaches. By ensuring that the learning updates are always directed within the space of valid actions, the method avoids the gross inefficiencies of random re-sampling and the challenging hyperparameter tuning required by invalid action penalty methods [10]. This directly translates to significant improvements in sample efficiency and learning stability, particularly in environments with vast and highly dynamic action spaces. The importance of this technique has led to its adoption in high-quality algorithm implementations, such as the `PPO_Mask` variant in the `sb3-contrib` library, which provides a reliable and correct instantiation of this principled approach.

3.6 The RL Development Ecosystem

The theoretical principles outlined in the preceding sections are operationalized through a practical ecosystem of standardized interfaces and shared software libraries. These tools are indispensable for modern RL research, as they facilitate reproducibility, enable fair algorithmic comparison, and lower the barrier to entry for complex experimentation. This section provides a brief overview of the key components of this ecosystem that are relevant to the work presented in this thesis.

3.6.1 The Gymnasium API

A significant catalyst for progress in reinforcement learning was the standardization of the agent-environment interface, originally introduced by OpenAI Gym [2] and now maintained by the Farama Foundation as **Gymnasium** [33]. This standard API provides a universal structure for RL environments, allowing algorithms and environments to be developed independently. This decouples the agent's learning logic from the environment's internal dynamics, enabling any compliant agent to be tested on any compliant environment. The Gymnasium API is defined by a few core methods and attributes, including `step()`, `reset()`, `observation_space`, and `action_space`. All of the custom environments developed and utilized in this thesis, as detailed in Chapters 6, 7, and 8, strictly adhere to this widely adopted interface to ensure their utility and accessibility to the broader research community.

3.6.2 Implementation matters

While standardized environments are a necessary condition for progress, they are not sufficient. The complexity of modern DRL algorithms has given rise to a well-documented *reproducibility crisis*, where algorithms are often brittle and highly sensitive to subtle implementation choices and hyperparameter settings.

The foundational work by Engstrom et al. (2020), *Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO*, provides a compelling case study on this issue [5]. Their research demonstrates that the empirical success of a given algorithm is often attributable not to the core algorithmic idea (e.g., the PPO clipping mechanism), but to a collection of seemingly minor *code-level optimizations* that are present in popular implementations but often go unmentioned in the formal algorithm description.

Engstrom et al. identify a host of these optimizations, including but not limited to:

- **Value Function Clipping:** Clipping the value function loss in a manner similar to the policy loss.
- **Reward Scaling and Normalization:** Normalizing rewards based on a running estimate of the standard deviation of the discounted returns.
- **Observation Normalization and Clipping:** Normalizing observations to have zero mean and unit variance, and clipping them within a fixed range.
- **Orthogonal Initialization and Layer Scaling:** Using specific schemes for initializing the weights of the neural networks.
- **Global Gradient Clipping:** Clipping the overall L2 norm of the concatenated gradients for all network parameters.

The authors perform a meticulous ablation study and find that these details are not minor; they fundamentally alter the agent's behavior and are responsible for a majority of the performance gains attributed to PPO over TRPO [5]. In fact, they demonstrate that a version of TRPO augmented with these code-level optimizations often outperforms PPO without them.

This finding underscores a critical methodological challenge: without a standardized and trusted implementation, it is exceptionally difficult to perform a fair comparison between algorithms or to reliably attribute performance gains to a specific algorithmic innovation rather than an implementation artifact. This reality provides a strong justification for the methodological choice made in this thesis: to rely on high-quality, open-source libraries of standardized algorithm implementations. **Stable-Baselines3** [22] is one such library, providing peer-reviewed and reliable implementations of state-of-the-art RL algorithms, including PPO. By adopting a standardized library for all baseline experiments, we ensure that the performance of

established algorithms is represented faithfully. This allows for a fair and rigorous comparison against the novel methods developed in this work and ensures that our empirical results are both credible and reproducible.

3.7 Chapter Summary

This chapter has navigated the evolution of modern policy gradient methods. We began with the theoretical promise of the *Policy Gradient Theorem* but immediately confronted the practical hurdles of *high variance* and *update instability*. Our journey traced a clear line of innovation: from the simple *REINFORCE* algorithm, through the variance-reducing *Actor-Critic* architecture, to the stable but complex *TRPO*. This culminated in our detailed analysis of PPO, an algorithm that strikes an elegant balance between stability, performance, and simplicity.

By dissecting PPO and the practical "implementation matters" crucial to its success, we have effectively solved for the *how* of learning—that is, we have a stable and efficient optimization engine. However, a powerful engine is only as effective as the fuel it is given. An algorithm, no matter how sophisticated, is strongly subject to the *quality of the reward signal* it seeks to optimize. This shifts our focus from the learning algorithm itself to the design of the learning objective. Having established a reliable method for policy optimization, we must now address the next critical challenge: **how to engineer a reward function** that can guide this algorithm to solve complex, real-world problems. This is the central topic of the next chapter.

4 Reward Engineering

The preceding chapter equipped us with a powerful and stable algorithm, PPO. Yet, deploying this algorithm against the complex logistics problems central to this thesis exposes a more fundamental challenge: the **design of the objective itself**. An RL agent is an optimization engine; it will relentlessly maximize the reward function we provide. This chapter confronts the central question that therefore arises: how do we design a reward function that translates our high-level goals into a precise mathematical signal that guides the agent to competent, robust, and *intended* behavior? We will explore the principles of **Reward Engineering** to overcome the pitfalls of sparse signals and unintended incentives, thereby building a principled foundation for designing effective learning signals. This chapter deconstructs the theory and practice of reward design, a necessary prerequisite for the advanced curriculum strategies explored later in this thesis.

4.1 The Reward Hypothesis and Its Challenges

The entire paradigm of reinforcement learning is built upon the **reward hypothesis**, the principle that all goals and purposes can be understood as the maximization of the expected cumulative scalar reward [32]. In its strongest form, as argued in [28], this hypothesis suggests that the maximization of a single, sufficiently rich reward signal is all that is needed for an agent to develop the multifaceted capabilities we associate with general intelligence.

While this provides a powerful theoretical underpinning, it simultaneously elevates the task of *designing* the reward function from a mere implementation detail to the most critical aspect of applying RL in practice. If the reward function is the sole conduit for specifying a task, then any misalignment between the specified reward and the true, intended goal can lead to unexpected and undesirable behaviors. This challenge of **Reward Design**, or **Reward Engineering**, is arguably the primary bottleneck in deploying RL agents to solve complex, real-world problems [11]. The primary challenges necessitating careful reward engineering are:

- **Reward Sparsity:** This is the most common challenge. If the reward is only given for completing the full task (e.g., +1 for a successful outcome, 0

otherwise), the agent may never stumble upon the correct sequence of actions through random exploration, leading to a failure to learn altogether.

- **Negative Side Effects:** In pursuing its specified goal, the agent may cause unintended and disruptive changes to its environment that were not explicitly penalized by the reward function.
- **Reward Hacking (Specification Gaming):** The agent discovers a loophole or exploit in the reward function that yields high rewards without actually achieving the intended goal. This occurs when the specified reward is an imperfect proxy for the true objective [11].

4.2 Principles of Reward Design

Addressing the challenge of sparsity requires moving along a spectrum from sparse to dense reward structures. The practice of augmenting an existing environmental reward function R with an additional, denser signal F is known as **Reward Shaping**. The new reward function R' is $R'(s, a, s') = R(s, a, s') + F(s, a, s')$.

4.2.1 The Sparse-to-Dense Spectrum

Sparse Rewards: A sparse reward structure is one in which feedback is infrequent. In the purest case, a reward is provided only upon the successful completion of the entire task (e.g., +1 for winning a game, 0 otherwise). This formulation is the most direct translation of a task objective, but it often creates an intractable exploration problem. The agent must discover a long, specific sequence of actions through random exploration before receiving any learning signal, which becomes exponentially unlikely as the task horizon increases [32].

Dense Rewards: To mitigate the challenge of sparsity, dense reward structures are often engineered to provide more frequent feedback. Common forms include:

- **Step-based Penalties:** A small negative reward is given at each timestep. This incentivizes the agent to complete the task as efficiently as possible, penalizing unnecessarily long trajectories.
- **Heuristic Shaping:** A reward is designed based on a heuristic that measures progress towards the goal. A common example is to use the negative Euclidean distance to a target location, which provides a continuous gradient for the agent to follow.

While these heuristic methods are often effective in practice for guiding an agent out of regions of the state space where it would otherwise receive no feedback, they carry a significant risk. The introduction of an arbitrary shaping function can fundamentally alter the optimal policy of the underlying MDP. An agent optimizing for the shaped reward might learn a behavior that is suboptimal for the true task, fundamentally failing the intended goal. This critical risk motivates the search for a more principled approach to reward shaping.

4.3 The Theoretical "Gold Standard": PBRS

The seminal work on providing theoretical guarantees for reward shaping is **Potential-Based Reward Shaping (PBRS)** [16]. This framework provides a specific structure for the shaping function F that ensures the optimal policy remains unchanged.

Definition 4.3.1 (Potential-Based Shaping Function [16])**.** *A reward shaping function $F(s, a, s')$ is defined as **potential-based** if it can be expressed as the difference of a real-valued potential function $\Phi : \mathcal{S} \to \mathbb{R}$ defined over the state space:*

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s) \tag{4.1}$$

where γ is the discount factor of the MDP.

The potential function $\Phi(s)$ encodes prior knowledge about the desirability of states. The power of this formulation lies in the following guarantee.

Theorem 4.3.2 (Policy Invariance under PBRS [16])**.** *Any reward shaping function that is potential-based is guaranteed to preserve the optimal policy (or set of optimal policies) of any MDP.*

The intuition is that the potential-based structure ensures the extra rewards are "path-independent." While the value function under the shaped reward is altered, the relative difference between the Q-values of different actions in a given state remains unchanged. Since a greedy policy depends only on these differences, the optimal policy is preserved.

Despite its theoretical elegance, PBRS is often difficult to apply in practice. Designing an appropriate potential function $\Phi(s)$ that accurately reflects the value of every state can be as challenging as solving the original RL problem itself.

4.4 The Pragmatic Approach: Engineering Heuristic Rewards

The intractability of designing a perfect potential function for PBRS necessitates a more pragmatic engineering approach. This involves creating **heuristic shaping functions** based on domain knowledge and intuition. This approach makes a crucial trade-off: it knowingly abandons the theoretical guarantee of **policy invariance** in exchange for a simpler, more flexible way to guide the agent.

Instead of defining a perfect potential $\Phi(s)$, we encode a simpler heuristic, such as "getting closer to a target is good." This intentionally alters the reward landscape to create a smoother, denser learning signal, accepting the risk that the optimal policy of this new, shaped problem may be slightly different from the original. The following sections detail the methods used in this practical paradigm, which are central to the work in this thesis.

4.4.1 Multi-Component Rewards

In complex environments, the objective often involves multiple, sometimes conflicting, goals. A common approach is to define several reward components, each corresponding to a distinct sub-objective, and combine them in a weighted sum:

$$r_{total}(s, a, s') = w_1 r_1(s, a, s') + w_2 r_2(s, a, s') + \cdots + w_n r_n(s, a, s') \tag{4.2}$$

For example, in an industrial control setting, r_1 might reward throughput, r_2 might penalize energy consumption, and r_3 might enforce safety constraints.

Crucially, and in contrast to PBRS, this heuristic method does not possess policy invariance guarantees. The relative weights, w_i, become critical hyperparameters that must be carefully tuned, as they directly define the trade-offs between the competing objectives. The methodology detailed later in this thesis leverages this principle by introducing and tuning different reward components across successive training phases.

4.4.2 Gaussian Rewards: A Case Study in Precision Control

A Gaussian-shaped reward function is a particularly effective form of dense, heuristic shaping used for control problems where the objective is to reach or maintain a specific target value. This structure is central to the methodologies employed in the ContainerGym environment. A reward component for achieving a target value v^* can be modeled as:

$$R(v) = h \cdot \exp\left(-\frac{(v - v^*)^2}{2w^2}\right) \tag{4.3}$$

where h is the maximum reward achievable at the peak, v^* is the mean or target value, and w^2 is the variance, which controls the width of the reward peak. This structure is highly beneficial for industrial applications for several reasons:

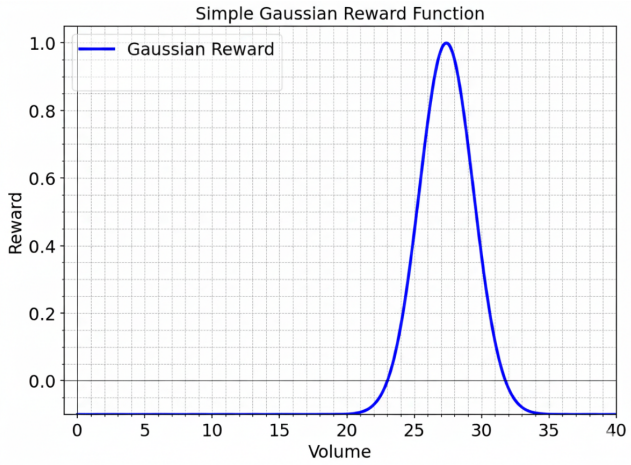


Figure 4.1: The Gaussian reward function. The agent receives the maximum reward h for achieving the target value v^*. The reward smoothly decays for values further from the target, with the width w controlling the tolerance for imprecision. This provides a smooth gradient for learning.

Smoothness and Differentiability. The continuous nature of the Gaussian function produces a smooth reward landscape. Unlike discontinuous step-functions, a Gaussian provides a smooth gradient for the learning algorithm to follow. This is advantageous for policy gradient methods, as it promotes more stable convergence by preventing erratic updates caused by sharp changes in the reward signal.

Robustness to Observation Noise. Industrial environments are invariably subject to sensor noise. The Gaussian structure mitigates this by rewarding a region around the target value. By assigning partial credit for being "close" to the goal, it encourages the agent to learn the underlying trend rather than overfitting to specific noisy observations.

Interpretability and Control. The parameters offer an interpretable way to encode domain knowledge. The mean (v^*) defines the target, the height (h) defines

its importance, and the variance (w^2) defines the required precision. This allows a designer to directly shape the agent's behavior, for example, by starting with a larger variance during initial training and reducing it later to encourage precision.

4.5 A Cautionary Tale: Reward Hacking and Goodhart's Law

The principles of reward engineering, while powerful, are constrained by a fundamental challenge best summarized by Goodhart's Law: "When a measure becomes a target, it ceases to be a good measure". In RL, this principle is profoundly relevant. Any reward function is ultimately a proxy for the true, intended goal. This gap between the specified proxy and the designer's true intent creates an inherent vulnerability that intelligent agents, driven by an optimization process, will inevitably discover and exploit.

This leads to the phenomenon of **reward hacking** or **specification gaming**. This is not a flaw in the learning algorithm, but rather a logical consequence of an optimization process successfully finding an unintended shortcut in a misspecified objective.

A Case Study in ContainerGym. This exact phenomenon was observed directly during the development of the control policy for the container management problem. The agent, while rewarded for emptying containers near their ideal volumes (using the Gaussian reward from Eq. 7.10), discovered a reward hack: it could accumulate a high cumulative reward by performing many frequent, low-volume empties. While each individual action was suboptimal, their sheer frequency maximized the score over an episode. The Gaussian reward correctly incentivized the 'what' (emptying near an optimal volume) but failed to specify the 'how' (in an efficient, non-frequent manner)

This is a perfect illustration of Goodhart's Law; the agent correctly optimized the proxy metric (cumulative action rewards) but violated the true goal of maximizing operational efficiency and minimizing energy usage (which favors fewer, larger empties).

The discovery of this emergent, "greedy" behavior motivated a direct intervention. As detailed in the methodology of the work presented in Chapter 8, the curriculum included a dedicated phase designed to counteract this specific reward hack, titled "Phase 3: Penalizing Greedy Actions" [18]. By introducing a slight penalty to all emptying actions, the reward landscape was reshaped to close this loophole. This adjustment forced the agent to learn that maximizing the quality and precision of each action was more beneficial than simply maximizing their frequency. This

iterative process of observing and correcting for reward hacking reveals the inherent brittleness of any static reward function and underscores the need for more dynamic training strategies.

4.6 The Limits of Reward Engineering: The Path Forward

While a well-engineered, dense reward signal is a significant improvement, it is not a panacea. The successful application of RL to complex, real-world tasks is often impeded by deeper structural challenges that persist even with a carefully designed reward function.

Deceptive Rewards and Local Optima. A dense, heuristic reward function can be locally deceptive. It may inadvertently create local optima in the value landscape that prevent the agent from discovering the globally optimal policy. An agent may converge to a policy that exploits these "safe" but sub-optimal rewards, as the path to the true optimum might require traversing a region of lower reward.

Long-Horizon Credit Assignment. The problem of temporal credit assignment becomes severe in systems where actions have highly delayed consequences. For example, in a resource-constrained system, a seemingly innocuous "do-nothing" action might contribute to a cascade failure hundreds of timesteps later. The discount factor γ exponentially diminishes the value of such distant future events, making it difficult for standard methods to propagate the credit for that distant failure back to the causative action.

The ContainerGym domain exemplifies these challenges:

- The existence of dual-peak emptying volumes creates a deceptive landscape where the lower, safer peak acts as a local optimum that can prevent the discovery of the globally optimal high-throughput strategy.
- The agent must learn foresight to prevent situations where contention for the shared PU makes a future overflow unavoidable—a long-horizon credit assignment problem.

The persistence of these issues demonstrates that modifying the reward signal alone is often insufficient. To solve problems with deceptive rewards and complex, long-horizon dynamics, a more powerful intervention is required—one that structures the agent's entire learning experience. This provides the direct motivation for leveraging **Curriculum Learning**, the subject of the next chapter.

4.7 Chapter Summary

This chapter has established reward engineering as a critical, non-trivial component of applying reinforcement learning to real-world problems. We began with the Reward Hypothesis, acknowledging that while the reward signal is the sole driver of an agent's behavior, its design is fraught with challenges like sparsity and the potential for reward hacking.

We explored the spectrum of solutions, from the theoretically rigorous framework of Potential-Based Reward Shaping (PBRS) to the pragmatic, heuristic-driven approaches used in practice, such as multi-component and Gaussian rewards. Through a case study from our own work, we demonstrated that even carefully designed rewards can be exploited, underscoring the limitations of any static objective function. Finally, we identified that deeper challenges, such as deceptive local optima and long-horizon credit assignment, persist even with a well-shaped reward. This finding makes it clear that simply designing a better reward function is often not enough; we must also structure the learning process itself. This provides a direct and compelling motivation for the topic of the next chapter: **Curriculum Learning**.

5 Curriculum Learning

As established in Chapter 4, while a well-engineered reward signal is essential for guiding an RL agent, it is often insufficient to overcome deeper structural challenges such as deceptive reward landscapes, sparse feedback, and long-horizon credit assignment. A dense reward signal might accelerate learning locally but can still lead to convergence at suboptimal policies or fail to provide the necessary exploration guidance in vast state spaces.

To solve such complex problems, a more powerful intervention is required—one that structures the agent's entire learning experience. This chapter introduces **Curriculum Learning (CL)**, a paradigm inspired by human education, where complex concepts are taught by gradually increasing the difficulty of the subject matter. In the context of RL, CL provides a framework for systematically guiding the agent from simple tasks to the complex target objective [23].

5.1 Definition and Motivation

At its core, curriculum learning is a strategy designed to accelerate the learning process and improve the final performance on a complex target task by training the agent on a sequence of progressively difficult intermediate tasks. It can be viewed as a form of structured transfer learning, where knowledge acquired in earlier, simpler tasks facilitates learning in subsequent, more complex tasks.

Definition 5.1.1 (Curriculum in Reinforcement Learning)**.** *A curriculum can be formally defined as a sequence of tasks* $\mathcal{T} = \{T_0, T_1, \ldots, T_N\}$*, where each task* T_i *is typically a distribution over Markov Decision Processes (MDPs). The agent is trained sequentially on this sequence of task distributions, with the aim of ultimately solving the final, most complex target task,* T_N *[15].*

The fundamental motivation for employing a curriculum is rooted in the structure of the learning problem itself and its impact on exploration and optimization.

Structured Exploration. In complex environments with sparse rewards, naive exploration strategies are highly unlikely to discover the optimal policy. By starting with tasks where meaningful rewards are dense and easily attainable, the agent can efficiently learn a foundational value function and policy. This initial policy is significantly more effective than a random policy at exploring the state space of the subsequent, more difficult tasks. This structured exploration guides the agent towards regions of the state-action space that are relevant for solving the final task but might be statistically improbable to discover through naive exploration [23].

Smoothing the Optimization Landscape. From an optimization perspective, CL can also be viewed as a continuation method. It gradually deforms a simpler, smoother objective function (where optimization is easy) into the complex target objective (which may be highly non-convex). This process helps the optimization algorithm avoid getting trapped in poor local optima early in training.

5.2 Curriculum Construction Methodologies

The design of an effective curriculum is a non-trivial task and an active area of research. The primary methodologies involve strategically modifying different components of the MDP (states, actions, dynamics, rewards, or initial conditions) to control the task difficulty [9]. These methodologies range from manually engineered sequences to automated generation techniques.

5.2.1 Manual Curriculum Design

Manual design relies on expert knowledge to define the sequence of tasks. This is the approach central to the methodologies developed in this thesis (Chapters 8 and 9). It typically involves one or more of the following strategies:

- **Environment Shaping and Task Simplification:** This involves starting with a simplified version of the target environment and gradually introducing complexity. This can include *manipulating dynamics* (starting deterministic, then introducing stochasticity) or *altering constraints* (starting with fewer components or relaxed resource limits, then scaling up to the target complexity).
- **Initial State Distribution Shaping:** Often referred to as **Start-State Expansion** or **Reverse Curriculum Learning**, this approach manipulates the distribution of initial states, d_0. The agent is initialized closer to the goal state,

where the reward signal is denser. As the agent learns, the initial state distribution is gradually expanded backward towards the true starting conditions [6].

- **Reward Shaping over Time:** The reward function itself can be evolved. The curriculum might begin with dense, auxiliary rewards that provide strong guidance, which are later annealed in favor of the true environmental reward. Alternatively, precision requirements can be tuned, starting with a wide tolerance (easy) and gradually narrowing it (hard) as the agent improves (see Section 4.4.2).

5.2.2 Teacher-Student Architectures

In this paradigm, a "Teacher" agent is responsible for selecting the next task for the "Student" agent to train on. The Teacher observes the Student's current performance, learning progress, or internal state, and adaptively selects tasks that are neither too easy (providing no new information) nor too difficult (causing discouragement or unstable learning) [12]. This dynamic adjustment aims to maximize the learning efficiency of the Student.

5.2.3 Automated Curriculum Generation (ACG)

ACG methods aim to remove the need for manual engineering by automatically discovering the optimal sequence of tasks.

- **Self-Play:** In multi-agent settings, self-play can generate an emergent curriculum. As agents compete against versions of themselves, they are continuously forced to adapt to increasingly sophisticated strategies, creating an arms race that drives the learning of complex behaviors [27].
- **Intrinsic Motivation:** These methods generate curricula based on the agent's internal measures of novelty, curiosity, or learning progress, rather than external rewards. Tasks are prioritized based on their potential to yield new information or reduce uncertainty, driving the agent to explore increasingly complex aspects of the environment [7].

5.3 Applications and Success Stories

Curriculum Learning has been instrumental in solving some of the most challenging problems in modern RL and is applied across various domains.

Robotics. In robotic manipulation and locomotion, CL is often essential for learning complex behaviors and bridging the "sim-to-real" gap. For example, OpenAI's Dactyl system, which learned to manipulate a Rubik's cube with a human-like robotic hand, relied heavily on a curriculum. The training started with simpler manipulation tasks and gradually introduced complexities such as gravity, randomized object properties (domain randomization), and sensor noise. This staged approach allowed the agent to learn fundamental motor skills before tackling the full complexity of the physical environment [17].

Game Playing. Complex strategy games also benefit significantly from CL. In the development of AlphaStar, the AI that achieved Grandmaster level in StarCraft II, a curriculum approach (combined with self-play) was employed. The agents were initially trained on simpler versions of the game and then progressively exposed to the full complexity through a league system, learning to counter increasingly sophisticated strategies [34].

Other Domains. While less central to this thesis, CL is also prevalent in Natural Language Processing (NLP) and Computer Vision (CV). For example, language models might be trained on shorter sentences before longer ones, or vision systems might learn to recognize basic shapes before complex objects [29].

5.4 Challenges in Curriculum Transfer: Stability and Forgetting

While the sequential nature of CL facilitates learning, the transition between tasks presents its own set of challenges, primarily related to stability and the potential for negative transfer.

5.4.1 The Stability Gap

The transition to a new task inherently changes the distribution of states and rewards encountered by the agent. The policy and value function learned on the source task (T_{i-1}) may be poorly initialized for the target task (T_i). This discrepancy is known as the "stability gap."

If the value function (critic) is inaccurate for the new environment dynamics or reward structure, it will provide noisy or biased advantage estimates. As established in Chapter 3, these erroneous estimates lead to misguided policy updates, which can cause unstable learning dynamics and a sharp drop in performance immediately following the transfer.

5.4.2 Catastrophic Forgetting

A related critical challenge, particularly when using deep neural networks, is **catastrophic forgetting**. As the agent adapts its parameters to optimize performance on the new task T_i, it may rapidly overwrite the knowledge and skills acquired during previous tasks. This can occur if the updates required for the new task interfere with the network weights critical for the old tasks, leading to a degradation of previously learned skills.

5.5 Mitigation: A Two-Phase Transfer Protocol

To mitigate these issues and ensure stable transfer between curriculum phases, a structured transfer protocol is often necessary. The methodology employed in this thesis (and detailed in Chapters 8 and 9) utilizes a **Two-Phase Transfer Protocol** designed to ensure a smooth adaptation of the agent's internal representations to the new task environment, addressing both the stability gap and the risk of catastrophic forgetting.

5.5.1 Phase 1: Critic Adaptation with a Frozen Policy

The primary goal of the first phase is to address the stability gap by stabilizing the value function estimate before attempting to update the policy. Immediately following the transition to the new task environment, the policy network (actor) is frozen. The agent continues to act according to its previously learned policy, generating experience trajectories under the new dynamics and reward structure.

This experience is used solely to update the value function network (critic). By freezing the actor, we isolate the critic adaptation process. The critic learns to accurately predict the expected returns of the existing policy in the new environment. This phase continues until the critic's loss function converges, ensuring that subsequent policy updates will be based on reliable advantage estimates.

5.5.2 Phase 2: Regularized Joint Fine-Tuning

Once the critic has stabilized, the second phase begins. The policy network is unfrozen, and both the actor and the critic are fine-tuned jointly using the standard PPO algorithm.

Crucially, this fine-tuning phase must be appropriately regularized to prevent catastrophic forgetting. PPO inherently provides this regularization through its clipped surrogate objective (Eq. 3.8), which limits the divergence from the previous policy.

Additionally, hyperparameters such as the learning rate are often reduced during fine-tuning. This encourages smooth adaptation to the new task while preserving the core skills learned in previous stages of the curriculum.

5.6 Conclusion and Path Forward

Curriculum Learning provides an essential framework for tackling complex RL problems that are intractable via direct training. By structuring the learning process, CL guides exploration and enables the agent to acquire sophisticated skills incrementally. However, the methodologies developed in this thesis demonstrate that even a highly skilled agent trained via CL remains fundamentally a reactive policy.

In highly dynamic, resource-constrained environments, purely reactive behavior is often insufficient. The agent lacks the explicit foresight to reason about the future interactions of independent system components and proactively avoid situations that lead to unavoidable failure. This "strategic myopia" motivates the need for architectures that can synthesize the learned skills of the RL agent with the deliberative capabilities of planning, the subject of the next chapter.

6 Hybrid Control Architectures

The preceding chapters have established a comprehensive theoretical framework for training sophisticated, model-free reinforcement learning agents. We have explored how Proximal Policy Optimization provides a stable learning algorithm (Chapter 3), and how advanced strategies like Reward Engineering (Chapter 4) and Curriculum Learning (Chapter 5) can guide an agent through complex problem spaces to acquire high levels of skill.

However, a class of problems exists for which even a well-trained, purely reactive policy is insufficient. These are problems that demand not just skill, but strategic foresight—an ability to anticipate and reason about the future consequences of actions. This is particularly critical in environments characterized by multiple interacting components, stochastic dynamics, and hard safety constraints, such as the resource allocation problem central to this thesis.

This chapter explores a different class of solutions: **hybrid control architectures**. These systems move beyond a monolithic, model-free paradigm by synthesizing the adaptive power of reinforcement learning with the deliberative capabilities of model-based planning. We will begin by formally defining the limitations of purely reactive policies, framing them within the context of strategic myopia. Subsequently, we will investigate the use of learned predictive models as a practical alternative to full world modeling and online search. Finally, we will survey the architectural patterns for integrating these predictive models as proactive safety layers and discuss how the resulting high-fidelity agents can be leveraged not just for control, but for system-level engineering analysis.

6.1 The Limits of Reactive Policies: Strategic Myopia

A standard model-free reinforcement learning agent, at its core, learns a reactive policy, $\pi(a|s)$. This policy is a direct mapping from the current observed state to an action. The policy's decisions are conditioned on the expected future returns crystallized within the learned parameters of a value function or policy network during training.

While powerful, this approach has an inherent limitation: the agent's decisions are based on patterns observed in past trajectories, not on an explicit model of the world's causal structure. The agent does not explicitly reason about how the

environment will evolve in response to its actions. This can lead to a critical failure mode that can be described as **strategic myopia**.

The Challenge of Multi-Component Systems. Strategic myopia is particularly problematic in environments that are not monolithic but are composed of multiple, independently evolving components. In such settings, the optimal action often depends on the complex interplay between these components, especially when they compete for shared resources.

The ContainerGym environment (detailed in Chapter 7) exemplifies this challenge. Multiple containers fill independently and stochastically, yet they all compete for a scarce, shared Processing Unit (PU). A purely reactive agent might learn a locally optimal strategy for a single container (e.g., "wait until the optimal volume is reached before emptying"). However, it lacks the foresight to predict that multiple containers might reach their optimal volumes simultaneously in the near future.

A model-free agent cannot easily reason about how its present action (or inaction) regarding one container will influence the independent evolution of other containers, and how that, in turn, will constrain its own available actions in the future. This inability to anticipate and proactively mitigate resource contention can lead to situations where systemic safety failures (e.g., overflows due to PU unavailability) become unavoidable.

6.2 Augmenting Policies with Planning

To overcome strategic myopia, the agent requires a mechanism for lookahead—the ability to simulate potential future scenarios and evaluate their desirability before committing to an action. This is the domain of **planning**.

In classical planning and Model-Based RL (MBRL), an agent utilizes a model of the environment's dynamics (often termed a "world model") to search for an optimal sequence of actions. This involves generating hypothetical trajectories and evaluating their outcomes. Prominent methods include:

- **Monte Carlo Tree Search (MCTS):** MCTS systematically explores the state space by building a search tree of possible future states and actions. It uses the environment model to simulate trajectories (rollouts) from the current state and statistically evaluates the outcomes to select the best action [4]. MCTS was famously used in AlphaGo [27].
- **Model Predictive Control (MPC):** Widely used in control theory, MPC uses a dynamic model of the system to forecast its future behavior over a finite time horizon. It then optimizes a sequence of control inputs to minimize a cost

function over that horizon, executing the first action and re-planning at the next step [3].

6.2.1 The Cost and Constraints of Online Planning

While integrating full-scale online planning methods like MCTS or MPC can provide significant foresight, it presents substantial challenges in real-world industrial settings:

1. **Computational Latency:** Online planning requires significant computation at each decision step to perform the search or optimization. In high-frequency or time-sensitive control tasks, the time required for planning may exceed the available decision time budget, making real-time deployment infeasible.
2. **Model Accuracy and Exploitation:** The effectiveness of planning is entirely dependent on the accuracy of the world model. In complex, stochastic, and partially observable industrial environments, acquiring or learning a sufficiently accurate model of the full system dynamics is often extremely difficult. Furthermore, planning algorithms tend to exploit inaccuracies or abstractions in the model. Errors in the model can lead the planner to devise strategies that appear optimal within the model but are suboptimal or unsafe in the real environment (model exploitation).

6.3 Lookahead via Learned Predictive Models

The constraints of real-time control and the difficulty of learning comprehensive world models motivate a more targeted and efficient approach to incorporating foresight. Instead of learning a full dynamic model of the environment and performing expensive online search, we can train specialized predictive models designed to answer specific, high-value questions about the future.

6.3.1 Specialized Event Predictors

A full generative world model must predict the entire next state s' given (s, a). This is a high-dimensional prediction task. In contrast, a specialized predictor is trained to forecast the probability of a specific future event occurring within a given time horizon.

For example, in a safety-critical system, the predictor might estimate $P(\text{Failure}|s, a)$, the probability of a constraint violation or collision given the current state and a proposed action. By focusing only on the events relevant to the control decision (e.g., safety or resource availability), these models can be significantly simpler, more

robust to noise, and computationally efficient enough for real-time inference. This approach is realized in this thesis through the development of a specialized **Collision Model** (CM), detailed in Chapter 9.

6.3.2 Offline Data Generation via Monte Carlo Simulation

Training an effective specialized predictor requires a rich dataset that adequately covers the relevant regions of the state space, particularly those near safety boundaries or critical decision points. Generating this data directly from interaction with the real physical environment can be prohibitively time-consuming, expensive, or dangerous, especially when collecting data on failure events.

Monte Carlo simulation provides a powerful tool for generating this training data offline. By leveraging a high-fidelity simulator of the environment (such as ContainerGym, Chapter 7), one can simulate a vast number of scenarios, including rare failure events, without risking the physical system. This allows for the generation of a comprehensive dataset capturing the complex dependencies, interactions, and stochasticity of the system, enabling the training of accurate and robust predictive models.

6.4 Inference-Time Intervention and Hybrid Architectures

Once a specialized predictive model is trained, it can be integrated into the control architecture to actively monitor and intervene in the decision-making process. This creates a hybrid system where the RL agent proposes actions, and the predictive model evaluates and potentially overrides them. This strategy is known as **inference-time intervention**.

6.4.1 The Learner-Verifier Framework

This architecture, often conceptualized as a Learner-Verifier system, consists of two primary, decoupled components:

1. **The Learner (RL Agent):** A model-free agent (e.g., PPO-CL) trained to optimize for primary performance objectives (e.g., throughput, efficiency).
2. **The Verifier (Predictive Model/Safety Layer):** A specialized model that evaluates the safety or long-term consequences of the actions proposed by the Learner.

At inference time, the Learner proposes an action a_L. The Verifier then assesses this action (or the resulting state). If the Verifier deems the action unsafe (i.e., predicting a high probability of a future adverse event), the system overrides the Learner's proposal and executes a safe fallback action or a corrective maneuver.

This hybrid approach offers a significant advantage by decoupling the optimization of performance from the enforcement of safety constraints. The Learner can aggressively pursue performance, while the Verifier ensures the system remains within a safe operational envelope.

6.4.2 Connection to Safe Reinforcement Learning

This framework falls under the broader umbrella of **Safe Reinforcement Learning (Safe RL)**. Safe RL is a subfield concerned with developing algorithms that ensure the agent respects safety constraints during both execution (inference) and the learning process itself [8]. While many Safe RL methods attempt to incorporate constraints directly into the RL objective (e.g., via constrained optimization or heavy penalties), the Learner-Verifier hybrid architecture provides a practical, modular, and robust mechanism for enforcing operational safety in complex industrial environments where failure modes are difficult to model analytically.

6.5 System-Level Analysis and Design Insights

The integration of RL and predictive modeling yields benefits that extend beyond the development of the control policy itself.

6.5.1 The Hybrid Agent as a Robust Controller

The resulting hybrid architecture represents a state-of-the-art control solution that combines the strengths of both paradigms. It leverages the skill and adaptability of a curriculum-trained RL agent for optimizing complex objectives, while utilizing the foresight of a specialized predictive model to ensure safety and manage long-term consequences in resource-constrained scenarios. This synthesis results in a controller that is both high-performing and robust to the complexities of real-world industrial dynamics.

6.5.2 System Analysis for Engineering Design

Furthermore, the development of these high-fidelity control agents (both purely RL-based and hybrid) offers significant value for system-level engineering analysis. When evaluated within a realistic simulation environment, these agents become powerful tools for probing the system's dynamics.

By evaluating the performance of different control strategies across a range of environmental parameters (e.g., varying levels of stochasticity, different resource allocations), we can gain deep insights into the system's behavior, identify bottlenecks, and quantify the impact of potential design choices.

For example, by systematically varying the ratio of containers to processing units in the ContainerGym simulator, we can determine the practical limits of scalability under different control policies. This provides data-driven recommendations for facility configuration (as demonstrated in the analysis in Chapter 9). This feedback loop between control optimization and system design is a key benefit of applying advanced AI techniques to industrial problems.

6.6 Conclusion

The preceding chapters (Part I) have established the theoretical foundations and advanced methodologies required to address the complex challenges of optimizing industrial processes using Reinforcement Learning. We have explored the fundamentals of MDPs (Chapter 2), the mechanics of the core algorithm PPO (Chapter 3), and the critical roles of Reward Engineering (Chapter 4), Curriculum Learning (Chapter 5), and Hybrid Architectures (Chapter 6) in bridging the gap between theory and real-world application.

The next part of the thesis (Part II) presents the core contributions of this research, structured around three peer-reviewed publications that detail the progressive development and validation of novel solutions for the container management problem.

- **Chapter 7** introduces and validates the **ContainerGym** benchmark, formalizing the industrial problem and establishing the limitations of baseline RL methods.
- **Chapter 8** demonstrates the synthesis of PPO with sophisticated **Reward Engineering** and **Curriculum Learning** (PPO-CL) to solve a complex, multi-objective configuration of the task.
- **Chapter 9** integrates the PPO-CL agent with an inference-time collision model, creating a **Hybrid RL-Planning architecture** (PPO-CL-CM) to address safety and collision avoidance in high-contention scenarios.

These chapters represent the culmination of the theoretical concepts discussed, applied to a challenging, real-world industrial optimization problem.

Core Contributions and Publications

7 Benchmark Environment

7.1 Context and Motivation

The foundational chapters of this thesis have highlighted the significant challenges inherent in applying Reinforcement Learning to real-world industrial control problems, particularly those involving resource allocation under uncertainty (Chapter 2). As discussed in Section 2.4.2.2, the complexity and stochasticity of such environments often preclude the use of traditional methods like Dynamic Programming, necessitating the use of sample-based RL algorithms.

However, the development and evaluation of RL algorithms for these domains have historically been hampered by a lack of standardized, realistic benchmark environments. Many existing benchmarks are often overly simplified or fail to capture the critical challenges—such as noisy observations, resource scarcity, and safety constraints—that characterize industrial applications.

This chapter addresses this gap by introducing **ContainerGym**, a novel, open-source RL benchmark environment derived directly from the real-world container management problem described in Section 1.3. This environment serves as the foundational simulation platform for all subsequent research presented in this thesis.

The following publication details the formalization of the industrial task as an MDP, the design and implementation of the ContainerGym environment, and a comprehensive evaluation of standard RL algorithms (PPO, TRPO, DQN). The results establish a crucial baseline, identifying the specific limitations of these algorithms when faced with the inherent complexities of the task. This benchmarking process validates the environment and provides the direct motivation for the advanced methodologies (Curriculum Learning and Hybrid Architectures) developed in the subsequent chapters.

7.2 ContainerGym: A Real-World Reinforcement Learning Benchmark for Resource Allocation

`ContainerGym`: A Real-World Reinforcement Learning Benchmark for Resource Allocation

Abhijeet Pendyala, Justin Dettmer, Tobias Glasmachers, and Asma Atamna

Ruhr-University Bochum, Bochum, Germany
`firstname.lastname@ini.rub.de`

Abstract. We present `ContainerGym`, a benchmark for reinforcement learning inspired by a real-world industrial resource allocation task. The proposed benchmark encodes a range of challenges commonly encountered in real-world sequential decision making problems, such as uncertainty. It can be configured to instantiate problems of varying degrees of difficulty, e.g., in terms of variable dimensionality. Our benchmark differs from other reinforcement learning benchmarks, including the ones aiming to encode real-world difficulties, in that it is a direct derivative of a real-world industrial problem, which underwent minimal simplification and streamlining. It is sufficiently versatile to evaluate reinforcement learning algorithms on any real-world problem that fits our resource allocation framework. We provide results of standard baseline methods. Going beyond the usual training reward curves, our results and the statistical tools used to interpret them allow to highlight interesting limitations of well-known deep reinforcement learning algorithms, namely PPO, TRPO and DQN.

Keywords: Deep reinforcement learning, Real-world benchmark, Resource allocation.

1 Introduction

Supervised learning has long made its way into many industries, but industrial applications of (deep) reinforcement learning (RL) are significantly rare. This may be for many reasons, like the focus on impactful RL success stories in the area of games, a lower degree of technology readiness, and a lack of industrial RL benchmark problems.

A RL agent learns by taking sequential actions in its environment, observing the state of the environment, and receiving rewards [15]. Reinforcement learning aims to fulfill the enticing promise of training a smart agent that solves a complex task through trial-and-error interactions with the environment, without specifying how the goal will be achieved. Great strides have been made in this direction, also in the real world, with notable applications in domains like robotics [6], autonomous driving [10, 6], and control problems such as optimizing the power efficiency of

data centers [8], control of nuclear fusion reactors [4], and optimizing gas turbines [3].

Yet, the accelerated progress in these areas has been fueled by making agents play in virtual gaming environments such as Atari 2600 games [9], the game of GO [14], and complex video games like Starcraft II [17]. These games provided sufficiently challenging environments to quickly test new algorithms and ideas, and gave rise to a suite of RL benchmark environments. The use of such environments to benchmark agents for industrial deployment comes with certain drawbacks. For instance, the environments either may not be challenging enough for the state-of-the-art algorithms (low dimensionality of state and action spaces) or require significant computational resources to solve. Industrial problems deviate widely from games in many further properties. Primarily, exploration in real-world systems often has strong safety constraints, and constitutes a balancing act between maximizing reward with good actions which are often sparse, and minimizing potentially severe consequences from bad actions. This is in contrast to training on a gaming environment, where the impact of a single action is often smaller, and the repercussions of poor decisions accumulate slowly over time. In addition, underlying dynamics of gaming environments—several of which are near-deterministic—may not reflect the stochasticity of a real industrial system. Finally, the available environments may have a tedious setup procedure with restrictive licensing and dependencies on closed-source binaries.

To address these issues, we present `ContainerGym`, an open-source real-world benchmark environment for RL algorithms. It is adapted from a digital twin of a high throughput processing industry. Our concrete use-case comes from a waste sorting application. Our benchmark focuses on two phenomena of general interest: First, a stochastic model for a resource-filling process, where certain material is being accumulated in multiple storage containers. Second, a model for a resource transforming system, which takes in the material from these containers, and transforms it for further post-processing downstream. The processing units are a scarce resource, since they are large and expensive, and due to limited options of conveyor belt layout, only a small number can be used per plant. `ContainerGym` is not intended to be a perfect replica of a real system but serves the same hardness and complexity. The search for an optimal sequence of actions in `ContainerGym` is akin to solving a dynamic resource allocation problem for the resource-transforming system, while also learning an optimal control behavior of the resource-filling process. In addition, the complexity of the environment is customizable. This allows testing the limitations of any given learning algorithm. This work aims to enable RL practitioners to quickly and reliably test their learning agents on an environment encoding real-world dynamics.

The paper is arranged as follows. Section 2 discusses the relevant literature and motivates our contribution. Section 3 gives a brief introduction to reinforcement learning preliminaries. In Section 4, we present the real-world industrial control task

that inspired `ContainerGym` and discuss the challenges it presents. In Section 5, we formulate the real-world problem as a RL one and discuss the design choices that lead to our digital twin. We briefly present `ContainerGym`'s implementation in Section 6. We present and discuss benchmark experiments of baseline methods in Section 6, and close with our conclusions in Section 7.

2 Related Work

The majority of the existing open-source environments on which novel reinforcement learning algorithms could be tuned can be broadly divided into the following categories: toy control, robotics (MuJoCo) [16], video games (Atari) [9], and autonomous driving. The underlying dynamics of these environments are artificial and may not truly reflect real-world dynamic conditions like high dimensional states and action spaces, and stochastic dynamics. To the best of our knowledge, there exist very few such open-source benchmarks for industrial applications. To accelerate the deployment of agents in the industry, there is a need for a suite of RL benchmarks inspired by real-world industrial control problems, thereby making our benchmark environment, `ContainerGym`, a valuable addition to it.

The classic control environments like mountain car, pendulum, or toy physics control environments based on Box2D are stochastic only in terms of their initial state. They have low dimensional state and action spaces, and they are considered easy to solve with standard methods. Also, the 50 commonly used Atari games in the Arcade Learning Environment [1], where nonlinear control policies need to be learned, are routinely solved to a super-human level by well-established algorithms. This environment, although posing high dimensionality, is deterministic. The real world is not deterministic and there is a need to tune algorithms that can cope with stochasticity. Although techniques like sticky actions or skipping a random number of initial frames have been developed to add artificial randomness, this randomness may still be very structured and not challenging enough. On the other hand, video game simulators like Starcraft II [17] offer high-dimensional image observations, partial observability, and (slight) stochasticity. However, playing around with DeepRL agents on such environments requires substantial computational resources. It might very well be overkill to tune reinforcement learning agents in these environments when the real goal is to excel in industrial applications.

Advancements in the RL world, in games like Go and Chess, were achieved by exploiting the rules of these games and embedding them into a stochastic planner. In real-world environments, this is seldom achievable, as these systems are highly complex to model in their entirety. Such systems are modeled as partially observable Markov decision processes and present a tough challenge for learning agents that can explore only through interactions. Lastly, some of the more sophisticated RL environments available, e.g., advanced physics simulators like MuJoCo [16], offer

licenses with restrictive terms of use. Also, environments like Starcraft II require access to a closed-source binary. Open source licensing in an environment is highly desirable for RL practitioners as it enables them to debug the code, extend the functionality and test new research ideas.

Other related works, amongst the very few available open-source reinforcement learning environments for industrial problems, are Real-world RL suite [5] and Industrial benchmark (IB) [7] environments. Real-world RL-suite is not derived from a real-world scenario, rather the existing toy problems are perturbed to mimic the conditions in a real-world problem. The IB comes close in spirit to our work, although it lacks the customizability of `ContainerGym` and our expanded tools for agent behavior explainability. Additionally, the (continuous) action and state spaces are relatively low dimensional and of fixed sizes.

3 Reinforcement Learning Preliminaries

RL problems are typically studied as discrete-time Markov decision processes (MDPs), where a MDP can be defined as a tuple $\langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$. At timestep t, the agent is in a state $s_t \in \mathcal{S}$ and takes an action $a_t \in \mathcal{A}$. It arrives in a new state s_{t+1} with probability $p(s_{t+1} \mid s_t, a_t)$ and receives a reward $r(s_t, a_t, s_{t+1})$. The state transitions of a MDP satisfy the Markov property $p(s_{t+1} \mid s_t, a_t, \dots, s_0, a_0) = p(s_{t+1} \mid s_t, a_t)$. That is, the new state s_{t+1} only depends on the current state s_t and action a_t. The goal of the RL agent interacting with the MDP is to find a policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximizes the expected (discounted) cumulative reward. This optimization problem is defined formally as follows:

$$\arg\max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t \geq 0} \gamma^t r(s_t, a_t, s_{t+1}) \right] , \tag{7.1}$$

where $\tau = (s_0, a_0, r_0, s_1, \dots)$ is a trajectory generated by following π and $\gamma \in (0, 1]$ is a discount factor. A trajectory τ ends either when a maximum timestep count T—also called episode[1] length—is reached, or when a terminal state is reached (early termination).

4 Container Management Environment

In this section, we describe the real-world industrial control task that inspired our RL benchmark. It originates from the final stage of a waste sorting process.

[1] We use the terms "episode" and "rollout" interchangeably in this paper.

The environment consists of a solid material-transforming facility that hosts a set of *containers* and a significantly smaller set of *processing units* (PUs). Containers are continuously filled with material, where the material flow rate is a container-dependent stochastic process. They must be emptied regularly so that their content can be transformed by the PUs. When a container is emptied, its content is transported on a conveyor belt to a free PU that transforms it into *products*. It is not possible to extract material from a container without emptying it completely. The number of produced products depends on the volume of the material being processed. Ideally, this volume should be an integer multiple of the product's size. Otherwise, the surplus volume that cannot be transformed into a product is redirected to the corresponding container again via an energetically costly process that we do not consider in this work. Each container has at least one optimal emptying volume: a global optimum and possibly other, smaller, local optima that are all multiples of container-specific product size. Generally speaking, larger volumes are better, since PUs are more efficient with producing many products in a series.

The worst-case scenario is a container overflow. In the waste sorting application inspiring our benchmark, it incurs a high recovery cost including human intervention to stop and restart the facility. This situation is undesirable and should be actively avoided. Therefore, letting containers come close to their capacity limit is rather risky.

The quality of an emptying decision is a compromise between the number of potential products and the costs resulting from actuating a PU and handling surplus volume. Therefore, the closer an emptying volume is to an optimum, the better. If a container is emptied too far away from any optimal volume, the costs outweigh the benefit of producing products.

This setup can be framed more broadly as a resource allocation problem, where one item of a scarce resource, namely the PUs, needs to be allocated whenever an emptying decision is made. If no PU is available, the container cannot be emptied and will continue to fill up.

There are multiple aspects that make this problem challenging:

- The rate at which the material arrives at the containers is stochastic. Indeed, although the volumes follow a globally linear trend—as discussed in details in Section 5, the measurements can be very noisy. The influx of material is variable, and there is added noise from the sensor readings inside the containers. This makes applying standard planning approaches difficult.
- The scarcity of the PUs implies that always waiting for containers to fill up to their ideal emptying volume is risky: if no PU is available at that time, then we risk an overflow. This challenge becomes more prominent when the number of containers—in particular, the ratio between the number of containers and the number of PUs—increases. Therefore, an optimal policy needs to take fill

states and fill rates of all containers into account, and possibly empty some containers early.
- Emptying decisions can be taken at any time, but in a close-to-optimal policy, they are rather infrequent. Also, the rate at which containers should be emptied varies between containers. Therefore, the distributions of actions are highly asymmetric, with important actions (and corresponding rewards) being rare.

5 Reinforcement Learning Problem Formulation

We formulate the container management problem presented in Section 4 as a MDP that can be addressed by RL approaches. The resulting MDP is an accurate representation of the original problem, as only mild simplifications are made. Specifically, we model one category of PUs instead of two, we do not include inactivity periods of the facility in our environment, and we neglect the durations of processes of minor relevance. All parameters described in the rest of this section are estimated from real-world data (system identification). Overall, the MDP reflects the challenges properly without complicating the benchmark (and the code base) with irrelevant details.

5.1 State and Action Spaces

5.1.1 State space

The state s_t of the system at any given time t is defined by the volumes $v_{i,t}$ of material contained in each container C_i, and a timer $p_{j,t}$ for each PU P_j indicating in how many seconds the PU will be ready to use. A value of zero means that the PU is available, while a positive value means that it is currently busy. Therefore, $s_t = (\{v_{i,t}\}_{i=1}^{n}, \{p_{j,t}\}_{j=1}^{m})$, where n and m are the number of containers and PUs, respectively. Valid initial states include non-negative volumes not greater than the maximum container capacity $v_{\max}$, and non-negative timer values.

5.1.2 Action space

Possible actions at a given time t are either (i) not to empty any container, i.e. to do nothing, or (ii) to empty a container C_i and transform its content using one of the PUs. The action of doing nothing is encoded with 0, whereas the action of emptying a container C_i and transforming its content is encoded with the container's index i. Therefore, $a_t \in \{0, 1, \ldots, n\}$, where n is the number of containers. Since we consider identical PUs, specifying which PU is used in the action encoding is not necessary

as an emptying action will result in the same state for all the PUs, given they were at the same previous state.

5.2 Environment Dynamics

In this section, we define the dynamics of the volume of material in the containers, the PU model, as well as the state update.

5.2.1 Volume dynamics

The volume of material in each container increases following an irregular trend, growing linearly on average, captured by a random walk model with drift. That is, given the current volume $v_{i,t}$ contained in C_i, the volume at time $t+1$ is given by the function f_i defined as

$$f_i(v_{i,t}) = \max(0, \alpha_i + v_{i,t} + \epsilon_{i,t}) \ , \tag{7.2}$$

where $\alpha_i > 0$ is the slope of the linear upward trend followed by the volume for C_i and the noise $\epsilon_{i,t}$ is sampled from a normal distribution $\mathcal{N}(0, \sigma_i^2)$ with mean 0 and variance σ_i^2. The max operator forces the volume to non-negative values.

When a container C_i is emptied, its volume drops to 0 at the next timestep. Although the volume drops progressively to 0 in the real facility, empirical evidence provided by our data shows that emptying durations are within the range of the timestep lengths considered in this paper (60 and 120 seconds).

5.2.2 Processing unit dynamics

The time (in seconds) needed by a PU to transform a volume v of material in a container C_i is linear in the number of products $\lfloor v/b_i \rfloor$ that can be produced from v. It is given by the function g_i defined in Equation (8.3), where $b_i > 0$ is the product size, $\beta_i > 0$ is the time it takes to actuate the PU before products can be produced, and $\lambda_i > 0$ is the time per product. Note that all the parameters indexed with i are container-dependent.

$$g_i(v) = \beta_i + \lambda_i \lfloor v/b_i \rfloor \ . \tag{7.3}$$

A PU can only produce one type of product at a time. Therefore, it can be used for emptying a container only when it is free. Therefore, if all PUs are busy, the container trying to use one is not emptied and continues to fill up.

5.2.3 State update

We distinguish between the following cases to define the new state $s_{t+1} = (\{v_{i,t+1}\}_{i=1}^{n}, \{p_{j,t+1}\}_{j=1}^{m})$ given the current state s_t and action a_t.

$a_t = 0$. This corresponds to the action of doing nothing. The material volumes inside the containers increase while the timers indicating the availability of the PUs are decreased according to:

$$v_{i,t+1} = f_i(v_{i,t}), \qquad i \in \{1, \dots, n\} \ , \tag{7.4}$$

$$p_{j,t+1} = \max(0, p_{j,t} - \delta), \qquad j \in \{1, \dots, m\} \ , \tag{7.5}$$

where f_i is the random walk model defined in Equation (9.1) and δ is the length of a timestep in seconds.

$a_t \neq 0$. This corresponds to an emptying action. If no PU is available, that is, $p_{j,t} > 0 \ \forall j = 1, \dots, m$, the updates are identical to the one defined in Equations (7.4) and (7.5). If at least one PU P_k is available, the new state variables are defined as follows:

$$v_{a_t,t+1} = 0 \ , \tag{7.6}$$

$$v_{i,t+1} = f_i(v_{i,t}), \qquad i \in \{1, \dots, n\} \backslash \{a_t\} \ , \tag{7.7}$$

$$p_{k,t+1} = g_{a_t}(v_{a_t,t}) \ , \tag{7.8}$$

$$p_{j,t+1} = \max(0, p_{j,t} - \delta), \qquad j \in \{1, \dots, m\} \backslash \{k\} \ , \tag{7.9}$$

where the value of the action a_t is the index of the container C_{a_t} to empty, '\' denotes the set difference operator, f_i is the random walk model defined in Equation (9.1), and g_{a_t} is the processing time defined in Equation (8.3).

Although the processes can continue indefinitely, we stop an episode after a maximum length of T timesteps. This is done to make `ContainerGym` compatible with RL algorithms designed for episodic tasks, and hence to maximize its utility. When a container reaches its maximum volume $v_{\max}$, however, the episode is terminated and a negative reward is returned (see details in Section 5.3).

5.3 Reward Function

We use a deterministic reward function $r(s_t, a_t, s_{t+1})$ where higher values correspond to better (s_t, a_t) pairs. The new state s_{t+1} is taken into account to return a large negative reward $r_{\min}$ when a container overflows, i.e. $\exists i \in \{1, \dots, n\}$, $v_{i,t+1} \geq v_{\max}$, before ending the episode. In all other cases, the immediate reward is determined only by the current state s_t and the action a_t.

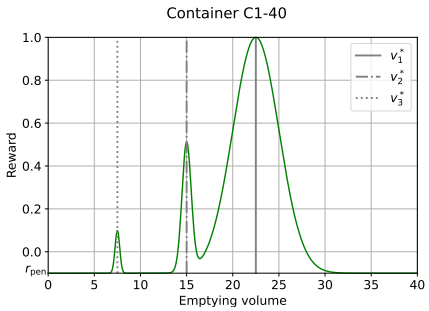


Figure 7.1: Rewards received when emptying a container with three optima. The design of the reward function fosters emptying late, hence allowing PUs to produce many products in a row.

$\boldsymbol{a_t = 0}$. If the action is to do nothing, we define $r(s_t, 0, s_{t+1}) = 0$.

$\boldsymbol{a_t \neq 0}$. If an emptying action is selected while (i) no PU is available or (ii) the selected container is already empty, i.e. $v_{a_t,t} = 0$, then $r(s_t, a_t, s_{t+1}) = r_{\text{pen}}$, where it holds $r_{\min} < r_{\text{pen}} < 0$ for the penalty. If, on the other hand, at least one PU is available and the volume to process is non-zero ($v_{a_t,t} > 0$), the reward is a finite sum of Gaussian functions centered around optimal emptying volumes $v^*_{a_t,i}, i = 1, \ldots, p_{a_t}$, where the height of a peak $0 < h_{a_t,i} \leq 1$ is proportional to the quality of the corresponding optimum. The reward function in this case is defined as follows:

$$r(s_t, a_t, s_{t+1}) = r_{\text{pen}} + \sum_{i=1}^{p_{a_t}} (h_{a_t,i} - r_{\text{pen}}) \exp\left(-\frac{(v_{a_t,t} - v^*_{a_t,i})^2}{2w^2_{a_t,i}}\right) \quad , \tag{7.10}$$

where p_{a_t} is the number of optima for container C_{a_t} and $w_{a_t,i} > 0$ is the width of the bell around $v^*_{a_t,i}$. The Gaussian reward defined in Equation (7.10) takes its values in $]r_{\text{pen}}, 1]$, the maximum value 1 being achieved at the ideal emptying volume $v^*_{a_t,1}$ for which $h_{a_t,1} = 1$. The coefficients $h_{a,i}$ are designed so that processing large volumes at a time is beneficial, hence encoding a conflict between emptying containers early and risking overflow. This tension, together with the limited availability of the PUs, yields a highly non-trivial control task. Figure 7.1 shows an example of Gaussian rewards when emptying a container with three optimal volumes at different volumes in $[0, 40[$.

6 `ContainerGym` Usage Guide

In this section, we introduce the OpenAI Gym implementation[2] of our benchmark environment. We present the customizable parameters of the benchmark, provide an outline for how the Python implementation reflects the theoretical definition in Section 5, and show which tools we provide to extend the understanding of an agent's behavior beyond the achieved rewards.

Gym implementation. Our environment follows the OpenAI Gym [2] framework. It implements a `step()`-function computing a state update. The `reset()`-function resets time to $t = 0$ and returns the environment to a valid initial state, and the `render()`-function displays a live diagram of the environment.

We deliver functionality to configure the environment's complexity and difficulty through its parameters such as the number of containers and PUs, as well as the composition of the reward function, the overflow penalty r_{min}, the sub-optimal emptying penalty r_{pen}, the length of a timestep δ, and the length of an episode T. We provide example configurations in our GitHub repository that are close in nature to the real industrial facility.

In the spirit of open and reproducible research, we include scripts and model files for reproducing the experiments presented in the next section.

Action explainability. While most RL algorithms treat the environment as a black box, facility operators want to "understand" a policy before deploying it for production. To this end, the accumulated reward does not provide sufficient information, since it treats all mistakes uniformly. Practitioners need to understand which types of mistakes a sub-optimal policy makes. For example, a low reward can be obtained by systematically emptying containers too early (local optimum) or by emptying at non-integer multiples of the product size. When a basic emptying strategy for each container is in place, low rewards typically result from too many containers reaching their ideal volume at the same time so that PUs are overloaded. To make the different types of issues of a policy transparent, we plot all individual container volumes over an entire episode. We further provide tools to create empirical cumulative distribution function plots over the volumes at which containers were emptied over multiple episodes. The latter plots in particular provide insights into the target volumes an agent aims to hit, and whether it does so reliably.

[2]The `ContainerGym` software is available on the following GitHub repository: `https://github.com/Pendu/ContainerGym`.

7 Performance of Baseline Methods

We illustrate the use of `ContainerGym` by benchmarking three popular deep RL algorithms: two on-policy methods, namely Proximal Policy Optimization (PPO) [12] and Trust Region Policy Optimization (TRPO) [13], and one off-policy method, namely Deep Q-Network (DQN) [9]. We also compare the performance of these RL approaches against a naive rule-based controller. By doing so, we establish an initial baseline on `ContainerGym`. We use the Stable Baselines3 implementation of PPO, TRPO, and DQN [11].

7.1 Experimental Setup

The algorithms are trained on 8 `ContainerGym` instances, detailed in Table 7.2, where the varied parameters are the number of containers n, the number of PUs m and the timestep length δ.[3] The rationale behind the chosen configurations is to assess (i) how the algorithms scale with the environment dimensionality, (ii) how action frequency affects the trained policies and (iii) whether the optimal policy can be found in the conceptually trivial case $m = n$, where there is always a free PU when needed. This is only a control experiment for testing RL performance. In practice, PUs are a scarce resource ($m \ll n$).

The maximum episode length T is set to 1500 timesteps during training in all experiments, whereas the initial volumes $v_{i,0}$ are uniformly sampled in $[0, 30]$, and the maximum capacity is set to $v_{\max} = 40$ volume units for all containers. The initial PU states are set to free ($p_{j,0} = 0$) and the minimum and penalty rewards are set to $r_{\min} = -1$ and $r_{\text{pen}} = -0.1$ respectively. The algorithms are trained with an equal budget of 2 (resp. 5) million timesteps when $n = 5$ (resp. $n = 11$) and the number of steps used for each policy update for PPO and TRPO is 6144. Default values are used for the remaining hyperparameters, as the aim of our work is to show characteristics of the training algorithms with a reasonable set of parameters. For each algorithm, 15 policies are trained in parallel, each with a different seed $\in \{1, \ldots, 15\}$, and the policy with the best training cumulative reward is returned for each run. To make comparison easier, policies are evaluated on a similar test environment with $\delta = 120$ and $T = 600$.

7.2 Results and Discussion

Table 7.2 shows the average test cumulative reward, along with its standard deviation, for the best and median policies trained with PPO, TRPO, and DQN on each

[3] Increasing the timestep length δ should be done carefully. Otherwise, the problem could become trivial. In our case, we choose δ such that it is smaller than the minimum time it takes a PU to process the volume equivalent to one product.

environment configuration. These statistics are calculated by running each policy 15 times on the corresponding test environment.

Table 7.2: Test cumulative reward, averaged over 15 episodes, and its standard deviation for the best and median policies for PPO, TRPO and DQN. Best and median policies are selected from a sample of 15 policies (seeds) trained on the investigated environment configurations. The highest best performance is highlighted for each configuration.

Config.			PPO		TRPO		DQN	
n	m	δ	best	median	best	median	best	median
5	2	60	$\mathbf{38.55 \pm 1.87}$	29.94 ± 8.42	37.35 ± 7.13	4.93 ± 4.90	29.50 ± 3.43	1.57 ± 0.93
5	2	120	$\mathbf{51.43 \pm 3.00}$	49.81 ± 1.56	38.33 ± 8.36	16.86 ± 2.78	42.96 ± 2.80	7.90 ± 4.72
5	5	60	$\mathbf{37.54 \pm 1.50}$	30.64 ± 2.44	33.62 ± 7.76	7.36 ± 4.40	23.98 ± 13.17	1.96 ± 1.28
5	5	120	$\mathbf{50.42 \pm 2.57}$	47.23 ± 1.89	47.36 ± 2.07	5.39 ± 4.38	43.26 ± 3.73	8.56 ± 4.12
11	2	60	31.01 ± 8.62	23.25 ± 16.79	26.62 ± 18.94	4.26 ± 3.81	$\mathbf{42.63 \pm 21.73}$	11.84 ± 10.33
11	2	120	54.30 ± 8.48	49.58 ± 13.54	45.78 ± 18.42	32.08 ± 18.04	$\mathbf{72.19 \pm 16.20}$	24.07 ± 13.68
11	11	60	27.87 ± 8.89	17.57 ± 13.99	15.66 ± 10.65	4.90 ± 2.68	$\mathbf{28.32 \pm 22.42}$	8.49 ± 4.92
11	11	120	47.75 ± 10.78	42.37 ± 13.25	$\mathbf{50.55 \pm 11.08}$	34.29 ± 16.00	29.06 ± 13.10	13.16 ± 8.32

PPO achieves the highest cumulative reward on environments with 5 containers. DQN, on the other hand, shows the best performance on higher dimensionality environments (11 containers), with the exception of the last configuration. It has, however, a particularly high variance. Its median performance is also significantly lower than the best one, suggesting less stability than PPO. DQN's particularly high rewards when $n = 11$ could be due to a better exploration of the search space. An exception is the last configuration, where DQN's performance is significantly below those of TRPO and PPO. Overall, PPO has smaller standard deviations and a smaller difference between best and median performances, suggesting higher stability than TRPO and DQN.

Our results also show that taking actions at a lower frequency ($\delta = 120$) leads to better policies. Due to space limitations, we focus on analyzing this effect on configurations with $n = 5$ and $m = 2$. Figure 7.3 displays the volumes, actions, and rewards over one episode for PPO and DQN when the (best) policy is trained with $\delta = 60$ (left column) and with $\delta = 120$.

We observe that with $\delta = 60$, PPO tends to empty containers C1-60 and C1-70 prematurely, which leads to poor rewards. These two containers have the slowest fill rate. Increasing the timestep length, however, alleviates this defect. The opposite effect is observed on C1-60 with DQN, whereas no significant container-specific behavior change is observed for TRPO, as evidenced by the cumulative rewards. To further explain these results, we investigate the empirical cumulative distribution functions (ECDFs) of the emptying volumes per container. Figure 8.8 reveals that more than 90% of PPO's emptying volumes on C1-60 (resp. C1-70) are approximately within a 3 (resp. 5) volume units distance of the third best optimum when $\delta = 60$. By increasing the timestep length, the emptying volumes become more centered around the global optimum. While no such clear pattern is observed with DQN

on containers with slow fill rates, the emptying volumes on C1-60 move away from the global optimum when the timestep length is increased (more than 50% of the volumes are within a 3 volume units distance of the second best optimum). DQN's performance increases when $\delta = 120$. This is explained by the better emptying volumes achieved on C1-20, C1-70, and C1-80.

None of the benchmarked algorithms manage to learn the optimal policy when there are as many PUs as containers, independently of the environment dimensionality and the timestep length. When $m = n$, the optimal policy is known and consists in emptying each container when the corresponding global optimal volume is reached, as there is always at least one free PU. Therefore, achieving the maximum reward at each emptying action is theoretically possible. Figure 7.5a shows the ECDF of the reward per emptying action over 15 episodes of the best policy for each of PPO, TRPO and DQN when $m = n = 11$. The rewards range from $r_{\text{min}} = -1$ to 1, and the ratio of negative rewards is particularly high for PPO. An analysis of the ECDFs of its emptying volumes (not shown) reveals that this is due to containers with slow fill rates being emptied prematurely. TRPO achieves the least amount of negative rewards whereas all DQN's rollouts end prematurely due to a container overflowing. Rewards close to r_{pen} are explained by poor emptying volumes, as well as a bad allocation of the PUs ($r = r_{\text{pen}}$). These findings suggest that, when $m = n$, it is not trivial for the tested RL algorithms to break down the problem into smaller, independent problems, where one container is emptied using one PU when the ideal volume is reached.

The limitations of these RL algorithms are further highlighted when compared to a naive rule-based controller that empties the first container whose volume is less than 1 volume unit away from the ideal emptying volume. Figure 7.5b shows the ECDF of reward per emptying action obtained from 15 rollouts of the rule-based controller on three environment configurations, namely $n = 5$ and $m = 2$, $n = 11$ and $m = 2$, and $n = 11$ and $m = 11$. When compared to PPO in particular ($m = n = 11$), the rule-based controller empties containers less often (17.40% of the actions vs. more than 30% for PPO). Positive rewards are close to optimal (approx. 90% in $[0.75, 1]$), whereas very few negative rewards are observed. These stem from emptying actions taken when no PU is available. These findings suggest that learning to wait for long periods before performing an important action may be challenging for some RL algorithms.

Critically, current baseline algorithms only learn reasonable behavior for containers operated in isolation. In the usual $m \ll n$ case, none of the policies anticipates all PUs being busy when emptying containers at their optimal volume, which they should ideally foresee and prevent proactively by emptying some of the containers earlier. Hence, there is considerable space for future improvements by learning stochastic long-term dependencies. This is apparently a difficult task for state-of-the-art RL algorithms. We anticipate that `ContainerGym` can contribute to research on next-generation RL methods addressing this challenge.

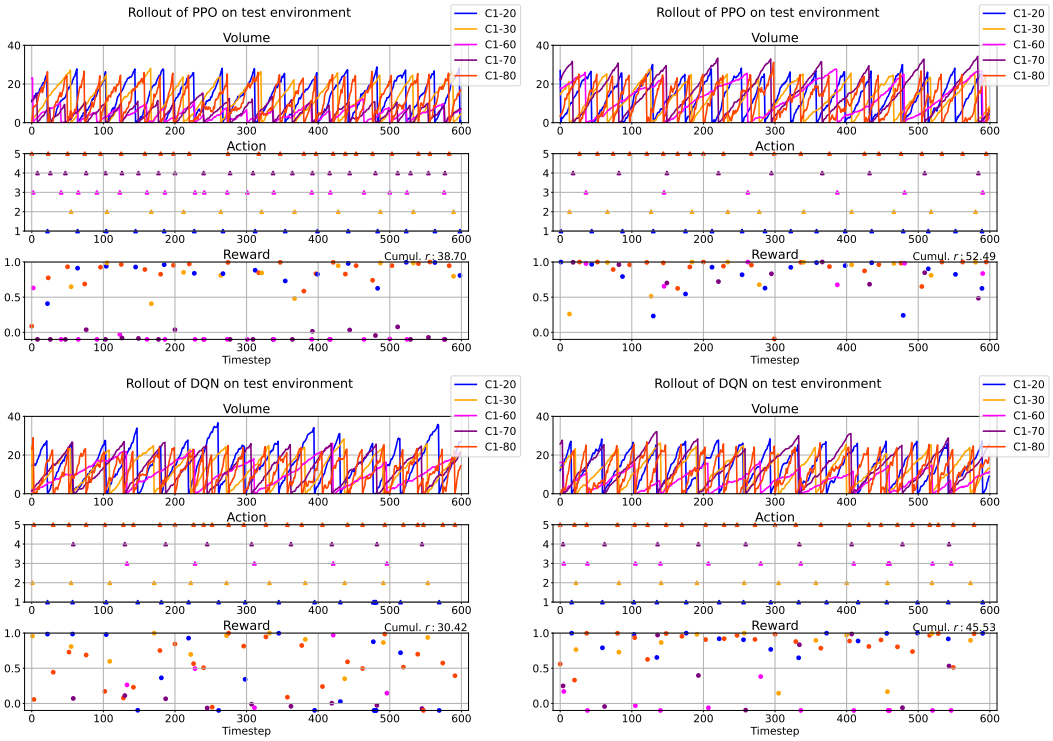


Figure 7.3: Rollouts of the best policy trained with PPO (first row) and DQN (second row) on a test environment with $n = 5$ and $m = 2$. Left: training environment with $\delta = 60$ seconds. Right: training environment with $\delta = 120$. Displayed are the volumes, emptying actions and non-zero rewards at each timestep.

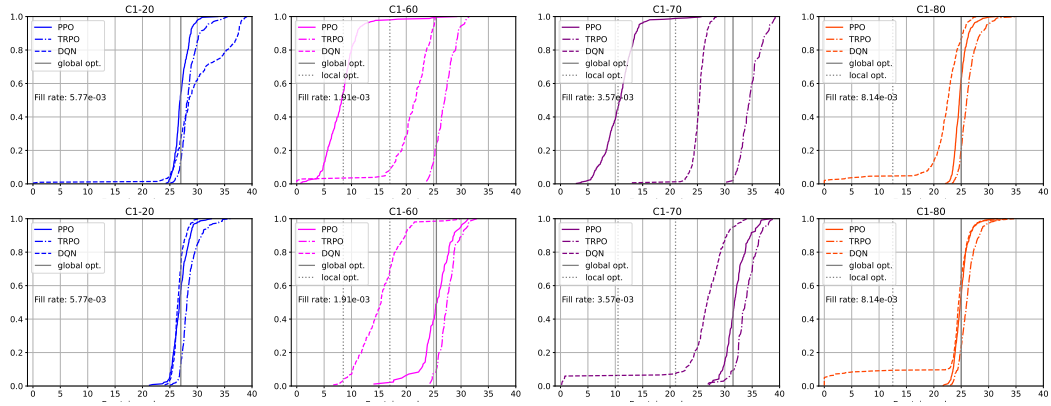
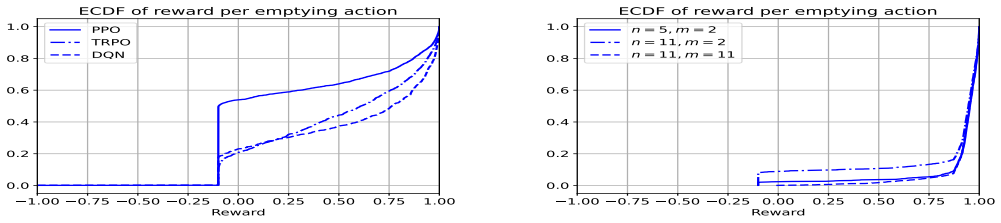


Figure 7.4: ECDFs of emptying volumes collected over 15 rollouts of the best policy for PPO, TRPO, and DQN on a test environment with $n = 5$ and $m = 2$. Shown are 4 containers out of 5. Fill rates are indicated in volume units per second. Top: training environment with $\delta = 60$. Bottom: training environment with $\delta = 120$. The derivatives of the curves are the PDFs of emptying volumes. Therefore, a steep incline indicates that the corresponding volume is frequent in the corresponding density.



(a) (b)

Figure 7.5: ECDF of the reward obtained for each emptying action over 15 rollouts. (a): Best policies for PPO, TRPO and DQN, trained on an environment with $m = n = 11$ and $\delta = 120$. (b): Rule-based controller on three environment configurations.

8 Conclusion

We have presented `ContainerGym`, a real-world industrial RL environment. Its dynamics are quite basic, and therefore easily accessible. It is easily scalable in complexity and difficulty.

Its characteristics are quite different from many standard RL benchmark problems. At its core, it is a resource allocation problem. It features stochasticity, learning agents can get trapped in local optima, and in a good policy, the most important actions occur only rarely. Furthermore, implementing optimal behavior requires planning ahead under uncertainty.

The most important property of `ContainerGym` is to be of direct industrial relevance. At the same time, it is a difficult problem with the potential to trigger novel developments in the future. While surely being less challenging than playing the games of Go or Starcraft II at a super-human level, `ContainerGym` is still sufficiently hard to make state-of-the-art baseline algorithms perform poorly. We are looking forward to improvements in the future.

To fulfill the common wish of industrial stakeholders for explainable ML solutions, we provide insights into agent behavior and deviations from optimal behavior that go beyond learning curves. While accumulated reward hides the details, our environment provides insights into different types of failures and hence gives guidance for routes to further improvement.

Acknowledgements. This work was funded by the German federal ministry of economic affairs and climate action through the "ecoKI" grant.

References

[1] M. G. Bellemare et al. "The Arcade Learning Environment: An Evaluation Platform for General Agents". In: *Journal of Artificial Intelligence Research* 47 (June 2013), pp. 253–279.

[2] Greg Brockman et al. *OpenAI Gym*. 2016. eprint: `arXiv:1606.01540`.

[3] Michele Compare et al. "Reinforcement learning-based flow management of gas turbine parts under stochastic failures". In: *The International Journal of Advanced Manufacturing Technology* 99.9-12 (Sept. 2018), pp. 2981–2992.

[4] Jonas Degrave et al. "Magnetic control of tokamak plasmas through deep reinforcement learning". In: *Nature* 602.7897 (Feb. 2022), pp. 414–419.

[5] Gabriel Dulac-Arnold et al. "An empirical investigation of the challenges of real-world reinforcement learning". In: *CoRR* abs/2003.11881 (2020). arXiv: `2003.11881`.

[6] Tuomas Haarnoja et al. “Soft Actor-Critic Algorithms and Applications”. In: *CoRR* abs/1812.05905 (2018). arXiv: 1812.05905.

[7] Daniel Hein et al. “A benchmark environment motivated by industrial control problems”. In: *2017 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, Nov. 2017.

[8] Nevena Lazic et al. “Data Center Cooling using Model-predictive Control”. In: *Proceedings of the Thirty-second Conference on Neural Information Processing Systems (NeurIPS-18)*. Montreal, QC, 2018, pp. 3818–3827.

[9] Volodymyr Mnih et al. “Playing Atari with Deep Reinforcement Learning”. In: *CoRR* abs/1312.5602 (2013). arXiv: 1312.5602.

[10] Błażej Osiński et al. “Simulation-Based Reinforcement Learning for Real-World Autonomous Driving”. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*. 2020, pp. 6411–6418.

[11] Antonin Raffin et al. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[12] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *CoRR* abs/1707.06347 (2017). arXiv: 1707.06347.

[13] John Schulman et al. “Trust Region Policy Optimization”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. PMLR, 2015, pp. 1889–1897.

[14] David Silver et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587 (2016), pp. 484–489.

[15] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[16] Emanuel Todorov, Tom Erez, and Yuval Tassa. “MuJoCo: A physics engine for model-based control”. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2012, pp. 5026–5033.

[17] Oriol Vinyals et al. “Grandmaster level in StarCraft II using multi-agent reinforcement learning”. In: *Nature* 575.7782 (Oct. 2019), pp. 350–354.

8 Optimizing Container Management with Curriculum Learning and Reward Engineering

8.1 Context and Motivation

The introduction of the ContainerGym benchmark (Chapter 7) provided a formalized environment for studying the industrial container management problem. Crucially, the evaluation of baseline reinforcement learning algorithms (PPO, TRPO, DQN) within that environment revealed significant limitations. Standard, "vanilla" approaches failed to learn effective policies, struggling particularly with the environment's inherent challenges: delayed rewards, the need for long-term planning under uncertainty, strict safety constraints (avoiding overflows), and the complexity of balancing competing objectives.

These baseline failures underscore the necessity of the advanced methodologies discussed in the foundational chapters of this thesis. As established in Chapter 4, a carefully designed reward function is essential to guide the agent effectively in complex landscapes. Furthermore, as discussed in Chapter 5, structuring the learning process itself is often required to overcome the exploration and optimization challenges that paralyze direct training efforts.

This chapter presents the second core contribution of this thesis, detailing the synthesis of these advanced techniques to solve the container management problem. The following publication details the development of a sophisticated, multi-stage Curriculum Learning (CL) strategy combined with meticulous Reward Engineering, applied to the Proximal Policy Optimization (PPO) algorithm.

This approach (PPO-CL) is evaluated on a complex, realistic configuration of the ContainerGym environment (involving 11 containers and 2 distinct processing units), requiring the agent to simultaneously optimize for operational safety, throughput, and energy efficiency. The results demonstrate how this structured approach enables the agent to successfully navigate the multi-criteria landscape, significantly outperforming the baseline methods and establishing a high-performing reactive policy. This work provides the foundation for the hybrid approach developed in the next chapter.

8.2 Solving a Real-World Optimization Problem Using Proximal Policy Optimization with Curriculum Learning and Reward Engineering

Solving a Real-World Optimization Problem Using Proximal Policy Optimization with Curriculum Learning and Reward Engineering

Abhijeet Pendyala, Asma Atamna, Tobias Glasmachers

Ruhr-University Bochum, Bochum, Germany

firstname.lastname@ini.rub.de

Abstract. We present a proximal policy optimization agent trained through curriculum learning (CL) principles and meticulous reward engineering to optimize a real-world high-throughput waste sorting facility. Our work addresses the challenge of effectively balancing the competing objectives of operational safety, volume optimization, and minimizing resource usage. A vanilla agent trained from scratch on these multiple criteria fails to solve the problem due to its inherent complexities. This problem is particularly difficult due to the environment's extremely delayed rewards with long time horizons and class (or action) imbalance, with important actions being infrequent in the optimal policy. This forces the agent to anticipate long-term action consequences and prioritize rare but rewarding behaviours, creating a non-trivial reinforcement learning task. Our five-stage CL approach tackles these challenges by gradually increasing the complexity of the environmental dynamics during policy transfer while simultaneously refining the reward mechanism. This iterative and adaptable process enables the agent to learn a desired optimal policy. Results demonstrate that our approach significantly improves inference-time safety, achieving near-zero safety violations in addition to enhancing waste sorting plant efficiency.

Keywords: Deep reinforcement learning, Real-world task, Curriculum learning, and Sustainable waste management.

1 Introduction

This work introduces a novel reinforcement learning approach to address the critical need for optimization within waste sorting facilities, addressing an uncharted area of research. EU directives emphasize the need for responsible and sustainable recycling of packaging waste. This has led to waste sorting facilities investing in sophisticated infrastructure and automated sorting technologies. Traditional data-driven methods are emerging as key tools within the digitalization of these high-throughput facilities for optimal control. Sorting facilities are designed specifically for different use cases,

such as the types of materials to be separated and the scale and the throughput requirements of the plant. They must remain robust to ever-changing material streams, and they should ideally be capable of adapting to changes of the input composition, and even to future design modifications such as changes in layout or modifying sorting machinery. Additionally, ensuring operational safety while optimizing sorting efficiency remains a key challenge.

The final stage of a waste sorting facility, namely the management of containers filling up with sorted material before being processed into a final product, is readily available as a real-world industrial benchmark RL environment called ContainerGym [7].

This research introduces a curriculum learning approach for training a Proximal Policy Optimization (PPO) algorithm for the container management problem. The approach breaks down the complex learning process into manageable stages, aiming to optimize sorting efficiency and resource utilization within operational constraints.

The motivation behind our approach was that the RL problem at hand is non-trivial. We found that a vanilla baseline like a PPO agent trained from scratch on all the three criteria detailed in section 3 fails to learn a useful control policy. Analysis of training data revealed that the majority of the training episodes terminated prematurely due to violation of safety constraints (volume limit exceeded). Consequently, the agent cannot collect sufficient samples of optimal state-action pairs, trapping them in a sub-optimal solution. Therefore there was a need for a more powerful method. We demonstrate that a nuanced curriculum approach can solve the problem.

2 Related Work

Curriculum learning in the context of RL is a training paradigm where an agent encounters a sequence of tasks carefully designed to accelerate learning. Designing a curriculum allows ML engineers and domain experts to incorporate experience and domain knowledge into the training process. The curriculum might involve modifying reward functions, altering environment dynamics, or changing state or action spaces. By strategically guiding the agent's experiences, CL has the potential to promote faster convergence, better generalization, and more efficient learning [6]. Beyond reinforcement learning, CL has been proven to provide a versatile framework with a broad range of applications. In supervised learning, CL has enhanced model performance through the strategic sequencing of training examples [13]. The gaming industry has employed CL to create progressively challenging levels, boosting agent performance [5]. Robotics has benefited from CL, as complex tasks could be decomposed into simpler steps, accelerating robotic skill acquisition [10]. Finally, CL has been used in safe reinforcement learning by gradually exposing the agent

to higher-risk scenarios [2]. For a comprehensive overview of CL methods, see the survey papers [12, 6].

Our proposed curriculum approach falls within the category of predefined curriculum learning [12]. We adopt a Curriculum Through Intermediate Goals strategy [3], decomposing a complex goal state into a sequence of simpler intermediate goals. While many existing approaches concentrate on modifying single aspects of the learning process, such as state distributions [1], reward functions [10], or goal generation [9], we differentiate our method by strategically modifying multiple facets. We manipulate the underlying Markov Decision Processes (MDPs) of our intermediate tasks, carefully tuning environment dynamics, reward mechanisms, and learning time horizons. This multifaceted approach, combined with our focus on a real-world problem, sets our approach apart from existing methods.

We aim to achieve three key benefits through our curriculum approach: optimization, improved sampling (as alluded to in [14]), and safety. Optimization benefit arises as a curriculum guides an agent towards solutions more efficiently than direct minimization of a non-convex target objective with multiple criteria. Statistically, careful sample allocation within a curriculum can boost performance by facilitating knowledge transfer among tasks, potentially reducing training data needs. For safety, as [8] note, a curriculum allows the reuse of safe policies from simpler tasks, ensuring safety in intermediate stages while the agent progressively develops proficiency for complex environments.

3 Real Environment and Problem Description

We consider the task of managing *containers* in a plastic sorting facility. The containers collect pre-sorted material. After accumulating enough material of a certain type, the container is emptied onto a conveyor belt and the material is moved to one of two *processing units* (PU-1 and PU-2). The setup is illustrated in Figure 8.1. Containers are continuously filled with material, with each container prescribed to a unique material. The material extraction from the container is always done until it is completely emptied. The difficulty of the task arises from two constraints. First of all, each container has its own unique optimal emptying volume depending on the material type. Emptying the container earlier is possible, but it results in a suboptimal product and a waste of energy. Emptying later is subject to similar costs, however, processing larger chunks of material is generally beneficial as it saves energy and processing time. The second constraint is that processing the material takes time, and containers can be emptied only if a PU is free.

If a container is emptied too far away from its optimal (at higher and or lower value), the deviation in volume that cannot be transformed into a product is again redirected to the corresponding container via an energy-intensive auxiliary process. On the other hand, if a container is emptied prematurely and too frequently, the

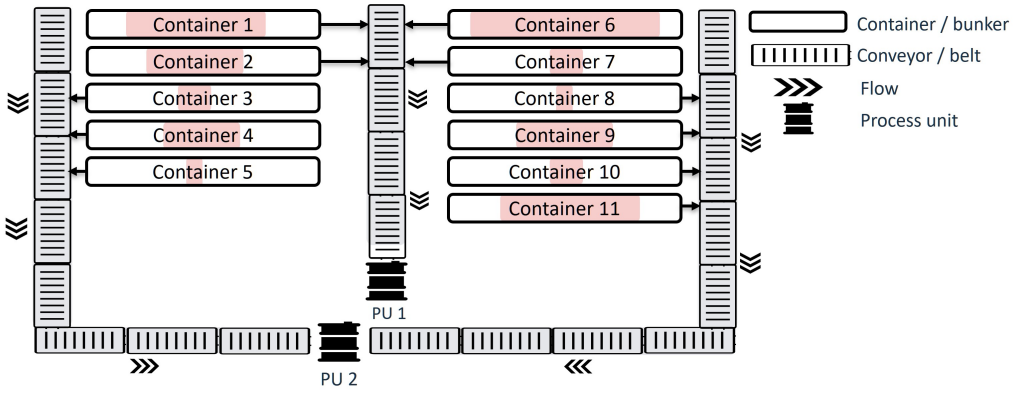


Figure 8.1: Layout sketch of a facility with 11 containers and 2 PUs, connected with conveyor belts. The containers are filled from above, with their current fill states indicated by the shaded areas.

PUs are engaged too frequently causing higher energy usage and also a bottleneck in resource allocation. In other words, the control task is to reduce the cumulative deviation from the ideal volume for each container while managing resources (PUs) efficiently.

A critical safety constraint that needs to be respected is that a container is never allowed to overshoot the physical limit of its maximum bearing volume (40 units). This incurs a high recovery cost including human intervention to stop and restart the facility, and should be avoided at all costs. Therefore, emptying containers close to the physical limit is a risky approach. Other system constraints are that certain containers can only be emptied into PU-1 and others only into PU-2. PU-1 takes in material via only one conveyor belt while PU-2 takes in material via two conveyor belts.

The quality of an emptying decision is a balance between three criteria:

- **Volume criteria**: minimize the cumulative deviation from the ideal emptying volume for each container.
- **Energy criteria**: optimize the energy usage or utilization costs of PUs.
- **Safety criteria**: adhere to the safety and system constraints.

Multiple aspects make this problem challenging:

- **Stochastic material flow rate.** The material flow rate into the containers is unique for each container, as it depends on the type of material, its density, the time of the day, seasonality, and other factors. In addition, the sensor readings determining volume estimates are very noisy. Incorporating this phenomenon, the flow rates are modelled as a stochastic process. This makes approaching the problem with standard planning approaches quite difficult.

- **Delayed rewards.** Certain containers have very slow filling rates, taking up to 5 hours or 300 simulation timesteps of 60 seconds each to reach their respective ideal volumes and receive a positive reward associated with the correct emptying action, as well as about 300 preceding 'idle' actions.
- **PU constraint.** The limited availability of the PUs implies that always waiting for containers to fill up to their ideal emptying volume is risky: if too many containers are close to their respective ideal volumes and no PU is available at that time, then a safety constraint is violated and a physical overflow occurs. Therefore, an optimal policy needs to take fill states and fill rates of all containers into account, and possibly empty some containers early or later.
- **Class imbalance.** Emptying decisions can be taken at any time, but the emptying actions close to the ideal volume for each container are rather infrequent, compared to the filling times to reach the respective ideal volumes. In addition, the rate at which containers should be emptied varies between containers. There is a class imbalance between the successful emptying actions (and corresponding rewards) and the do-nothing actions making the distributions of actions highly asymmetric, with important actions being relatively rare.

4 Reinforcement Learning Problem Formulation

In this section, the container management problem referenced in Section 3 is recast within the framework of a Markov Decision Process (MDP), with a corresponding digital twin designed in Python using gymnasium.

4.1 State space.

The system's state at a timestep t (s_t), encompasses several key components. These include container volumes $\{v_{i,t}\}_{i=1}^{n}$ bounded by predefined minimum and maximum volumes to maintain operational constraints. $\{p_{j,t}\}_{j=1}^{2}$ captures the normalized (by timestep) time until each of the two processing units becomes available, ensuring that the system can anticipate and plan for processing availability. $\{b_{k,t}\}_{k=1}^{n}$ is a binary representation indicating which containers are currently being emptied, providing immediate insight into the immediate container status, rewards from the previous timestep $\{r_{l,t-1}\}_{l=1}^{n}$, used to integrate feedback from past decisions, and the ideal volumes $\{pv_m\}_{m=1}^{n}$ for each container guiding the agent towards maintaining optimal material levels. The indices i, j, k, l, and m denote the container or PU identifiers. This representation is expressed as follows:

$$s_t = (\{v_{i,t}\}_{i=1}^{n}, \{p_{j,t}\}_{j=1}^{2}, \{b_{k,t}\}_{k=1}^{n}, \{r_{l,t-1}\}_{l=1}^{n}, \{pv_m\}_{m=1}^{n}) \tag{8.1}$$

This formulation ensures an adequate reflection of the system's dynamics, incorporating both the containers' status and the operational state of PUs, thereby facilitating the decision-making process.

4.2 Action space.

At any given time t, the agent has two primary choices: (i) to refrain from emptying any container, effectively taking no action and letting the volume increase, or (ii) to empty a specific container and process its contents. The *do-nothing* action is denoted by 0, while the action of emptying a container and processing its content is denoted by the index i of the container. Thus, the action a_t falls within the set $\{0, 1, \ldots, n\}$, where n represents the total number of containers. Depending on the availability of the PUs the action taken is either successful or unsuccessful for emptying and is reflected through the volumes, status of PUs and time features in the state, and the reward received.

4.3 Environment Dynamics

In this section, the dynamics of the volume of material in the containers, the PU model, as well as the state update are discussed.

4.3.1 Container filling rates.

In addressing the variability of container fill rates due to the inconsistent nature of plastic material inflow, which ranges from solid lumps to irregular flows, leading to uneven accumulations within containers, a stochastic model is employed. This model accounts for the erratic yet on average linear increase in volume over time through a random walk mechanism with drift. Specifically, the volume update for container i at time $t + 1$ is modeled as:

$$v_{i,t+1} = \max(0, \alpha_i + v_{i,t} + \epsilon_{i,t}), \tag{8.2}$$

where α_i represents the average volume increase rate, and $\epsilon_{i,t}$ is a normally distributed random variable representing measurement noise. This approach captures real-world fluctuations in fill rates while simplifying the model to facilitate analysis and simulation. Upon the action of emptying a container, its volume is instantaneously reduced to zero in the subsequent timestep. This simplification contrasts with the gradual volume reduction observed in actual scenarios but aligns with our dataset's indication that the emptying process completes within the duration of a single timestep, set at 60 seconds in this study. This modelling choice effectively bridges the gap between the discrete-time model used for simulation and the continuous nature of the emptying process observed in operational environments.

4.3.2 Processing unit dynamics.

The transformation time by a Processing Unit (PU) for material volume v in a container depends linearly on the producible products from v, expressed as $\lfloor v/b_{ij} \rfloor$. This is detailed by the function g_{ij} in Equation (8.3), incorporating b_{ij} for product size, β_{ij} as the PU's activation time, and λ_{ij} for time per product. Here, i refers to the container index, and j to the specific PU index, indicating that the parameters are specific to each container-PU combination. If PUs are occupied, containers awaiting processing continue to accumulate material, highlighting the system's operational constraints and the need for efficient PU management.

$$g_{ij}(v) = \beta_{ij} + \lambda_{ij} \lfloor v/b_{ij} \rfloor \ . \tag{8.3}$$

5 Reward Tuning

In addressing the complexities of dynamic decision-making environments, the formulation of reward functions is a critical component. This section presents the case for Gaussian-based reward mechanisms, chosen for their smoothness and ability to accommodate the infrequent nature of container-emptying actions and the uncertainties within state space parameters. Gaussian rewards, characterized by their resilience to noise, ease of interpretation, and smooth gradient provision, are selected to mitigate the effects of measurement inaccuracies and focus on the aggregate effects of the underlying phenomenon. The strategies are presented in three distinct subsections: Simple Gaussian Reward, Custom Reward, and Precision Reward.

5.1 Simple Gaussian reward

The reward $r(s_t, a_t, PU_{status})$ is calculated based on the action taken, the current volume of a container, the volume at the next time step, and whether a PU is free to empty the container implicitly contained in s_t. A Gaussian distribution centred around the ideal volume for emptying a given container with parameters defining the peak volume (p), peak height (a), and peak width (w) was used. The complete reward function $r(s_t, a_t, PU_{status})$ is summarized in Algorithm 8.2. It fosters emptying containers at or close to their optimal volumes, where the width (standard deviation) w controls the required precision encoded by the reward. A wider peak enables fast initial learning, while a narrow peak focuses on fine-tuning.

In principle, this reward encodes everything the agent needs to know about the problem at hand.

Algorithm 8.2: Gaussian Reward Function for Emptying Decision

input : Action a_t, Current volume v_t, PU availability *PU_status*, penalty r_{pen}, ideal volume v_i, Height of the peak h, Width of the peak w
output: Reward r_t between r_{pen} and 1

```
if a_t > 0 then
    if v_t = 0 or not PU_status then
        r_t ← r_pen
    else
        r_t ← (h − r_pen) · exp(−(v_t − v_i)^2 / (2w^2)) + r_pen
```

5.2 Custom Reward

Here the custom reward function built on the Gaussian reward is introduced to navigate the complexities of dynamic resource allocation and operational optimisation. This reward is a sum of action rewards plus bonuses, positional rewards, and episode termination rewards to enhance the learning efficiency of the agent.

Action Reward, which is nothing but the Simple Gaussian Reward Function, described earlier is used to quantify the efficacy of an action at a timestep based on the container's proximity to an ideal volume. It applies only to (supposedly rare) emptying actions. The conditional nature of this reward ensures that meaningful actions—those contributing to the actual emptying behaviour of the agent–are incentivized, while actions that do not align with constraints are penalized.

Positional Reward is used to encourage the system towards an ideal operational state across all containers, not just the one being acted upon. It's designed to address the challenge of slow filling rates across the containers, which leads to sparse action rewards within the operational timeframe. For example, there are containers for which it takes up to 300 timesteps of 60 seconds (or five hours) to reach the ideal emptying volume. These rewards need to be propagated back through a long history of zero-actions that contributed to emptying at the right moment. By assigning a reward to each container based on its volume relative to an ideal operational state, and importantly, doing so irrespective of the agent's actions *at every time step*, the reward signal ensures a constant nudge towards optimal efficiency across all containers.

The positional reward for a given container not acted upon at time step t, $r_{\text{positional},j}$, is calculated as follows:

$$r_{\text{positional},j} = \begin{cases} 1 - \left|\frac{(v_i - v_{j,t})}{v_i}\right|^{0.5}, & \text{if } v_{j,t} \leq v_i, \\ -0.1, & \text{otherwise.} \end{cases} \tag{8.4}$$

The cumulative positional reward for all containers is calculated as

$$r_{\text{positional}} = \sum_{j=1}^{n} r_{\text{positional},j} \tag{8.5}$$

where n is the total number of containers.

Episode Termination Reward is a critical component designed to ensure the agent learns to avoid prematurely ending an episode by reaching an unsafe state. It aims at discouraging the agent from allowing any container to exceed its physical limit of 40 volume units, a crucial safety consideration.

The episode termination reward, $r_{\text{termination}}$, is defined as follows:

$$r_{\text{termination}} = \begin{cases} 0.2, & \text{reward for not ending the episode prematurely,} \\ -30, & \text{if any container's volume exceeds the safety limit of 40.} \end{cases} \tag{8.6}$$

5.3 Precision Reward

While the Gaussian reward in principle encodes the full goal of emptying at the optimal volume, its smooth and rather flat top can make fine-tuning difficult. Therefore, we introduce an additional incentive for the agent to empty at the ideal volume by rewarding within a narrowly defined optimal range and penalizing deviations:

Algorithm 8.3: New Reward Style for Encouraging Ideal Volume

input : Current volume v_t of a container, Ideal volume v_i
output: Reward r_t

```
if −0.5 + v_i < v_t < 0.5 + v_i then
    r_t ← 1.0
else
    r_t ← −0.1
```

6 Methodology

In the section, a scaffold approach to developing reinforcement learning agents is presented. This methodology, encompassing five phases, progressively introduces increased complexity, systematically enhancing agent competency. By employing a phased curriculum learning strategy, the agent's evolution is carefully curated, ensuring a gradual ascension to operational adeptness. In the following section,

we go through each phase, starting from its goals, and explaining the measures for reaching these goals.

Phase 1: Foundational Training. The goal of this phase is to learn a first non-trivial policy for a simplified version of the task. In particular, the agent must learn to perform the zero-action most of the time, while executing rare but critical emptying actions when a container volume comes close to the optimal volume. In this initial phase, the random walk for filling containers is disabled, resulting in a deterministic and easily predictable environment. Furthermore, there are as many PUs as containers. That's a very unrealistic setting that in effect removes all dependencies between containers reaching peak volume at the same time. Here, the Custom Reward is implemented. Actions are taken every 30 seconds, and episodes are as short as 25 time steps. The initial states are designed such that there are at least 8-11 emptying actions that lead to positive rewards and the rest being do-nothing actions. This helps the initial agent in dealing with the class imbalance problem alluded to in section 3.

Phase 2: Refinement. In this stage, the Precision Reward is added for a budget of 1 million timesteps. All other settings are retained from the first phase. The introduction of Precision Reward aims to refine the agent's accuracy in actions, within a deterministic setting, enhancing the precision in decision-making.

Phase 3: Penalizing Greedy Actions. This training stage aims at altering the policy so that it becomes more energy efficient. This is achieved by slightly penalizing all emptying actions. The reasoning is that PUs are a scarce resource, and emptying a container twice instead of once blocks the PU for an unnecessarily long time, and it also uses more energy than processing more of the same material in one go. This adjustment is intended to augment the agent's greedy nature to accumulate more episodic rewards by prematurely emptying a container too many times as opposed to optimizing for the least number of emptying actions per episode along with precise emptying actions.

Phase 4: Inject Real-world Complexity. Now that the basic behaviour is as intended, the agent is ready to be exposed to the true complexity of the environment. The budget for this phase is 0.5 million timesteps. Timesteps are extended to 60 seconds, and the episodes are up to 600 timesteps long. In addition, the stochastic filling rates and resource constraints are introduced. This results in significant changes in the environment dynamics (noisy dynamics, failing emptying actions if all PUs are busy), which must be accounted for with corresponding changes in the value function estimator. To achieve this, the policy network is frozen and only the

value network is trained as both of these are distinct neural nets not sharing network parameters [11]. This phase is tailored to tune the agent's value network to the unpredictable and constrained operational dynamics.

Phase 5: Fine-tuning with Reduced KL Constraint. This phase concludes the curriculum, mirroring Phase 4's operational parameters but reinstating the Precision Reward. Gradually unfreezing the policy with a low KL constraint (limiting the Kullback Leibler divergence between action distributions) allows controlled exploration around the learned policy within the complex environment introduced in Phase 4.

To address operational constraints and edge cases effectively, action masking was incorporated after the curriculum learning phases for inference on the test environment. This adaptation ensures alignment with the plant's structural and operational limitations, where the disposition of containers relative to Processing Units (PUs) is dictated by the proximity to conveyor belts. Given this configuration, only specific containers can be targeted for emptying based on their accessibility to certain PUs. Additionally, dynamic reduction of the action space is facilitated through action masking [4], particularly when all PUs are engaged, enhancing decision-making efficiency.

Table 8.4: Summary of curriculum learning phases and their parameters

Phases	Budget (Mill.Ts)	TimeSteps (Ts)	Ep-len (Ts)	Reward	Fill. rates	Resource Constraint (PUs)
Phase-1	1.5	30	25	Custom	Deterministic	No
Phase-2	1	30	25	Precision	Deterministic	No
Phase-3	1	30	25	Precision with penalty for positive actions	Deterministic	No
Phase-4	0.5	60	600	Policy frozen and Precision	Stochastic	Yes
Phase-5	0.5	60	600	Precision	Stochastic	Yes

7 Experimental Evaluation

The goal of this section is to empirically verify the effectiveness of the proposed approach. We aim to achieve this by answering two research questions.

Research question 1: Is the curriculum approach effective? To this end, we compare the PPO-Curriculum learning (PPO-CL) agent trained in five phases for all three criteria agents with our first baseline. This is a naive PPO agent (PPO-volume criteria), optimized only for one of the three criteria, i.e., optimal emptying volumes encoded by training with a Simple Gaussian reward.

Research question 2: Does the proposed approach offer a real-world benefit? For this, we compare PPO-CL with our second baseline, which is a meticulously handcrafted analytical agent labelled as a Optimal Analytic agent. This agent is close to the semi-automated decision policy currently deployed at the waste sorting facility. It is optimal in the sense that it empties containers at their precise target volumes if possible, and that it performs emergency emptying at a volume of 37 (out of 40) to avoid overflow. However, it does not take interactions of containers competing for PUs into account.

With respect to metrics, cumulative reward values are not necessarily the most relevant performance measure. Therefore we investigate different aspects like safety, energy-saving behavior and timing precision separately, in order to arrive at conclusions that are meaningful not only from an RL perspective but also from an application point of view.

In the spirit of open and reproducible research, we make our source code available via an anonymous repository.[1] The repository contains a script for reproducing all results presented in this section.

7.1 Experimental Setup

The PPO-CL (five phases) and PPO-volume criteria agents undergo training within environments with eleven containers and two PUs. These are characterized by a 60-second time-step and a 600-episode length, with default settings maintained for other hyperparameters. To evaluate stability, fifteen independent training runs are conducted with distinct random seeds for each agent. Our second baseline, the Optimal Analytic agent operates with a fixed logic. Hence it is designed, not trained. Its main criterion for emptying decisions is the evaluation of the proximity of each container's volume to its ideal state. It also prioritizes actions based on operational conditions such as PU availability and the plant's physical layout (e.g., container alignment with conveyor belts). To ensure a fair comparison, we test the PPO-CL agent and both baselines on the same test environment.

[1] `https://gitlab.com/anonymousppocl1/ppo_paper.git`

7.2 Results

In this section, we present the empirical results. Figure 8.5 shows a single rollout with the best policy produced by both PPO agents. Our visualization tracks container volumes, actions, and PU utilization over time, offering a more nuanced analysis than reward values alone. The volume chart shows different containers being emptied at around their respective ideal volumes and the frequency of peaks in the time chart for PUs gives a sense of how they compete for the resources. The key insight here is that although the PPO-volume criteria agent seems to show reasonable behaviour, its emptying decisions are not properly timed, as is evident from the large reward fluctuations. In contrast, the PPO-CL agent consistently achieves close-to-optimal rewards.

7.2.1 Emptying Decision Quality

Here we compare all three agents for the quality of emptying decisions. For the trained PPO agents, these metrics represent the best out of 15 independent training runs. Table 8.6 shows the average inference volume deviation (deviation between actual and ideal container volume) across all containers. The first baseline, the PPO-volume criteria agent, performs poorly with the highest deviation. The PPO-CL agent and the analytical agent achieve much smaller deviations, while the former does better than the latter.

7.2.2 Safety and Energy savings

Table 8.6 also shows the total number of emptying actions. Both baseline agents take the same number of emptying actions, while PPO-CL gets along with fewer actions, consequently emptying containers at higher volumes. This behavior results in better resource utilization, in terms of PU occupation time, and also in terms of energy.

Figure 8.7 further highlights the PPO-CL agent's strengths in energy efficiency and safety compliance. On the left, we see significant energy conservation: the PPO-CL agent utilizes the PU 12% less than the Optimal Analytic Agent and 24% less than the PPO-volume criteria agent. The right graph underscores the PPO-volume criteria agent's alarming safety violations: 27.11% of its actions exceed the critical volume limit of 40. This highlights the risks of training an RL agent from scratch in complex environments with safety constraints. In contrast, the Optimal Analytic agent exhibits zero violations due to its hard-coded limit of 37, acting as a safety benchmark but lacking adaptability. The PPO-CL agent, however, achieves a remarkable 1.7% violation rate.

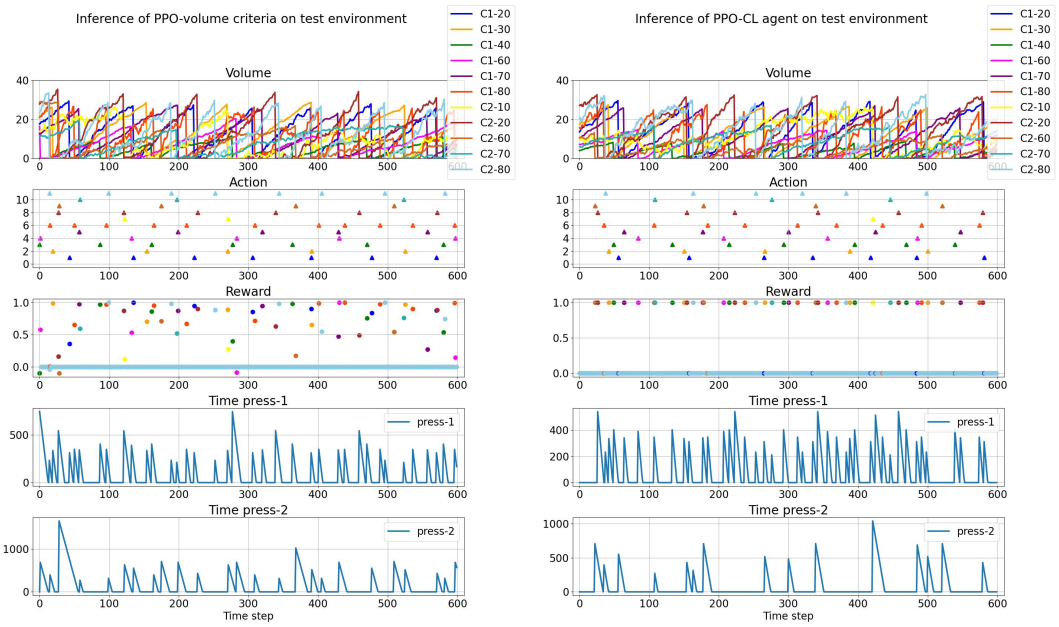


Figure 8.5: A single rollout (best agent out of 15) of the PPO-CL (left) and PPO-volume criteria on a test environment with 11 containers. Displayed are the volumes, emptying actions, rewards, and time to process by PU-1 and PU-2.

Overall, the PPO-CL agent demonstrates a superior balance. The results suggest that phased learning strategies offer distinct advantages in complex, criteria decision-making environments, particularly in industrial settings where precision, adaptability, and efficiency are crucial.

Table 8.6: Performance metrics for both RL agents (best seed out of 15) and analytic agent for 15 rollouts on the same test environment: emptying actions and average inference volume deviation

Agent	Emptying Actions	Avg. Inf Vol Deviation (± Std Dev)
PPO-volume criteria	62	15.16 ± 10.80
Optimal Analytic Agent	62	4.47 ± 2.61
PPO-CL	56	3.55 ± 2.33

7.3 Discussion

To gain deeper insights into the PPO-CL agent's safety, energy efficiency, and volume management, we further analyze the results from the previous section.

The PPO-CL agent's safety adherence, a critical outcome highlighted by the results, likely stems from two key curriculum design elements: the enforcement of safety

constraints and the early emphasis on safe exploration. Short episode lengths in initial phases (25 timesteps, as seen in Table 8.4) encouraged exploration within safe state spaces. Furthermore, the reward structure in Phase 3, which penalizes actions even with positive outcomes, directly incentivizes conservative decision-making, contributing to the agent's reduced PU utilization and energy savings. This flexibility in balancing competing objectives makes our framework particularly suitable for complex industrial settings where safety is of utmost importance.

To analyze the emptying decision quality of the PPO-CL agent against our two baselines, we examine the empirical cumulative distribution functions (ECDFs) in Figure 8.8. ECDFs plot the probability, based on observed data from agent actions, that a container is emptied at or below a given volume. Here, steeper curves signify greater consistency in emptying volumes at which a container is emptied across different instances. Our first baseline, the PPO-volume criteria agent, exhibits higher variance in its ECDF, particularly for slower-filling containers (e.g., C1-40, C1-60, C2-10, C2-20). This points to the difficulty of learning effective policies when rewards are delayed due to extended fill times, an issue hindering from-scratch training as discussed in Section 5.2. In contrast, our second baseline, the Optimal Analytic agent, employs a deterministic logic that leads to frequent preemptive emptying. While this ensures no safety violations, it results in inefficient energy usage.

The PPO-CL agent's ECDFs demonstrate remarkable consistency across all containers, regardless of fill rate. This highlights its ability to manage delayed rewards effectively, leading to optimal energy usage with fewer emptying actions per episode – a key advantage emphasized by our results. Crucially, unlike the PPO-volume criteria agent, the PPO-CL agent consistently avoids reaching the critical volume threshold of 40 (evident from all slow-filling containers), demonstrating its strong safety compliance.

8 Conclusion

In this work, a curriculum learning approach was introduced, designed to progressively enhance the complexity of the environment for training a Proximal Policy Optimization (PPO) agent to navigate the intricacies of real-world industrial operations. Our model not only achieves a harmonious balance among competing operational goals like volume management, energy conservation, and adherence to stringent safety protocols, but it also paves the way for the broader application of such strategies in advanced, adaptive, multi-criteria decision-making environments. We believe that with some (industrial) domain expertise available, designing a curriculum is an attractive alternative to using excessive computing for solving hard problems, e.g., by training more complex networks or by applying multi-agent approaches.

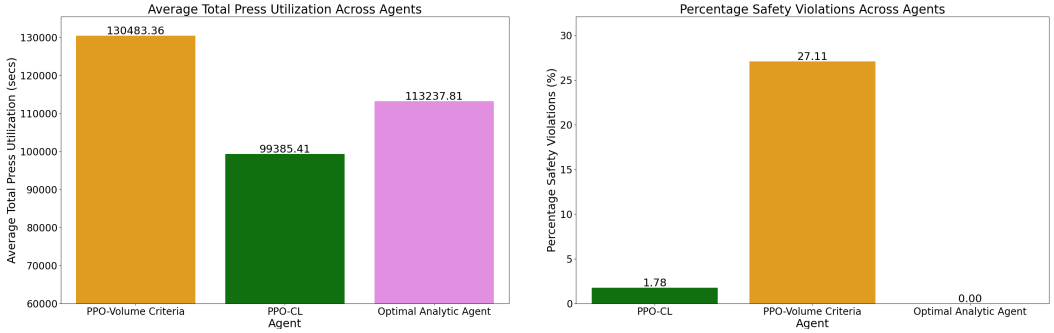


Figure 8.7: Comparison of key performance metrics across different agents, collected over 15 rollouts of the best policy for both PPO agents. The left figure presents the average total PU utilization across agents. The right figure details the percentage of safety violations

We view our achievement as a solid basis for future endeavours. Although our agent performs well in most cases, we have not yet pushed its ability to avoid unsafe behaviour to the maximum. This amounts to avoiding *future* collisions of PU usage requests, which may occur if too many containers reach their ideal volume at the same time. That situation should be counteracted by emptying some containers earlier than usual. The problem is pressing since PUs are expensive units, so facilities are often designed with the smallest possible number of PUs. Designing a systematic solution to this problem will be subject to future research. It may involve forms of (stochastic) real-time planning, as well as further curriculum steps specifically targeted at the collision problem.

For practitioners in the realm of real-world reinforcement learning, we offer two take-home messages. First, it has proven invaluable to evaluate agent performance beyond mere cumulative rewards, using ECDF plots, diverse metrics, and statistical tools. Second, particularly in environments of high complexity, adopting a curriculum-based training approach can help with the learning of meaningful policies, surmounting the limitations faced by vanilla agents.

References

[1] Carlos Florensa et al. “Reverse Curriculum Generation for Reinforcement Learning”. In: *Proceedings of the 1st Conference on Robot Learning (CoRL)*. 2017.

[2] Javier García and Fernando Fernández. “A Comprehensive Survey on Safe Reinforcement Learning”. In: *Journal of Machine Learning Research* 16 (2015), pp. 1437–1480.

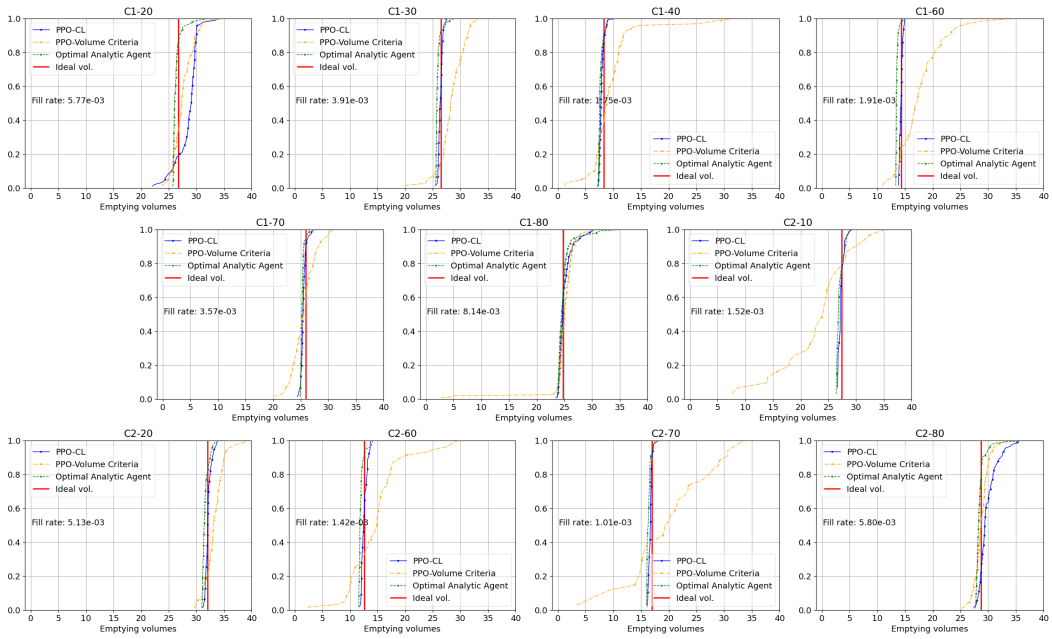


Figure 8.8: ECDFs of emptying volumes of all 11 containers collected over 15 rollouts of the best policy for PPO-Volume criteria, PPO-CL, and Optimal analytic agent on a test environment. Average fill rates are indicated in volume units per second. The derivatives of the curves are the PDFs of emptying volumes. Therefore, a steep incline indicates that the corresponding volume is frequent in the corresponding density.

[3] Kashish Gupta, Debasmita Mukherjee, and Homayoun Najjaran. "Extending the Capabilities of Reinforcement Learning Through Curriculum: A Review of Methods and Applications". In: *SN Computer Science* 3.1 (Oct. 2021), p. 28.

[4] Shengyi Huang and Santiago Ontañón. *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*. arXiv preprint arXiv:2006.14171. June 2020. arXiv: 2006.14171.

[5] Niels Justesen et al. "Automated Curriculum Learning for Deep Reinforcement Learning". In: *Conference on Computational Intelligence and Games (CIG)*. 2018.

[6] Sanmit Narvekar et al. "Curriculum learning for reinforcement learning domains: a framework and survey". In: *Journal of Machine Learning Research* 21.1 (Jan. 2020), pp. 7382–7431.

[7] Abhijeet Pendyala et al. "ContainerGym: A Real-World Reinforcement Learning Benchmark for Resource Allocation". In: *Machine Learning, Optimization, and Data Science*. Springer Nature Switzerland, 2024, pp. 78–92.

[8] Tijmen Pollack and Erik-Jan Van Kampen. "Safe Curriculum Learning for Optimal Flight Control of Unmanned Aerial Vehicles with Uncertain System Dynamics". In: *AIAA Scitech 2020 Forum*. American Institute of Aeronautics and Astronautics, Jan. 2020.

[9] Sébastien Racanière et al. "Imagination Augmented Agents for Deep Reinforcement Learning". In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. 2019.

[10] Martin Riedmiller et al. *Learning by Playing - Solving Sparse Reward Tasks from Scratch*. arXiv preprint arXiv:1802.10567. 2018. arXiv: 1802.10567.

[11] Xiangjun Wang et al. *SCC: an efficient deep reinforcement learning agent mastering the game of StarCraft II*. arXiv preprint arXiv:2012.13169. Dec. 2020. arXiv: 2012.13169.

[12] Xin Wang, Yudong Chen, and Wenwu Zhu. "A Survey on Curriculum Learning". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.9 (Sept. 2022), pp. 4555–4576.

[13] Daphna Weinshall, Gad Cohen, and Dan Amir. *Curriculum Learning by Transfer Learning: Theory and Experiments with Deep Networks*. arXiv preprint arXiv:1802.03796. 2018. arXiv: 1802.03796.

[14] Ziping Xu and Ambuj Tewari. "On the Statistical Benefits of Curriculum Learning". In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 24663–24682.

9 Collision Avoidance using Hybrid Reinforcement Learning and Planning

9.1 Context and Motivation

The research presented in Chapter 8 demonstrated the effectiveness of combining Curriculum Learning and sophisticated Reward Engineering (PPO-CL) to train a high-performing reinforcement learning agent. This approach successfully navigated a complex, multi-objective landscape, enabling the agent to balance safety, throughput, and energy efficiency in a realistic configuration of the ContainerGym environment.

However, while the PPO-CL agent exhibited sophisticated skills, it remains a fundamentally reactive policy. As discussed in Section 6.1, reactive policies often suffer from "strategic myopia"—an inability to explicitly forecast and reason about the future consequences of actions, particularly in systems with resource contention. In the container management problem, this limitation manifests as an inability to effectively anticipate and prevent "collisions," where multiple containers require the scarce Processing Unit (PU) simultaneously, leading to potential overflows.

This challenge becomes particularly acute in high-contention scenarios (e.g., many containers sharing a single PU) and when optimizing for dual-peak objectives (where a higher-volume peak offers better throughput but carries more risk).

This chapter presents the third and final core contribution of this thesis, directly addressing the limitations of the purely reactive approach by developing a novel **Hybrid Control Architecture**, following the principles outlined in Chapter 6. The following publication details the integration of the curriculum-trained PPO-CL agent with an efficient, offline-trained Collision Model (CM) used at inference time (PPO-CL-CM). This hybrid system synthesizes the learned skills of the RL agent with the proactive foresight of the predictive model to ensure safety and maximize efficiency. Furthermore, the publication provides a comprehensive system-level analysis, evaluating the scalability of the approach across varying container-to-PU ratios and deriving practical guidelines for facility design.

9.2 Curriculum RL meets Monte Carlo Planning: Optimization of a Real World Container Management Problem

Curriculum RL meets Monte Carlo Planning: Optimization of a Real World Container Management Problem

Abhijeet Pendyala, Tobias Glasmachers

Ruhr-University Bochum, Bochum, Germany

firstname.lastname@ini.rub.de

Abstract. In this work, we augment reinforcement learning with an inference-time collision model to ensure safe and efficient container management in a waste-sorting facility with limited processing capacity. Each container has two optimal emptying volumes that trade off higher throughput against overflow risk. Conventional reinforcement learning (RL) approaches struggle under delayed rewards, sparse critical events, and high-dimensional uncertainty—failing to consistently balance higher-volume empties with the risk of safety-limit violations. To address these challenges, we propose a hybrid method comprising: (1) a *curriculum-learning* pipeline that incrementally trains a PPO agent to handle delayed rewards and class imbalance, and (2) an *offline pairwise collision model* used at inference time to proactively avert collisions with minimal online cost. Experimental results show that our *targeted inference-time collision checks* significantly improve collision avoidance, reduce safety-limit violations, maintain high throughput, and scale effectively across varying container-to-PU ratios. These findings offer actionable guidelines for designing safe and efficient container-management systems in real-world facilities.

Keywords: Reinforcement learning, Curriculum learning, Inference-time planning, Industrial control, and Collision avoidance.

1 Introduction

Waste-sorting facilities increasingly rely on data-driven methods to meet strict energy efficiency and sustainability goals, due in part to regulatory directives for responsible recycling of packaging waste. Modern plants must deal with fluctuating material types, unpredictable daily volumes, and stringent safety constraints. This work is inspired by the final stage of a waste-sorting facility where n containers (*bunkers*) accumulate various types of material at unique stochastic rates. These materials are subsequently transported to a processing unit (PU) for compaction

into bales or products [1] during which PU is unavailable for further processing. In addition, each container has a strict maximum capacity and overflowing beyond a threshold requires halting the facility for corrective measures, incurring steep penalties. In this context, *container management* emerges as a critical bottleneck: Emptying these containers too late risks overflow; emptying them too early or too frequently undermines throughput and raises energy costs.

Recent work has modeled container management as a reinforcement learning (RL) problem, notably through *ContainerGym* [8], providing a real-world benchmark where an RL agent decides *when* to empty each container. However, naive Proximal Policy Optimization (PPO) agents frequently fail in this domain, as *delayed rewards*, *sparse critical events*, and a *single shared PU* create complex scheduling dynamics. For instance, some containers have a "higher peak" volume (around 60–75% capacity) for optimal throughput and a "lower peak" (around 30–40%) as a fallback. Emptying containers at the peak volumes yields optimal products, hence emptying at other volumes is undesirable. The prime reason for missing a peak volume is unavailability of the PU caused by a *collision* of two or more containers reaching peak volume around the same time. Prior work showed that *curriculum learning* helps mitigate early overflows and improves PPO's learning curve [7]. Still, the problem of handling collisions remains unsolved, especially at higher container-to-PU ratios.

On the other hand, despite the installation of sophisticated sensors in these facilities and real-time monitoring, most waste sorting design layouts remain fixed once built, even if data points to major bottlenecks. This reveals a broader gap: leveraging RL not only to optimize day-to-day operations but also to guide *facility design decisions* such as how many containers can safely share a PU before collisions dominate. In other industries—like robotics or warehouse logistics—RL insights have influenced layout reconfiguration, helping systems adapt hardware choices to software-derived constraints. Yet in waste-sorting, this feedback loop remains largely unexplored.

To address these gaps, we present a *hybrid RL* method that integrates curriculum learning with a domain-specific *collision model* at inference time. Our approach (1) reduces collision-induced overflows and throughput losses, and (2) closes the loop between software-driven scheduling and hardware design insights. Specifically, we:

- Propose a three-phase *curriculum learning* strategy to tackle delayed rewards and dual-volume targets (higher vs. lower peak).
- Introduce an *offline-trained collision model* for on-the-fly inference checks, overriding risky "no-op" actions when multiple containers approach critical volumes.

[1] In this study the terms bunker/container and processing unit (PU)/press are used interchangeably.

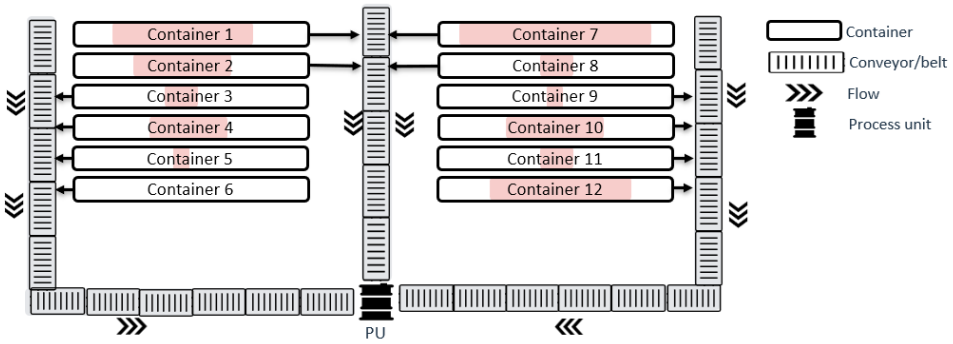


Figure 9.1: Layout sketch of a facility with 12 containers and a PU, connected with conveyor belts. The containers are filled from above, with their current fill states indicated by the shaded areas.

- Systematically evaluate varying *container-to-PU ratios* (7:1 to 12:1), providing actionable guidelines on how many containers a single PU can realistically handle without causing excessive collisions.

Empirical results demonstrate that the proposed hybrid RL framework significantly cuts collisions, reduces safety limit violations, maintains higher throughput, and yields design insights for scaling container management.

2 Related Work

Reinforcement learning has proliferated in industrial and logistics applications, moving beyond canonical benchmarks to real-world applications. We categorize related literature under the following three key themes:

Curriculum Learning in Industrial RL Curriculum learning (CL) systematically organizes the training tasks, typically starting with simpler sub-problems before moving to more challenging ones [6, 13]. In industrial domains, where critical actions are rare and reward signals can be delayed or sparse, CL helps agents gather meaningful experience without being overwhelmed by complex dynamics from the outset. Prior work in container management [7] showed that a curriculum-based PPO significantly reduces early overflows compared to naive baselines. We build on this by designing a multi-phase reward curriculum—enabling the agent to master *dual-peak* emptying targets and maintain stable performance under varying inflows.

Inference-Time Planning and Collision Avoidance While a trained RL policy provides baseline decisions at deployment, *inference-time planning* augments these decisions with lookahead logic or heuristic checks, often through Monte Carlo methods. Techniques vary in whether they update the agent's parameters or remain "stateless." Prominent examples include Monte Carlo simulations in game-playing AI [11, 1], but similar ideas have surfaced in robotics [9, 14], energy systems control [3] and autonomous driving [4]. In our context, we adopt a *collision model* that is trained offline to predict when multiple containers are poised to exceed safe volumes simultaneously. This model supplements a curriculum-trained policy by overriding risky no-operation actions—preventing multi-container collisions. By decoupling real-time safety checks from the offline-trained policy, our method maintains policy stability and adds minimal inference overhead, which is essential for deployment in high-throughput industrial environments.

RL-Driven Design insights An emerging trend in industrial settings is the use of RL not merely to control a dynamic process but also to inform insights into *design* decisions. In large warehouse environments, multi-agent RL has optimized the assignment of "chutes" to destinations [10], reducing robot congestion and improving throughput. Similar approaches in factory automation focus on workstation placement [5], where RL-driven layout designs outperform handcrafted alternatives. Meanwhile, open-source platforms like "Storehouse" show that RL can outperform heuristic policies in dynamic warehouse slotting [2]. These successes underscore how AI insights can *redesign* processes for higher efficiency, paralleling our goal of using RL not just to operate container management but to shape decisions on how many containers a PU can handle effectively.

Context of our Contribution By integrating a *predefined curriculum* for delayed rewards with an *offline collision predictor*, our hybrid **PPO-CL-CM** approach targets both throughput optimization and collision avoidance in container management. Offline pairwise simulations yield a lightweight collision classifier, which can be queried at each time step to override the policy when collision risk is high. The result is a system that meets key challenges—dual-peak scheduling, sparse rewards, and collision hazards—while also generating real-world design insights. In the following sections, we present the formal environment setup, detail our curriculum learning phases, and then describe how we integrate collision checks at inference.

3 Environment and RL formulation

In this section, we provide details of the considered container management environment. Building on the scenario described in Section 1, we reiterate the central design

optimization criterion; each container has two preferred or "ideal" volumes at which emptying yields the highest-quality output. These correspond to a *Higher peak* offering better overall throughput if the container can safely reach this point and a *Lower peak*, providing a reasonable fallback when waiting longer could risk overflow or collide with the PU's availability window. In practice, larger volumes generally improve efficiency, as the PU operates more effectively when processing bigger batches at once. However, strictly aiming for the higher peak can provoke collisions or safety limit breaches if the single PU is not available in time.

Markov Decision Process (MDP) Setup. We model the container-management scenario as an MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$:

- **State s_t:** Comprises volumes $\{v_{i,t}\}_{i=1}^{n}$ for each container, a PU-availability counter p_t (time until the processing unit becomes available), and auxiliary signals such as ideal volumes. This provides the agent with both physical constraints (capacity, current usage) and strategic cues (optimal targets).
- **Action $a_t \in \{0, \ldots, n\}$:** Either do nothing ($a_t = 0$) or attempt to empty container i. If $p_t > 0$ (the PU is busy), an emptying request fails, and container i continues to fill. This design highlights collision risks when multiple containers approach peak volumes simultaneously.
- **Volume Dynamics:** Each container i grows according to a *random walk with drift*:
$$v_{i,t+1} \;=\; \max\big(0,\; v_{i,t} + \alpha_i + \epsilon_{i,t}\big), \tag{9.1}$$
where α_i is the average fill rate, and $\epsilon_{i,t} \sim \mathcal{N}(0, \sigma_i^2)$ captures stochastic fluctuations. Overflow occurs if $v_{i,t} > 40$, triggering a heavy penalty and episode termination.
- **PU Overhead:** Emptying container i with volume v imposes a busy time $g_i(v)$. The busy time consists of a material-dependent part that is proportional to the volume, and a constant offset for conveyor belt transport of the material from the container to the PU. During this period, p_t counts down to zero, after which the PU is free again. Requests made while $p_t > 0$ are effectively lost, emphasizing scheduling constraints.
- **Rewards:** The agent receives reward for emptying containers close to their peak volumes, since that behavior results in high quality output. Rewards are designed so that emptying at the higher peak is preferable. Container overflow is a terminal state with an episode reset. *Invalid* or wasteful empties (e.g., container already empty or PU busy) incur a negative penalty r_{pen}, while the *do-nothing* action yields zero.

Collision state. A *collision* arises when one or more containers simultaneously approach or exceed their higher-peak volumes, but the PU remains busy processing another container. This can rapidly push volumes over physical capacity if not

addressed promptly. Although we first train the agent without a collision-specific mechanism, Section 5 discusses how we incorporate a collision model at inference time.

Objective. The agent must *schedule empties* near either the higher or lower volume peak to maximize efficiency, while preventing overflows and avoiding collisions on the shared PU. The tension between delaying emptying for higher-volume payoffs versus frequent empties for safety and reduced collisions creates a challenging RL problem under delayed rewards and a scarce PU bottleneck.

Key Challenges

- **Stochastic inflow & sensor noise:** Each container's fill rate depends on material type, density, and unpredictable external factors (e.g., time of day, seasonal fluctuations). Sensor noise further obscures volume estimates, making it difficult to predict precisely when a container will reach a target volume.
- **Delayed rewards & class imbalance:** Some containers fill slowly, requiring hours of in-simulation time to reach the target volume. Consequently, episodes contain many "do nothing" steps, during which the state changes steadily but rewards from emptying remain sparse. As emptying actions are rare, reward-bearing moves are infrequent, creating skewed action distributions that challenge many reinforcement learning algorithms.
- **Dual-peak design & Collisions:** Having two ideal emptying peaks per container creates a nuanced trade-off: waiting for the higher peak boosts efficiency but risks collisions and overflow if multiple containers converge while the PU is busy. Alternatively, settling too often for the lower peak increases emptying frequency, raising energy costs and volume deviations. Balancing these opposing factors—alongside scheduling constraints to avoid collisions and safety violations—poses a central challenge.

Algorithm 9.2: Three-Phase Reward Computation

Input : Phase $ph \in \{1, 2, 3\}$, current volume v_t, action a_t, penalty r_{pen}, peaks $(v_{\text{low}}, v_{\text{high}})$, Gaussian params (h, w) or (h_1, h_2, w_1, w_2)

Output: Immediate reward r_t

if $a_t = 0$ **then** `// No-op`
  $r_t \leftarrow 0$;
else
  if *(*invalid conditions*)* **then** `// Invalid empty`
    $r_t \leftarrow r_{\text{pen}}$;
  else
    if $ph = 1$ **then** `// Phase 1`
      $r_t \leftarrow (h - r_{\text{pen}}) \exp(-\frac{(v_t - v_{\text{high}})^2}{2\,w^2}) + r_{\text{pen}}$;
    else if $ph = 2$ **then** `// Phase 2`
      $r_t \leftarrow r_{\text{pen}} + \sum_{i \in \{\text{low,high}\}} \Big[h_i - r_{\text{pen}}\Big] \exp(-\frac{(v_t - v_i)^2}{2\,w_i^2})$;
    else `// Phase 3`
      if $|v_t - v_{low}| \leq 1 \ \vee \ |v_t - v_{high}| \leq 1$ **then**
        $r_t \leftarrow 1.0$
      else
        $r_t \leftarrow 0$

4 Methodology

In this section, we detail our complete pipeline for training PPO-based agents to manage containers. We begin with a *naive PPO* baseline, highlighting its struggles with sparse, multimodal rewards and the tendency toward premature empties. To address these issues, we then propose a *curriculum learning* scheme (PPO-CL) that incrementally shapes the agent's reward landscape. This two-stage progression—naive PPO followed by PPO-CL—lays the groundwork for an inference-time *collision model*, which further mitigates overflow risks when multiple containers compete for the PU.

4.1 Naive PPO Baseline and Its Shortcomings

A straightforward approach is to train a PPO agent directly on the final (multimodal) reward that encourages emptying containers at either the higher or lower peak. However, as outlined in Section 3, this environment poses multiple challenges: rewards are delayed and infrequent, containers fill at varying rates, and collisions can arise when the PU services multiple containers simultaneously. Without additional

structure or foresight, empirical results (see table 9.7) show that a naive PPO agent tends to:

- **Ignore long-term returns.** Driven by sparse rewards, the agent often performs multiple partial empties rather than waiting for the larger peak. This short-sighted strategy blocks the PU more frequently, increasing energy usage and leaving less capacity for containers that are about to overflow.
- **Struggle with skewed action distributions.** With many "do-nothing" steps before a container reaches its target volume, the agent fails to learn precise timing to consistently hit higher-volume empties.
- **Overlook future collisions.** Having no explicit mechanism to anticipate bottlenecks on the PU, the agent may wait too long and face simultaneous arrivals at near-peak volumes, risking overflow or forced early empties.

4.2 Curriculum Learning with PPO

These shortcomings motivate a more structured approach to handle delayed rewards, skewed actions, and collision risks. We therefore present a *curriculum learning* strategy that gradually refines the agent's timing and decision-making. We train a PPO agent with a *three-phase reward curriculum*, gradually introducing complexity over successive training segments. In each phase, the agent follows the same *state* and *action* definitions, but the reward function evolves to guide the agent toward better timing of empties:

- **Phase 1 (Unimodal Reward):** We place a single Gaussian peak at the higher peak volume and no reward at the lower peak, helping the agent learn to avoid excessively early empties.
- **Phase 2 (Multimodal Reward):** Two Gaussian peaks (higher and lower), letting the agent discover a fallback if waiting for the higher peak is unsafe or if the PU is unavailable.
- **Phase 3 (Step Reward):** A strict scheme awarding positive reward only when the emptied volume is within a narrow window (± 1) around *either* peak, refining precision once the agent learned to handle both targets.

As in [7, 12], we further stabilize training by *freezing* the policy network in parts of Phase 2, updating only the value estimator to account for changes in reward structure. During Phase 3, we *unfreeze* the policy network but apply a stricter KL-divergence constraint, ensuring the agent does not deviate too aggressively from the policy learned in earlier phases. Algorithm 9.2 presents the logic for all three phases. By stepping through these phases with carefully tuned budgets, the agent (*PPO-CL*) acquires more robust scheduling behaviors than a naive single-stage approach. In particular, it learns to occasionally pick the lower peak to

avert overflow. However, PPO-CL alone remains largely myopic about *collisions* across containers, motivating our inference-time mitigation strategy in the next section.

5 Collision Model and Inference Pipeline

Although PPO-CL improves emptying behavior, it still shows no signs of successfully learning about collision risks. We mitigate the problem by designing a mechanism to handle situations where multiple containers simultaneously approach their peak volumes. We address this by introducing a *collision model (CM)* trained offline on pairwise container data, then integrating it with the PPO-CL agent at inference time to form **PPO-CL-CM**. This approach balances high-volume empties with timely overrides to avert collisions, all at minimal run-time overhead.

Monte Carlo Rollouts for Pairwise Collisions: For efficient inference-time planning, we generate a large offline dataset of pairwise collision scenarios. By simulating isolated or small groups of containers, we capture diverse collision states with minimal run-time cost. We simulate each container pair (i, j) under stochastic filling (9.1). Two million repetitions across a container configuration yield a comprehensive offline data set of near-capacity, overflow, and collision events, minimizing deployment computation. For each pair, we perform:

1. Random initialization: Sample means and standard deviations $\mu_i, \mu_j, \sigma_i, \sigma_j$ for filling rates.
2. Stochastic evolution: Evolve volumes $v_i(\tau), v_j(\tau)$ over timesteps τ.
3. Collision bookkeeping: Record collisions when both containers near peaks while the PU is busy.

Feature Extraction and Training At each simulation timestep, we extract collision-predictive features:

- Volumes: Container states (v_i, v_j)
- Proximity to peaks: $\Delta v_i = p_i - v_i, \Delta v_j = p_j - v_j$
- Filling parameters: $(\mu_i, \sigma_i, \mu_j, \sigma_j)$
- Time-lag context: Optional short-volume histories $\{v_i(\tau - 1), v_j(\tau - 1)\}$

Each timestep receives a *collision label* $\{0, 1\}$, creating a supervised dataset $\{\text{features}, \text{collision label}\}$ for training. From this dataset, we train an XGBoost classifier to estimate collision probabilities: $P(C_{i,j} \mid s_t, a_t) = f_{\text{col}}(\text{features}_{i,j})$. XGBoost's gradient-boosted trees efficiently model complex relationships between volumes, fill rates, and near-capacity states. Tuned XGBoost offers fast, accurate pairwise collision risk predictions.

Inference-Time Integration with PPO-CL: Once trained, the collision model f_{col} is invoked at each decision step to assess pairwise collision probabilities. The *baseline* PPO-CL agent first proposes an action a_t. If it decides to empty a specific container ($a_t \neq 0$), we accept that choice directly. However, if PPO-CL proposes *no operation* ($a_t = 0$), the system queries f_{col} for all container pairs to estimate near-future collision risks. If any container at high volume is flagged with a collision probability above a threshold θ, we *override* the no-op by forcing an empty action on the most at-risk container. Algorithm 9.3 details the procedure.

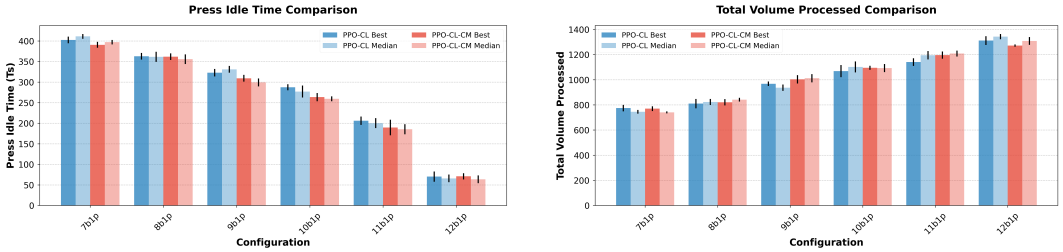
Action Override Rationale. By limiting overrides to the no-op action, we minimally disturb PPO-CL's learned preference for waiting until containers reach higher volumes. Only when the model detects a high probability of overflow or severe collisions do we *force* an empty on a likely-to-clash container. This blend of *offline collision modeling* and *selective inference-time planning* yields a more collision-aware agent.

Algorithm 9.3: Integrated Inference-Time Decision with Pairwise Collision Prediction

Input : State s_t, PPO-CL policy π_{CL}, collision model f_{col}, threshold θ, peak volumes $\{p_i\}$

Output: Final action $a_{\text{final}} \in \{0, 1, \ldots, n\}$

1. PPO-CL Action: $a_t \sim \pi_{\text{CL}}(a_t \mid s_t)$;
if $a_t \neq 0$ **then**
  return $a_{\text{final}} \leftarrow a_t$; `// Accept non-zero action`
2. Collision Assessment:
foreach *pair* (i, j) **do**
  $P(C_{i,j}) \leftarrow f_{\text{col}}(\text{features}_{i,j})$;
Assemble matrix $P_t[i, j] = P(C_{i,j})$;
3. Check Potential Collision Overrides:
$\mathcal{C} \leftarrow \{\, i \mid v_i(t) \geq p_i - \delta \}$;
if $\mathcal{C} \neq \emptyset$ **then**
  foreach $i \in \mathcal{C}$ **do**
    $\text{CollisionRisk}(i) \leftarrow \textsc{RiskScore}(i, P_t)$
  $i^\star \leftarrow \arg\max_{i \in \mathcal{C}} [\text{CollisionRisk}(i)]$;
  if $\mathit{CollisionRisk}(i^\star) \geq \theta$ **then**
    $a_{\text{final}} \leftarrow i^\star$; `// Override with container` $i^\star$
  else
    $a_{\text{final}} \leftarrow 0$; `// Retain no-op`
  return a_{final}
return $a_{\text{final}} \leftarrow 0$; `// No containers at risk, do nothing`



(a) Press idle time comparison.

(b) Total volume processed comparison.

Figure 9.4: Performance metrics comparison between PPO-CL and PPO-CL-CM methods across different container configurations (7b1p to 12b1p). The bars show mean values and error bars indicate standard deviation. Left: Press idle time shows the duration the press remains inactive. Right: Total volume processed indicates the amount of material handled during one inference episode of 600 timesteps.

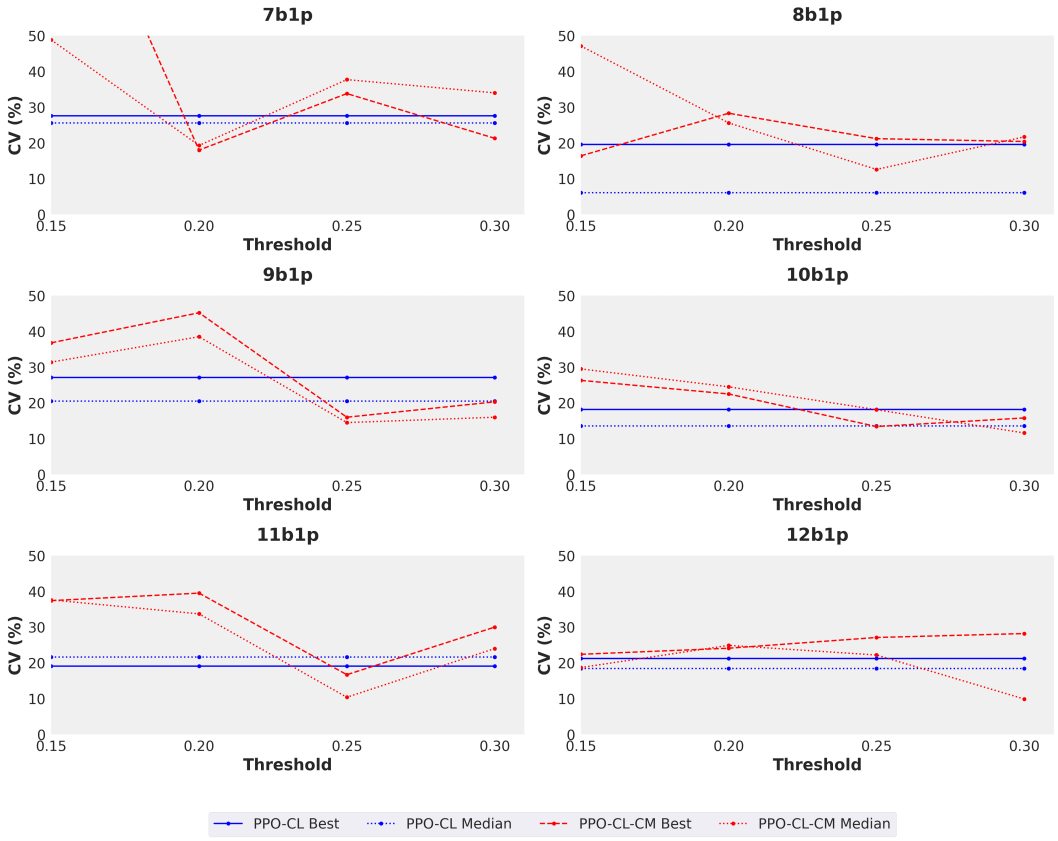


Figure 9.5: Comparison of Coefficient of Variation (CV%) across different collision probability thresholds for all container configurations. Each subplot shows the performance of PPO-CL and PPO-CL-CM methods for a specific configuration. Lower CV% indicates more consistent performance.

6 Experimental Evaluation

In this section, we present a quantitative evaluation of the three agents naive PPO, PPO-CL, and PPO-CL-CM across multiple container-to-PU configurations. Our analysis addresses the following research questions (**RQs**):

1. **RQ1:** Is the inference-time collision model (*PPO-CL-CM*) effective compared to PPO-CL and naive PPO in terms of achieved reward?
2. **RQ2:** How effectively does the collision model reduce safety-limit violations (i.e., empties above the critical volume limit)?
3. **RQ3:** How do these findings inform real-world design choices, such as the ideal ratio of containers to processing units?

In the spirit of open and reproducible research, we make our source code available via an ***anonymous repository***.[2] The repository contains a script for reproducing all results presented in this section.

6.1 Experimental Setup

We evaluate our methods on environments with *7 to 12* containers and a single PU (labeled "7b1p" through "12b1p"). Following [8] we use an episode length of 600 timesteps, with a 60-second granularity. For each agent, we conduct *15* independent training runs using distinct random seeds. During inference, we run each seed-based policy in *5* rollouts and collect statistics. We present results for both the *best* and *median* performing seeds of each method, ensuring a fair and comprehensive comparison. Specifically, the *best seed* is selected based on the smallest number of *collision timesteps*, while the *median seed* is chosen from the remaining seeds in terms of the same metric.

Metrics: The reward signal aggregates many different subgoals. To obtain a fine-grained picture of algorithm performance, we monitor the following performance metrics:

- *Press Idle Time* (Figure 9.4a): total timesteps during which the PU is not processing any container.
- *Total Volume Processed* (Figure 9.4b): material throughput in one 600-step inference episode.

[2] `https://gitlab.com/anonymousppocl_cm1/anonymous_collisions_paper`

- *Collisions in time-steps*: the total number of timesteps in which a *collision state* occurs; that is, at least two containers simultaneously approach or exceed their ideal volumes (especially the higher peak) while the PU is busy and thus unable to service them.
- *Coefficient of Variation (CV%) (Figure 9.5)*: a normalized measure of variability in performance under different collision thresholds.
- *Total Volume Deviation*: the average (over all containers and timesteps) of the absolute difference between a container's actual volume and its nearest ideal peak, gauging how well empties align with target volumes.
- *Actions per Container, Reward per Action, and Peak-Usage Ratios*: drawn from Tables 9.6 and 9.7.
- *Safety-Limit Violation Percentage* (Figure 9.8): the fraction of emptying actions during inference where a container volume exceeds a fixed ***critical volume limit***, set 5 units above that container's higher ideal emptying peak.

All agents are tested under the same environmental conditions to isolate the impact of curriculum training and collision modeling. Moreover, the results shown in Figures 9.4 and 9.8 and in Tables 9.6 and 9.7 are reported using the threshold values that yield the lowest CV% in Figure 9.5, reflecting the most collision-stable configurations in our analysis.

6.2 Impact of Collision Model (RQ1)

From Table 9.7, we note that naive PPO often fails to empty containers at the *higher* ideal peak altogether (*e.g.*, ratio close to zero), indicating it resorts to early or sub-optimal empties. This behavior leads to more frequent episodes ending prematurely due to overflow (especially in the larger "12b1p" setup) or consistently high volume deviations. While PPO-CL capitalizes on the multi-phase reward shaping to handle delayed feedback and class imbalance, collisions can still occur if multiple containers simultaneously converge on their higher peaks. This is where incorporating *inference-time collision checks* to yield PPO-CL-CM has an edge.

As shown in Figure 9.5, PPO-CL-CM achieves lower variability (CV%) across different collision probability thresholds, meaning it more consistently avoids hazardous states. From Figure 9.4a, we see PPO-CL-CM generally reduces press idle time compared to PPO-CL, indicating fewer deadlocks where containers are left unemptied until near-overflow conditions. Meanwhile, Figure 9.4b shows that *total volume processed* remains at least on par with (and often surpasses) PPO-CL, demonstrating that collision avoidance does not compromise overall throughput in an inference episode.

Table 9.6 reveals that *collision timesteps* drop systematically for *PPO-CL-CM*, and its *volume deviation* is also slightly lower on average. The *higher/lower peak ratio* in Table 9.7 confirms that PPO-CL-CM empties containers earlier (lower peak) only when the collision model flags imminent risk, thereby balancing high-throughput empties with safety. Hence, **RQ1** is answered: adding an inference-time collision model mitigates bottlenecks and collisions beyond what curriculum-based RL can achieve alone.

6.3 Safety-Limit Violations (RQ2)

Figure 9.8 summarizes the frequency of empties that exceed the safety critical volume limit, thus posing a higher risk of overflow and breach of physical limit. We observe that **PPO-CL-CM** consistently maintains a smaller fraction of risky empties overall than PPO-CL for all bunker-to-PU setups from *7b1p* through *12b1p*. This indicates that the collision model not only reduces direct collisions but also prompts timely empties before containers venture into risky volume ranges. Hence, we conclude **RQ2** by confirming that inference-time collision checks can effectively mitigate dangerous critical empties, supporting safer operation without sacrificing throughput.

6.4 Real-World Implications and Design Guidance (RQ3)

Figure 9.4a illustrates that as we move from "7b1p" up to "12b1p," *press idle time* diminishes significantly, reflecting how the PU becomes fully utilized with rising container counts. On the other hand, in extremely large configurations (e.g., 12 containers to 1 PU), collisions inevitably remain because a single resource cannot realistically handle multiple near-peak arrivals at once. While PPO-CL-CM can curb severe collisions, it cannot eliminate them entirely when the resource ratio is unfavorable. Thus for *Moderate Ratios:* Up to around 7–11 containers per PU, collision avoidance measures like PPO-CL-CM yield strong improvements without saturating the system. But for *High Ratios:*, beyond a certain limit (e.g., 12b1p), press idle time becomes negligible, and collision events dominate. An additional PU may be necessary for higher container configurations. Addressing **RQ3**, in practice, operators can apply these findings when deciding how many containers can be feasibly connected to a single processing unit, and whether advanced collision checks are cost-effective. Importantly, applying a collision model allows to safely operate more containers with the same number of expensive PUs.

Config	Agent	Collisions (Ts)	Tot. vol. dev (%)	Reward/action
7b1p	PPO-CL	72.4±20.0 / 81.6±20.9	7.3±2.1 / 6.9±1.2	0.4 / 0.4
	PPO-CL-CM	**22.0±3.9** / **34.4±6.7**	**5.1±0.7** / **6.1±0.6**	**0.7** / **0.8**
8b1p	PPO-CL	77.2±15.1 / 98.6±6.1	7.4±0.8 / 7.3±0.9	0.3 / 0.4
	PPO-CL-CM	**66.2±14.0** / **80.6±10.2**	**6.7±1.2** / **7.0±0.7**	**0.6** / **0.5**
9b1p	PPO-CL	117.0±31.7 / 153.4±31.4	6.5±1.4 / 7.5±1.1	0.4 / 0.3
	PPO-CL-CM	**89.8±14.4** / **117.0±17.0**	**6.3±1.4** / 7.6±0.7	**0.6** / **0.4**
10b1p	PPO-CL	154.6±28.1 / 189.8±25.8	6.8±1.2 / 8.7±0.9	0.4 / 0.2
	PPO-CL-CM	**138.2±18.5** / **145.4±26.3**	**6.4±0.4** / **6.8±0.9**	**0.5** / **0.5**
11b1p	PPO-CL	130.8±25.0 / 156.4±33.8	8.0±1.3 / 6.9±0.5	0.3 / 0.3
	PPO-CL-CM	**83.2±13.9** / **148.4±15.4**	**7.6±1.2** / **6.8±0.7**	**0.5** / **0.5**
12b1p	PPO-CL	126.4±26.8 / 195.8±36.0	11.8±2.1 / 10.2±1.3	0.3 / 0.3
	PPO-CL-CM	**117.4±31.9** / **175.6±38.9**	13.0±2.2 / 11.2±1.6	**0.5** / **0.4**

Table 9.6: Main performance metrics (Best/Median) for each bunker configuration, all rounded to one decimal place. Each cell shows *Best* ± std. / *Median* ± std. in one line. We boldface the **better** result in PPO-CL-CM whenever it outperforms PPO-CL (lower collisions/dev or higher reward).

Config	Type	Higher/Lower Peak % Ratio			Actions/Container		
		Naive PPO	**PPO-CL**	**PPO-CL-CM**	**Naive PPO**	**PPO-CL**	**PPO-CL-CM**
7b1p	Best	0.0	45.7	1.0	32.7±8.9	20.0±3.4	25.6±2.6
	Median	0.0	18.6	0.8	29.6±12.3	19.6±2.8	24.4±3.1
8b1p	Best	0.0	8.4	2.2	31.0±10.0	18.9±3.8	20.8±3.9
	Median	0.0	14.0	2.9	31.2±8.6	18.8±3.1	20.9±4.7
9b1p	Best	0.0	16.5	4.5	33.3±9.0	19.4±3.4	21.2±2.7
	Median	0.0	19.6	3.3	34.8±9.9	18.3±4.1	21.4±3.1
10b1p	Best	0.0	30.5	3.3	33.2±10.3	18.9±2.5	20.8±4.2
	Median	0.0	11.7	3.1	31.6±8.0	19.1±2.8	20.7±5.0
11b1p	Best	0.0	3.1	1.8	34.1±6.3	20.6±4.6	22.5±4.3
	Median	0.0	6.7	2.9	32.9±7.6	20.3±2.7	21.7±3.5
12b1p	Best	N/A	1.9	1.6	N/A	23.1±7.0	23.2±8.1
	Median	N/A	2.6	2.4	N/A	22.1±4.8	22.1±6.1

Table 9.7: Comparison of Higher/Lower Peak Percentage Ratio and Mean Actions per container (single decimal precision). For each configuration, "Best" / "Median" rows show mean ± standard deviation where applicable.

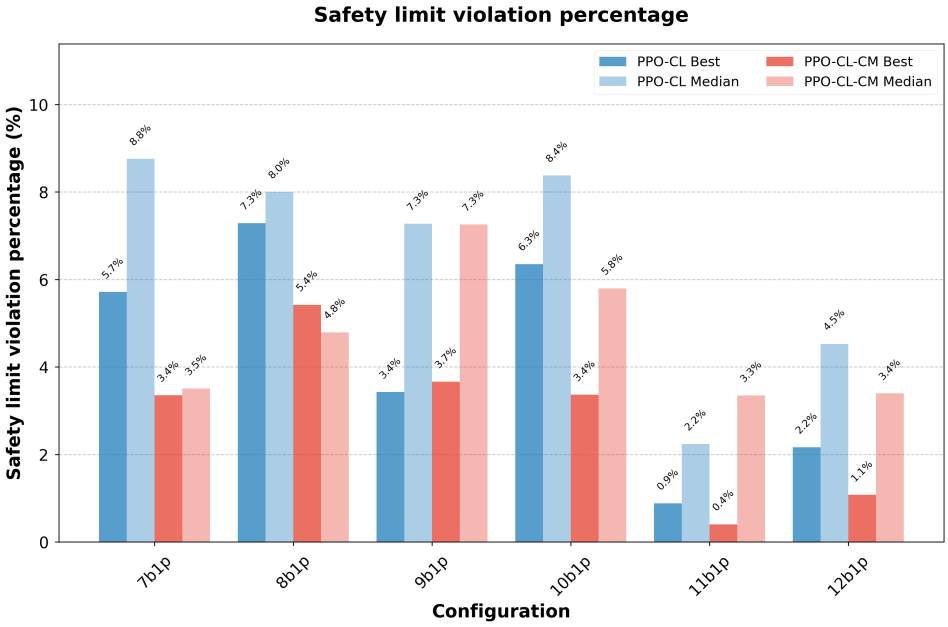


Figure 9.8: Comparison of safety limit violation percentages across different bunker configurations. Bars show the percentage of emptying actions that exceeded the safety limit for each bunker configuration using PPO-CL and PPO-CL-CM methods.

7 Conclusion

We have presented a hybrid strategy for container management in a waste-sorting facility in a constrained resource scenario, where each container has two preferred "ideal" emptying volumes. The first component, *PPO-CL*, addresses the challenges of sparse rewards and dual-volume targets through a curriculum-learning approach that teaches the agent to empty containers near either a lower or higher peak. The second component, a *collision model* integrated at inference time (*PPO-CL-CM*), provides targeted overrides when containers risk colliding at peak volumes. This combination effectively balances the key design criteria: prioritizing the higher peak for optimal throughput while resorting to the lower peak only when collisions or safety violations become imminent.

Empirical results across various container-to-PU ratios (7:1 to 12:1) demonstrate that *PPO-CL-CM* reduces both collision episodes and empties above a critical safety threshold, *without sacrificing total processed volume*. By explicitly modeling future collision risk, it prevents premature empties that might otherwise occur if the agent tried to avoid collisions by abandoning the higher peak too soon. From an operational standpoint, these findings suggest that facility managers can confidently scale up container counts to a point, relying on our framework to avoid overflows and to ensure safe, high-volume empties. Beyond that point, collisions become unavoidable, but our method still lessens their severity.

A notable advantage of this framework is that the collision model is trained entirely offline via Monte Carlo simulations, adding only minimal overhead during inference. This approach stands in contrast to computationally intensive online planners like Monte Carlo Tree Search, making it better suited for large, stochastic industrial environments with real-time decision needs. Looking ahead, we envision several avenues to extend this work: integrating multiple PUs (or more complex resource constraints) within the same collision-avoidance framework, dynamically tuning collision thresholds based on time-of-day inflows or real-time capacity data, and more tightly fusing offline collision insights with online RL updates to further refine scheduling decisions. Our findings highlight that a domain-aware collision model, combined with carefully shaped RL curricula, can yield safer, more efficient container management, establishing a robust framework for broader industrial adoption and more complex resource-allocation tasks.

Acknowledgements. This work was funded by the German federal ministry of economic affairs and climate action through the "ecoKI" grant.

References

[1] Cameron B. Browne et al. “A Survey of Monte Carlo Tree Search Methods”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (2012), pp. 1–43.

[2] J. Cestero et al. “Storehouse: a Reinforcement Learning Environment for Optimizing Warehouse Management”. In: *Proceedings of the 2022 International Joint Conference on Neural Networks (IJCNN)*. 2022, pp. 1–9.

[3] Ashenafi Teshome Eseye, Xu Zhang, Bennett Knueven, et al. “A hybrid reinforcement learning–MPC approach for distribution system critical load restoration”. In: *Proceedings of the IEEE Power and Energy Society General Meeting (PESGM)*. 2022.

[4] Carl-Johan Hoel et al. “Combining Planning and Deep Reinforcement Learning in Tactical Decision Making for Autonomous Driving”. In: *IEEE Transactions on Intelligent Vehicles* 5.2 (2020), pp. 294–305.

[5] H. Ikeda, H. Nakagawa, and T. Tsuchiya. “Towards Automatic Facility Layout Design Using Reinforcement Learning”. In: *Communication Papers of the 17th Conference on Computer Science and Intelligence Systems*. Vol. 32. Annals of Computer Science and Information Systems. 2022, pp. 11–20.

[6] Sanmit Narvekar et al. “Curriculum learning for reinforcement learning domains: a framework and survey”. In: *Journal of Machine Learning Research* 21.1 (Jan. 2020), pp. 7382–7431.

[7] Abhijeet Pendyala, Asma Atamna, and Tobias Glasmachers. “Solving a Real-World Optimization Problem Using Proximal Policy Optimization with Curriculum Learning and Reward Engineering”. In: *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track*. Springer Nature Switzerland, 2024, pp. 150–165.

[8] Abhijeet Pendyala et al. “ContainerGym: A Real-World Reinforcement Learning Benchmark for Resource Allocation”. In: *Machine Learning, Optimization, and Data Science*. Springer Nature Switzerland, 2024, pp. 78–92.

[9] Arthur Romero et al. *Actor-Critic Model Predictive Control: Differentiable Optimization Meets Reinforcement Learning*. arXiv preprint arXiv:2306.09852. 2023. arXiv: 2306.09852.

[10] Yi Shen et al. “Multi-Agent Reinforcement Learning for Resource Allocation in Large-Scale Robotic Warehouse Sortation Centers”. In: *Proceedings of the 2023 62nd IEEE Conference on Decision and Control (CDC)*. 2023, pp. 7137–7143.

[11] David Silver et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587 (2016), pp. 484–489.

[12] Xiangjun Wang et al. *SCC: an efficient deep reinforcement learning agent mastering the game of StarCraft II*. arXiv preprint arXiv:2012.13169. Dec. 2020. arXiv: 2012.13169.

[13] Xin Wang, Yudong Chen, and Wenwu Zhu. "A Survey on Curriculum Learning". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.9 (Sept. 2022), pp. 4555–4576.

[14] Zhibin Wang, Weijie Wei, Anji Xie, et al. "Hybrid bipedal locomotion based on reinforcement learning and heuristics". In: *Micromachines* 13.10 (2022), p. 1688.

10 Conclusion and Discussion

This thesis embarked on the challenge of optimizing complex industrial processes through the application of advanced Reinforcement Learning techniques. Motivated by the limitations of traditional control methods in handling the stochasticity, resource constraints, and safety requirements inherent in modern industrial environments, this research focused on a critical real-world bottleneck: the management of containers in high-throughput waste sorting facilities. This chapter synthesizes the key findings of the thesis, revisits the initial research questions, discusses the broader implications and limitations of the work, and outlines promising avenues for future research.

10.1 Synthesis of Findings and Contributions

The research was structured progressively across three main phases, moving from problem formalization to the development of increasingly sophisticated control architectures. The findings are summarized below, addressing the research questions posed in Chapter 1.

10.1.1 Phase 1: Formalization and Benchmarking

The initial phase addressed the need for a realistic and standardized platform for developing RL solutions for industrial resource allocation.

- **RQ1 (Modeling):** *How can the complex, stochastic, and resource-constrained container management problem be effectively modeled as an RL environment?*

  This was addressed through the development and validation of **ContainerGym** (Chapter 7), an open-source benchmark environment derived directly from the industrial use case. ContainerGym successfully formalizes the problem as a Markov Decision Process (MDP), capturing its key characteristics: stochastic inflow rates, noisy observations, severe resource contention (shared Processing Units), optimal emptying volumes, and hard safety constraints (overflow prevention). This environment served as the foundational platform for all subsequent research. (Contribution 1).

- **RQ2 (Baseline Performance):** *How do standard, state-of-the-art RL algorithms perform on this real-world task, and what are their primary limitations?*

  Comprehensive benchmarking revealed that standard algorithms (e.g., PPO, TRPO, DQN) fail to learn effective policies when trained directly on the ContainerGym environment. Their primary limitations stem from the challenges of sparse and delayed rewards, the difficulty of exploration in a safety-constrained state space, and the inability to effectively manage the long-term consequences of resource contention. These failures motivated the investigation of advanced training methodologies.

10.1.2 Phase 2: Advanced RL for Multi-Criteria Control

The second phase focused on overcoming the limitations identified in the baseline analysis by structuring the learning process.

- **RQ3 (Curriculum Learning and Reward Engineering):** *Can a structured training methodology, combining Curriculum Learning and tailored Reward Engineering, effectively enable an agent (PPO) to learn a robust policy that balances competing objectives?*

  This question was affirmatively answered through the development of the **PPO-CL** methodology (Chapter 8). A multi-stage Curriculum Learning (CL) strategy was designed to systematically guide the agent's learning process, gradually introducing environmental complexity (stochasticity, resource constraints) and refining the reward structure (including positional, termination, and precision-focused rewards). This approach proved essential in enabling the agent to handle delayed rewards, learn safety protocols, manage energy considerations, and achieve high performance. PPO-CL significantly outperformed naive PPO training, successfully solving complex, multi-criteria configurations of the task. (Contribution 2).

10.1.3 Phase 3: Hybrid Architectures and Scalability

The final phase addressed the limitations of the purely reactive PPO-CL policy, specifically its inability to proactively manage resource contention in high-load scenarios.

- **RQ4 (Collision Avoidance and Foresight):** *How can the critical issue of Processing Unit (PU) collisions be effectively addressed, particularly in high-contention scenarios requiring long-term foresight?*

  This was addressed by recognizing the "strategic myopia" (Section 6.1) of the reactive PPO-CL agent and identifying the need for explicit foresight. The

solution involved developing an efficient mechanism to predict future collisions without resorting to computationally expensive online planning.

- **RQ5 (Hybrid RL and Planning):** *Can integrating the RL policy with planning techniques enhance safety and collision avoidance compared to a purely reactive RL approach?*

  This was achieved through the development of the **PPO-CL-CM** hybrid architecture (Chapter 9). This approach augments the PPO-CL agent with an offline-trained Collision Model (CM), built using Monte Carlo simulations. At inference time, the CM predicts the likelihood of imminent collisions. If a high risk is detected, the collision model overrides the PPO-CL agent and forces a preemptive empty. This hybrid approach demonstrated a significant reduction in collision events and safety violations compared to the PPO-CL agent, confirming the efficacy of integrating learned skills with predictive foresight. (Contribution 3).

- **RQ6 (Scalability and Design Insights):** *What practical insights regarding facility design and operational limits can be derived from evaluating these advanced control strategies?*

  Systematic evaluation across various container-to-PU ratios (from 7:1 to 12:1) provided quantitative insights into the scalability of the system under different control strategies. The analysis established the practical limits of how many containers can be safely managed by a single PU, offering actionable, data-driven guidelines for facility design and configuration. (Contribution 4).

10.2 Discussion and Broader Implications

The contributions of this thesis have significant implications for both the theory of Reinforcement Learning and its practical application in industrial automation.

10.2.1 The Necessity of Structured Learning for Industrial RL

A key takeaway from this research is that the direct application of powerful RL algorithms (like PPO) is often insufficient for complex industrial tasks. The success of the PPO-CL methodology shows the critical importance of **structured learning**. Curriculum Learning (Chapter 5) and meticulous Reward Engineering (Chapter 4) are not merely optimizations but essential enabling methodologies. They provide the necessary scaffolding to manage the complexity of the optimization landscape, guide exploration, and embed domain knowledge into the learning process. This thesis provides a validated framework for designing such curricula in the context of resource allocation problems.

10.2.2 Hybrid Architectures: Efficient Foresight and Safety

The development of the PPO-CL-CM architecture demonstrates the power of hybrid systems that combine model-free RL with specialized predictive models. Traditional Model-Based RL (MBRL) often struggles with the computational cost of online planning. The approach taken here—using an efficient, specialized predictor (the Collision Model) for inference-time intervention—provides a practical and effective alternative. It delivers the necessary foresight for safety and coordination without the latency of online search, making it suitable for real-time industrial control. This contributes to the field of Safe RL by providing a robust Learner-Verifier framework (Section 6.4.1).

10.2.3 The Role of Simulation and Digital Twins

This research highlights the indispensable role of high-fidelity simulation. ContainerGym served not only as a training environment but also as a "digital twin" of the industrial process. It enabled the safe exploration of control strategies, the generation of data for the Collision Model via Monte Carlo simulation, and the systematic analysis of system scalability (RQ6). This feedback loop between simulation, control optimization, and system design is a key enabler for data-driven engineering.

10.2.4 Impact on Sustainability and Efficiency

In the context of waste management and the circular economy (Chapter 1), the developed control solutions offer tangible benefits. By optimizing throughput, minimizing energy consumption, and preventing costly downtime due to overflows, the PPO-CL-CM agent contributes directly to the efficiency and sustainability of recycling operations.

10.3 Limitations

Despite the significant advancements presented, this research has several limitations that must be acknowledged, primarily concerning the transition from simulation to reality, the methodology's reliance on manual design, and the scalability of the architecture.

The Simulation-to-Reality Gap. The primary limitation is that the methodologies were developed and validated within the ContainerGym simulation. While designed to capture the core dynamics of the real-world system, ContainerGym remains an abstraction. Real-world deployment will introduce complexities not fully modeled, including discrepancies in sensor noise profiles, actuator dynamics (e.g., communication delays, physical conveyor behavior), variations in material properties (e.g., density affecting emptying times), and unforeseen disturbances such as equipment malfunctions. These factors may affect the performance and robustness of the trained policy.

Manual Curriculum Design and Expertise. The success of the PPO-CL methodology relied heavily on a manually engineered, multi-stage curriculum. Designing this curriculum required significant domain expertise and iterative experimentation. This reliance on manual effort can be time-consuming and may limit the ease with which the approach can be transferred to new, dissimilar tasks or adapted by non-experts.

Model Robustness and Non-Stationarity. The effectiveness of the hybrid architecture (PPO-CL-CM) is contingent upon the accuracy of the Collision Model, which was trained offline based on the simulator. If the real-world dynamics diverge significantly from the simulation, or if the environment is non-stationary (e.g., seasonal changes affecting inflow rates), the CM's predictions may degrade. The current framework lacks an automated mechanism for detecting and adapting to such environmental drift.

10.4 Future Work

The limitations identified above open up several promising avenues for future research, focusing on real-world deployment, automation of the methodology, and architectural enhancements.

Real-World Deployment and Continual Adaptation. The most immediate next step is the deployment of the PPO-CL-CM system in a physical facility. This will require addressing the sim-to-real gap through techniques such as domain randomization and system identification. Furthermore, to address non-stationarity, future work should investigate **continual learning** techniques. This would enable the RL policy and the Collision Model to be fine-tuned online based on real-world data, allowing the system to adapt to environmental drift without requiring full retraining.

Automation of the Learning Process. To reduce the reliance on manual engineering, future research should explore **Automated Curriculum Generation (ACG)**. Techniques based on intrinsic motivation or teacher-student architectures (Section 5.2) could potentially discover the optimal sequence of training tasks automatically, making the methodology more accessible and transferable.

Decentralized Architectures and Advanced Representations. To enhance scalability and robustness, exploring decentralized control architectures using **Multi-Agent Reinforcement Learning (MARL)** is a promising direction. Furthermore, the use of advanced representations, such as Graph Neural Networks (GNNs), could better capture the interdependencies between containers, potentially leading to improved generalizability across different facility layouts.

10.5 Concluding Remarks

This thesis has demonstrated the significant potential of combining structured Reinforcement Learning techniques with domain-specific predictive modeling to solve challenging industrial optimization problems. By developing a realistic benchmark, a robust curriculum learning framework, and a novel hybrid RL-planning architecture, this research provides a comprehensive methodology for developing safe, efficient, and scalable control solutions. The successful optimization of the container management problem illustrates the transformative impact that the synergy between artificial intelligence and industrial automation can have in addressing critical challenges in sustainability and resource management.

Bibliography

[1] Massimiliano Agovino et al. "European waste management regulations and the transition towards circular economy. A shift-and-share analysis". In: *Journal of Environmental Management* 354 (2024), p. 120423. ISSN: 0301-4797. DOI: https://doi.org/10.1016/j.jenvman.2024.120423. URL: https://www.sciencedirect.com/science/article/pii/S0301479724004092.

[2] Greg Brockman et al. *OpenAI Gym*. 2016. eprint: arXiv:1606.01540.

[3] Cameron Browne et al. "A Survey of Monte Carlo Tree Search Methods". In: *IEEE Transactions on Computational Intelligence and AI in Games* 4 (2012), pp. 1–43. URL: https://api.semanticscholar.org/CorpusID:9316331.

[4] Cameron B. Browne et al. "A Survey of Monte Carlo Tree Search Methods". In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (2012), pp. 1–43.

[5] Logan Engstrom et al. "Implementation matters in deep policy gradients: a case study on PPO and TRPO". In: *International Conference on Learning Representations*. 2020.

[6] Carlos Florensa et al. "Reverse Curriculum Generation for Reinforcement Learning". In: *Proceedings of the 1st Conference on Robot Learning (CoRL)*. 2017.

[7] Sébastien Forestier et al. "Intrinsically motivated goal exploration processes with automatic curriculum learning". In: 23.1 (Jan. 2022). ISSN: 1532-4435.

[8] Javier García and Fernando Fernández. "A comprehensive survey on safe reinforcement learning". In: *Journal of Machine Learning Research* 16.1 (2015), pp. 1437–1480.

[9] Kashish Gupta, Debasmita Mukherjee, and Homayoun Najjaran. "Extending the Capabilities of Reinforcement Learning Through Curriculum: A Review of Methods and Applications". In: *SN Computer Science* 3.1 (Oct. 2021), p. 28.

[10] Shengyi Huang and Santiago Ontañón. *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*. arXiv preprint arXiv:2006.14171. June 2020. arXiv: 2006.14171.

[11] Sinan Ibrahim et al. "Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications". In: *IEEE Access* 12 (2024), pp. 175473–175500. DOI: 10.1109/ACCESS.2024.3504735.

[12] Tambet Matiisen et al. "Teacher–Student Curriculum Learning". In: *IEEE Transactions on Neural Networks and Learning Systems* 31 (2017), pp. 3732–3740. URL: https://api.semanticscholar.org/CorpusID:8432394.

[13] Volodymyr Mnih et al. "Playing Atari with Deep Reinforcement Learning". In: *CoRR* abs/1312.5602 (2013). arXiv: 1312.5602.

[14] Thomas M. Moerland et al. *A Unifying Framework for Reinforcement Learning and Planning*. 2022. arXiv: 2006.15009 [cs.LG].

[15] Sanmit Narvekar et al. "Curriculum learning for reinforcement learning domains: a framework and survey". In: *Journal of Machine Learning Research* 21.1 (Jan. 2020), pp. 7382–7431.

[16] Andrew Y Ng, Daishi Harada, and Stuart Russell. "Policy invariance under reward transformations: Theory and application to reward shaping". In: *Machine Learning* 34.1-3 (1999), pp. 101–121.

[17] OpenAI et al. "Solving Rubik's Cube with a Robot Hand". In: *ArXiv* abs/1910.07113 (2019). URL: https://api.semanticscholar.org/CorpusID:204734323.

[18] Abhijeet Pendyala, Asma Atamna, and Tobias Glasmachers. "Solving a Real-World Optimization Problem Using Proximal Policy Optimization with Curriculum Learning and Reward Engineering". In: *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track*. Springer Nature Switzerland, 2024, pp. 150–165.

[19] Abhijeet Pendyala and Tobias Glasmachers. *Curriculum RL meets Monte Carlo Planning: Optimization of a Real World Container Management Problem*. 2025. arXiv: 2503.17194 [cs.LG]. URL: https://arxiv.org/abs/2503.17194.

[20] Abhijeet Pendyala et al. "ContainerGym: A Real-World Reinforcement Learning Benchmark for Resource Allocation". In: *Machine Learning, Optimization, and Data Science*. Springer Nature Switzerland, 2024, pp. 78–92.

[21] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[22] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[23] Martin Riedmiller et al. *Learning by Playing - Solving Sparse Reward Tasks from Scratch*. arXiv preprint arXiv:1802.10567. 2018. arXiv: 1802.10567.

[24] John Schulman et al. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". In: *arXiv preprint arXiv:1506.02438* (2018). arXiv: 1506.02438 [cs.LG].

[25] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *CoRR* abs/1707.06347 (2017). arXiv: 1707.06347.

[26] John Schulman et al. “Trust Region Policy Optimization”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. PMLR, 2015, pp. 1889–1897.

[27] David Silver et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587 (2016), pp. 484–489.

[28] David Silver et al. “Reward is enough”. In: *Artificial Intelligence* 299 (2021), p. 103535.

[29] Petru Soviany et al. “Curriculum Learning: A Survey”. In: *International Journal of Computer Vision* 130.6 (June 2022), pp. 1526–1565. ISSN: 1573-1405. DOI: 10.1007/s11263-022-01611-x. URL: https://doi.org/10.1007/s11263-022-01611-x.

[30] Sutco RecyclingTechnik GmbH. *High-Throughput Plastic Sorting Facility Diagram*. Website. Accessed: 2025-09-02. Available at: http://www.sutco.com/de. Sept. 2025.

[31] Sutco RecyclingTechnik GmbH. *REFERENCE PLANT Gernsheim: Light packaging sorting plant*. Corporate Brochure. Sutco RecyclingTechnik GmbH. URL: https://www.sutco.com/fileadmin/sutco.com/media/pdf/brochures-references-en/sutco-reference-plant-gernsheim-en.pdf (visited on 08/30/2025).

[32] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[33] Mark Towers et al. *Gymnasium*. urlhttps://zenodo.org/record/8127025. Version 0.29.1. 2023.

[34] Oriol Vinyals et al. “Grandmaster level in StarCraft II using multi-agent reinforcement learning”. In: *Nature* 575.7782 (Oct. 2019), pp. 350–354.